

A. TRANG BÌA

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Đề tài:

XÂY DỰNG HỆ THỐNG NHẬN DIỆN BIỂN SỐ XE BẰNG TRÍ TUỆ NHÂN TẠO

Developing an AI-based vehicle license plate recognition system

Ngành: Trí tuệ nhân tạo

Chuyên ngành: Trí tuệ nhân tạo

Sinh viên thực hiện: **Phạm Công Thành** — MSSV **25410013**

Nguyễn Minh Hiếu — MSSV **25410007**

Lớp: AI503.F3.LT.TTNT

Khoá: 2025

Giảng viên hướng dẫn: ThS. Cáp Phạm Đình Thăng

TP. Hồ Chí Minh, tháng 9 năm 2026

B. MỤC LỤC

A. TRANG BÌA	1
ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC	1
XÂY DỰNG HỆ THỐNG NHẬN DIỆN BIỂN SỐ XE BẰNG TRÍ TUỆ NHÂN TẠO.....	1
B. MỤC LỤC	1

C. TÓM TẮT ĐỒ ÁN.....	3
CHƯƠNG 1. GIỚI THIỆU	5
1.1. Đặt vấn đề	5
1.2. Mục tiêu đề tài	6
1.3. Đối tượng và phạm vi nghiên cứu	8
1.4. Phương pháp nghiên cứu	10
1.5. Đóng góp của đề tài	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	12
2.1. Phạm vi và bố cục cơ sở lý thuyết	12
2.2. Quy chuẩn biển số xe Việt Nam	12
2.3. Cơ sở lý thuyết về phát hiện đối tượng	15
2.4. Cơ sở lý thuyết về nhận dạng ký tự	17
2.5. Các công trình liên quan	20
CHƯƠNG 3. KHẢO SÁT CÔNG NGHỆ VÀ LỰA CHỌN MÔ HÌNH	22
3.1. Phương pháp khảo sát và tiêu chí lựa chọn	22
3.2. Mô hình phát hiện: YOLO11	23
3.3. Engine nhận dạng ký tự	23
3.4. Runtime suy luận trên CPU: ONNX Runtime	25
3.5. Các lựa chọn công nghệ nền tảng khác	26
3.6. Độ phân giải đầu vào: 640 thay vì 416	26
CHƯƠNG 4. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG	28
4.1. Phân tích yêu cầu	28
4.2. Kiến trúc hệ thống	29
4.3. Môi trường và công cụ phát triển	32
4.4. Xây dựng bộ dữ liệu	32
4.5. Huấn luyện mô hình	34
4.6. Tầng AI — thiết kế và cài đặt	36
4.7. Backend và cơ sở dữ liệu	41
4.8. Giao diện người dùng	42
4.9. Triển khai bằng Docker	43
4.10. Những chỗ cài đặt lệch khỏi thiết kế, và lý do	44
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ	45
5.1. Mục tiêu và phương pháp đánh giá	45

5.2. Môi trường thực nghiệm.....	47
5.3. Bộ dữ liệu thực nghiệm.....	47
5.4. Đánh giá bộ phát hiện biển số	49
5.5. Đánh giá khối OCR và hậu xử lý	50
5.6. Đánh giá hiệu năng.....	53
5.7. Đối chiếu toàn bộ chỉ tiêu phi chức năng.....	56
5.8. Phân tích lỗi.....	57
5.9. Bàn luận	58
5.10. Đối chiếu với các công trình đã công bố	59
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
6.1. Kết quả đạt được.....	61
6.2. Hạn chế.....	62
6.3. Hướng phát triển	62
6.4. Kết luận chung	63
TÀI LIỆU THAM KHẢO	64
PHỤ LỤC	76
Phụ lục A. Mã nguồn	76
Phụ lục B. Siêu tham số huấn luyện	77
Phụ lục C. Bộ dữ liệu.....	78
Phụ lục D. Hướng dẫn cài đặt và chạy	79
Phụ lục E. Kết quả kiểm thử.....	81
Phụ lục F. Giao diện lập trình và cấu hình triển khai	82
Phụ lục H. Đặc tả yêu cầu và thiết kế dữ liệu	83
Phụ lục O. Tệp cấu hình gốc, báo cáo đo và mã nguồn.....	86

C. TÓM TẮT ĐỀ ÁN

TÓM TẮT ĐỀ ÁN

Nhận dạng biển số xe tự động (ALPR) là bài toán nền tảng của bãi đỗ xe thông minh, thu phí không dừng và giám sát giao thông. Tại Việt Nam, biển số hai dòng chiếm tỷ lệ lớn do mật độ xe máy cao, trong khi đa số bộ dữ liệu quốc tế chỉ có biển một dòng, khiến các mô hình huấn luyện trên dữ liệu nước ngoài không áp dụng trực tiếp được. Khác biệt này đã được đo

lường: trên bộ RodoSol-ALPR của Brazil, hệ thống OpenALPR đạt 94,3% với ô tô biển một dòng nhưng chỉ 45,7% với xe máy biển hai dòng [1].

Đồ án xây dựng một hệ thống ALPR hoàn chỉnh cho biển số xe Việt Nam theo hướng tiếp cận hai giai đoạn: phát hiện vùng biển số bằng YOLO11, nhận dạng ký tự bằng PaddleOCR, và hậu xử lý bằng bộ luật chuẩn hoá theo quy chuẩn hiện hành. Hệ thống hỗ trợ cả biển một dòng và hai dòng, nhận đầu vào là ảnh, video hoặc khung hình thời gian thực gửi qua API (endpoint `POST /api/detect/frame`), và suy luận hoàn toàn trên CPU.

Đóng góp chính là bộ luật hậu xử lý **ràng buộc theo vị trí ký tự**, xây dựng trên Thông tư 79/2024/TT-BCA [2] và QCVN 08:2024/BCA [3]: mã tỉnh thuộc 81 giá trị hợp lệ, chữ cái sê-ri thứ nhất và thứ hai thuộc hai tập ký tự khác nhau. Cách tiếp cận này khắc phục hạn chế của các hệ thống áp một danh sách ký tự phẳng cho toàn chuỗi.

Về mặt kỹ nghệ, đồ án cài đặt kiến trúc phân tầng tách biệt tầng AI khỏi tầng API, gồm backend FastAPI, giao diện web React và đóng gói Docker.

Trên tập kiểm tra của split v3 (1.514 ảnh, đã khử trùng lặp giữa các tập), bộ phát hiện YOLO11n đạt $mAP@0.5 = 0,9829$ và $mAP@0.5:0.95 = 0,7834$ (precision 0,9837; recall 0,9714). Khối nhận dạng đạt độ chính xác mức ký tự ($1 - CER$) 0,9454; độ chính xác toàn chuỗi tăng từ 0,6373 lên 0,7512 nhờ bộ luật hậu xử lý (sửa đúng 319 biển, không làm hỏng biển nào), và độ chính xác end-to-end đạt 0,5552. Khoảng cách lớn nhất nằm ở layout: biển một dòng đạt 0,9541 còn biển hai dòng chỉ đạt 0,6996 — chênh 25,45 điểm phần trăm, trong khi biển hai dòng chiếm 79,8% tập đánh giá. Độ trễ xử lý một ảnh ở phân vị 95 là 1.143,10 ms trên CPU (trung vị 405,77 ms), đạt ngưỡng tối thiểu 1.500 ms nhưng chưa đạt mục tiêu 800 ms — cái giá đã định lượng của bậc thang thử-lại dành cho biển nghiêng, méo. Chi tiết và phân tích lỗi được trình bày ở Chương 5.

Từ khoá: nhận dạng biển số xe, biển số Việt Nam, YOLO11, PaddleOCR, biển số hai dòng, hậu xử lý theo vị trí, suy luận trên CPU.

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

1.1.1. Bối cảnh giao thông Việt Nam và nhu cầu tự động hoá

Tính đến tháng 9/2024, Việt Nam có khoảng **77 triệu xe máy đã đăng ký, tương đương 770 xe trên 1.000 dân**; xe máy chiếm khoảng **85–90% lưu lượng phương tiện trên đường** [1]. Con số đó là **ràng buộc kỹ thuật trực tiếp**, kéo theo ba hệ quả (Chương 4): (i) **biển hai dòng gần vuông chiếm đa số tuyệt đối** vì mọi xe mô tô đều mang biển hai dòng; (ii) **mật độ cao gây che khuất**, mỗi khung hình thường có **nhiều biển số**; (iii) biển xe mô tô chỉ 140 × 190 mm nên đây là **bài toán phát hiện đối tượng nhỏ**. Ở quy mô đó, ghi nhận thủ công không khả thi — lý do tồn tại của **ALPR (Automatic License Plate Recognition)** [2] [3].

1.1.2. Ứng dụng thực tế của hệ thống ALPR

ALPR là lõi của bốn nhóm ứng dụng tại Việt Nam: **bãi đỗ xe thông minh** [4]; **thu phí không dừng ETC** [5]; **giám sát giao thông** (xử phạt nguội); **kiểm soát ra vào** [6]. Cả bốn đo cùng một thứ: **không phải mAP của khâu phát hiện, mà là tỉ lệ đọc đúng toàn bộ chuỗi biển số** (plate-level exact match) — đọc đúng 9 trên 10 ký tự vẫn là đọc sai biển. Nhận định này định hình chỉ tiêu ở mục 1.2.2 và cách đánh giá ở Chương 5.

1.1.3. Vì sao không thể dùng trực tiếp giải pháp nước ngoài

(a) Biển hai dòng là điểm gây đã đo được, không phải rủi ro giả định. Trên tập kiểm thử cân bằng có chủ ý của bộ **RodoSol-ALPR (Brazil)** — 4.000 ảnh ô tô biển **một dòng**, 4.000 ảnh xe máy biển **hai dòng** — hệ thống thương mại **OpenALPR** nhận đúng **94,3% biển một dòng nhưng chỉ 45,7% biển hai dòng, chênh 48,6 điểm phần trăm**, không biển số nào khác thay đổi ngoài bố cục biển [7]. Cũng nghiên cứu này: cả 12 phương pháp và 2 hệ thống thương mại đều **không vượt quá 70% recognition rate**, và có công trình phải **loại bỏ hoàn toàn xe máy** khỏi thí nghiệm [7].

⚠ **Cảnh báo phạm vi áp dụng của số liệu.** Cặp **94,3% / 45,7%** (chênh **48,6 điểm phần trăm**) đo trên **RodoSol-ALPR của Brazil, không phải trên dữ liệu Việt Nam**; dẫn như một *analogue* định lượng về độ khó của biển hai dòng, chọn Brazil vì cũng là nước có tỉ lệ xe máy cao. **Tuyệt đối không được trình bày cặp số này như số liệu Việt Nam** — số liệu Việt Nam do chính đồ án đo nằm ở **Chương 5**.

(b) Căn cứ pháp lý và cấu trúc chuỗi ký tự là đặc thù quốc gia. Biển số xe Việt Nam theo **TT 79/2024/TT-BCA** (hiệu lực 01/01/2025) [8], sửa đổi bởi **TT 13/2025/TT-BCA** [9] và **TT 51/2025/TT-BCA** [10]; kích thước vật lý theo **QCVN 08:2024/BCA** [11]. **Đính chính:** nhiều tài liệu trong nước vẫn viện dẫn **TT 24/2023/TT-BCA** [12] — đã hết hiệu lực từ **01/01/2025**. Ba đặc thù sau **không học được từ dữ liệu nước ngoài**. (i) **Tập ký tự seri phụ thuộc vị trí:** seri **vị trí thứ nhất** thuộc tập **20 chữ cái** (có G, không có R) [13]; **vị trí thứ hai** của biển xe mô tô thuộc tập **20 chữ cái khác** (có R, không có G); tập loại trừ đúng cho toàn hệ thống chỉ gồm **5 chữ I, J, O, Q, W** — charset theo "danh sách phẳng 20 chữ cái" sẽ **sai hệ thống trên**

toàn bộ lớp biển xe máy có R ở vị trí thứ hai, lỗi không cứu được ở hậu xử lý. (ii) Mã địa phương hữu hạn, có lỗ hổng: dải 11–99 chỉ có 81 mã đang được sử dụng; 8 mã — 13, 42, 44, 45, 46, 87, 91, 96 — không được gán [14], nên \d{2} cho qua 8 chuỗi không bao giờ tồn tại. (iii) Tỷ lệ khung hình phân tách rõ hai bố cục [11]: 110 × 520 mm → 4,727 (1 dòng); 165 × 330 mm → 2,000 (2 dòng); 140 × 190 mm → 1,357 (2 dòng); không loại biển nào rơi vào khoảng mở (2,000 ; 4,727) — cơ sở hình học để phân loại số dòng.

(c) Điều kiện thu nhận ảnh khác biệt: biển bị che, bám bụi, cong vênh, chụp nghiêng, ngược sáng, ảnh đêm — khác các bộ dữ liệu quốc tế lớn (CCPD, AOLP, SSIG) vốn lấy ô tô làm trung tâm. Kết luận mục 1.1: bài toán này cần hệ thống huấn luyện trên dữ liệu Việt Nam và khối hậu xử lý xây theo quy chuẩn Việt Nam.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu tổng quát

Xây dựng một hệ thống nhận dạng biển số xe Việt Nam hoàn chỉnh, chất lượng gần sản phẩm thực tế: mô hình phát hiện tự huấn luyện trên dữ liệu Việt Nam, khối nhận dạng ký tự, khối hậu xử lý theo quy chuẩn Việt Nam, backend REST API, giao diện web, cơ sở dữ liệu lịch sử, đóng gói triển khai, tài liệu học thuật đầy đủ; hỗ trợ cả biển một dòng và hai dòng, suy luận hoàn toàn trên CPU. Đây không phải demo dạng notebook, cũng không phải sản phẩm thương mại.

1.2.2. Mục tiêu cụ thể

Mỗi chỉ tiêu có mục tiêu và ngưỡng tối thiểu (bắt buộc đạt). Về chức năng và chất lượng phần mềm: 34 yêu cầu chức năng (21 Must, 6 Should, 3 Could, 4 Won't) trong sáu nhóm (mục 4.1.3); NFR-M1 mã pipeline AI không import FastAPI; NFR-M5 thay được bộ OCR không sửa mã tầng API; NFR-M2 độ bao phủ kiểm thử tầng nghiệp vụ ≥ 70%; khởi động một lệnh docker compose up, demo không cần Internet. NFR-A5 và NFR-A6 đo tách bạch có chủ đích — hiệu số là đóng góp định lượng của khối hậu xử lý (mục 1.5); thêm NFR-A8 (tách riêng biển một dòng / hai dòng) và NFR-A9 (theo điều kiện ảnh, nếu có nhân phù hợp).

Bảng 1.1. Nhóm chỉ tiêu độ chính xác

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-A1	mAP@0.5 của bộ phát hiện biển số	≥ 0,90	≥ 0,85
NFR-A2	mAP@0.5:0.95 của bộ phát hiện	≥ 0,65	≥ 0,55
NFR-A3	Precision / Recall phát hiện	≥ 0,92 / ≥ 0,90	≥ 0,88 / ≥ 0,85
NFR-	Độ chính xác OCR mức ký tự (1 – CER)	≥ 0,95	≥ 0,92

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
A4			
NFR-A5	Độ chính xác biến đầy đủ trước hậu xử lý	$\geq 0,85$	$\geq 0,80$
NFR-A6	Độ chính xác biến đầy đủ sau hậu xử lý	$\geq 0,90$	$\geq 0,85$
NFR-A7	Độ chính xác E2E toàn trình (ảnh vào → biến đúng)	$\geq 0,88$	$\geq 0,82$

Bảng 1.2. Nhóm chỉ tiêu hiệu năng trên CPU

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-P1	Độ trễ E2E một ảnh (p95)	$\leq 800\text{ ms}$	$\leq 1500\text{ ms}$
NFR-P2	Tốc độ khung hình thời gian thực (webcam — đo ở tầng API)	$\geq 5\text{ FPS}$ hiệu dụng	$\geq 3\text{ FPS}$
NFR-P3	Tốc độ xử lý video	$\geq 0,3\times$ thời gian thực	$\geq 0,15\times$
NFR-P4	Thời gian nạp mô hình khi khởi động	$\leq 15\text{ s}$	$\leq 30\text{ s}$
NFR-P5	Overhead của tầng API (không tính suy luận)	$\leq 50\text{ ms}$	$\leq 100\text{ ms}$
NFR-P6	Thời gian truy vấn lịch sử (10.000 bản ghi)	$\leq 500\text{ ms}$	$\leq 1000\text{ ms}$
NFR-P7	Bộ nhớ thường trú của backend	$\leq 2\text{ GB}$	$\leq 4\text{ GB}$

Các chỉ tiêu độ trễ "rộng rãi" hơn bài báo ALPR vì máy phát triển **không có GPU CUDA** (CON-02): huấn luyện trên GPU Colab/Kaggle, **toàn bộ suy luận và demo chạy trên CPU**, còn bài báo thường đo trên GPU RTX/V100. Mọi số liệu hiệu năng **bắt buộc kèm cấu hình phần cứng**.

⚠ **Bốn yêu cầu mức *Won't* — phải nói thẳng.** Cả bốn đều **thuần giao diện**, chuyển mức trong hai đợt thu gọn giao diện web ngày **2026-07-20**: đợt 1 gỡ trang Webcam → **FR-3.1 và FR-3.4 chuyển M → W** (nhận dạng thời gian thực vẫn phục vụ và vẫn có kiểm thử ở tầng API qua POST /api/detect/frame); đợt 2 gỡ trang Tổng quan (Dashboard) → **FR-4.1 chuyển M → W, FR-4.2 chuyển S → W** (thống kê và biểu đồ theo thời gian vẫn truy vấn được và vẫn có kiểm thử tích hợp qua GET /api/statistics, GET /health). **FR-4.1 là yêu cầu mức *Must* đầu tiên và duy nhất bị đưa ra khỏi phạm vi trong toàn bộ đề án** — nêu ở đây, ở mục 4.1.3, mục

6.3 và trong đặc tả yêu cầu, không để hội đồng tự phát hiện. Đây là **quyết định phạm vi có chủ đích**, không phải hạng mục bỏ sót: cả bốn mất **màn hình hiển thị**, không mất **năng lực hệ thống**, mã giao diện còn nguyên trong lịch sử git. Đánh đổi do được của đợt 2: gỡ thư viện biểu đồ recharts cùng trang Tổng quan làm gói tải về của giao diện giảm từ ~730 KB xuống **328,8 KB** (-55%).

1.2.3. Tiêu chí thành công

Đề tài thành công khi **đồng thời**: (1) toàn bộ yêu cầu *Must* hoạt động và demo được — theo bộ 21 *Must* sau hai đợt thu gọn 2026-07-20, bốn yêu cầu đã chuyển *Won't* (FR-3.1, FR-3.4, FR-4.1, FR-4.2) **không** tính là đạt; (2) mọi chỉ tiêu đạt **ngưỡng tối thiểu** và **có bằng chứng đo đạc**; (3) đủ 12 sản phẩm bàn giao; (4) khởi động trên máy sạch bằng một lệnh docker compose up; (5) demo trực tiếp không cần Internet.

Trạng thái tại thời điểm viết chương này. Các chỉ tiêu ở mục 1.2.2 là **chỉ tiêu đặt ra**. Hệ thống chạy pipeline nhận dạng **thật** với models/best.pt (imgsz=640, split v3); chỉ tiêu phát hiện **đều đạt** ($mAP@0.5 = 0,9829$), độ trễ NFR-P1 **đạt** (p95 731/780 ms); chỉ tiêu độ chính xác OCR **đã đo** và biểu hai dòng **chưa đạt** (kết quả thật). **Đối chiếu đầy đủ từng chỉ tiêu ở Chương 5.**

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

Ba nhóm. **(1) Biển số xe cơ giới Việt Nam** theo TT 79/2024/TT-BCA [8] (sửa đổi bởi TT 13/2025 [9] và TT 51/2025 [10]) và QCVN 08:2024/BCA [11]; biển nền đỏ quân đội thuộc TT 169/2021/TT-BQP [15], chỉ xử lý ở mức nhận biết ký hiệu. **(2) Mô hình phát hiện họ YOLO** — cụ thể YOLO11 [16]. **(3) Engine OCR** không cần phân đoạn ký tự: PaddleOCR [17] là **baseline**, EasyOCR ngang hàng, Tesseract là mốc dưới, kèm bộ luật hậu xử lý theo vị trí. **Lựa chọn engine OCR chưa chốt ở giai đoạn thiết kế** — quyết định thuộc về benchmark do chính đồ án chạy (**mục 3.3, Chương 5**).

1.3.2. Phạm vi trong nghiên cứu

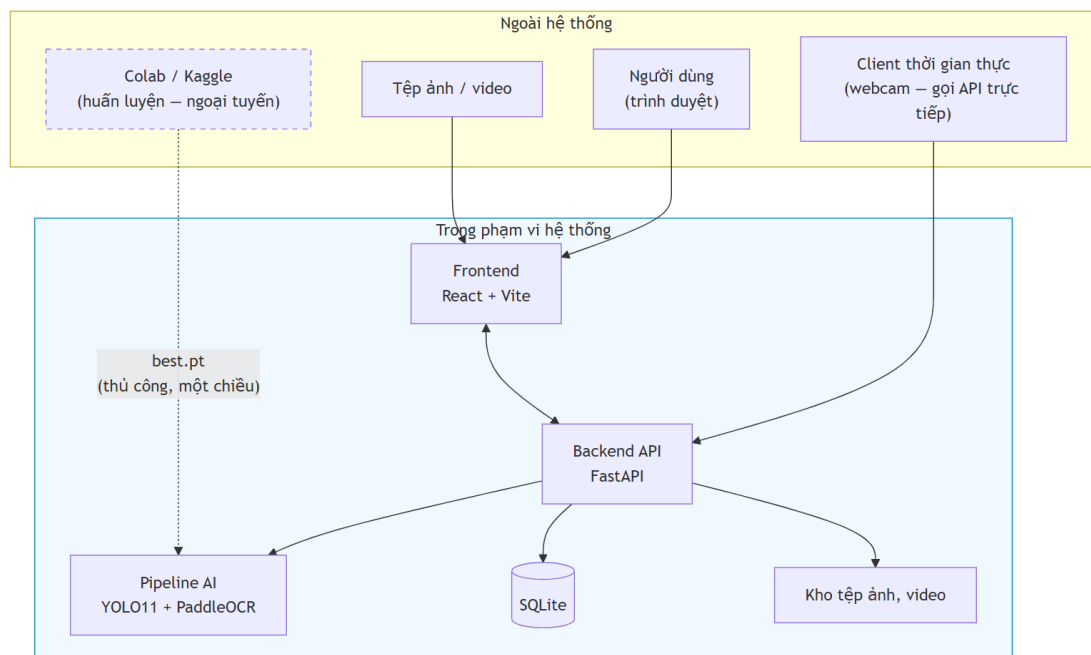
Bốn nhóm. **(a) Trí tuệ nhân tạo**: huấn luyện YOLO11 trên dữ liệu Việt Nam, so sánh biến thể n / s / m, kèm YOLO26n đối chứng (mục 3.2); **benchmark các engine OCR** trên chính tập kiểm thử biển số Việt Nam rồi tích hợp engine thắng cuộc; hậu xử lý theo luật hợp lệ theo vị trí; **hỗ trợ cả biển một dòng và hai dòng**; đánh giá đầy đủ (mAP, precision, recall, F1, ma trận nhầm lẫn); đo hiệu năng trên CPU. **(b) Dữ liệu**: thu thập, gộp, làm sạch bộ công khai; sửa nhãn; loại ảnh trùng lặp; tăng cường; chia train / val / test **có kiểm soát rò rỉ dữ liệu**; thống kê. **(c) Phần mềm**: FastAPI + Swagger; nhận dạng ảnh, video và khung hình thời gian thực qua POST /api/detect/frame; lịch sử bằng SQLite + SQLAlchemy + Alembic; giao diện React + Vite + TypeScript + TailwindCSS **ba trang** (Nhận dạng ảnh — trang chủ, Nhận dạng video, Lịch sử) sau hai đợt thu gọn 2026-07-20; tìm kiếm, lọc, tải về; thống kê ở tầng API (GET /api/statistics); Docker và Docker Compose. **(d) Kiểm thử, tài liệu**: unit

test, integration test, kiểm thử độ chính xác AI, hiệu năng, chịu tải; tài liệu học thuật và kỹ thuật.

1.3.3. Phạm vi ngoài nghiên cứu

Danh sách này là **hàng rào trước câu hỏi "sao không làm X"**; không hạng mục nào bị loại vì "không kịp làm". **Mười một hạng mục:** (1) **xác thực, phân quyền** — chạy nội bộ localhost/LAN (giả định A-04); (2) **đa camera / đa luồng**; (3) **bấm vết qua khung hình (SORT / DeepSORT)** — thay bằng **gộp trùng theo chuỗi ký tự**; (4) **phân loại loại xe** — từ 01/01/2025 TT 79/2024 **đã bỏ** quy tắc suy loại xe từ chữ cái seri; (5) **ước lượng tốc độ, phát hiện vi phạm** — cần hiệu chuẩn camera riêng; (6) **biển số nước ngoài**; (7) **barie / cổng tự động** — cần thiết bị vật lý; (8) **cloud, multi-tenant, CI/CD production** — **Docker Compose đã đủ**; (9) **ứng dụng di động** — web responsive đã đáp ứng; (10) **huấn luyện engine OCR từ đầu** — dùng pre-trained rồi **tinh chỉnh**; tinh chỉnh nằm **trong** phạm vi (mục 2.4.3(f)); (11) **suy luận thời gian thực trên GPU** — máy phát triển **không có GPU CUDA** (CON-02) $\Rightarrow$ mọi số liệu là **số liệu CPU**.

1.3.4. Ranh giới hệ thống



Hình 1.1. Ranh giới hệ thống — phần bên trong là hệ thống bàn giao, Colab/Kaggle nằm ngoài

Colab/Kaggle nằm **ngoài** ranh giới khi vận hành — chỉ là công cụ ngoại tuyến sản xuất best.pt; hệ thống khi chạy **không phụ thuộc dịch vụ ngoài nào** (mục 1.2.3). Sau khi trang webcam bị gỡ (2026-07-20), client gửi khung hình trực tiếp qua POST /api/detect/frame.

1.4. Phương pháp nghiên cứu

Đề tài dùng **ba phương pháp bổ trợ nhau**: nghiên cứu lý thuyết (khảo sát tài liệu có trích dẫn, đối chiếu văn bản pháp quy hiện hành); nghiên cứu thực nghiệm (**mọi khẳng định về hiệu năng và độ chính xác đều phải có số đo tái lập được**, kèm cấu hình phần cứng và cỡ mẫu); và quy trình phát triển theo giai đoạn, mỗi giai đoạn khép lại bằng một bộ tài liệu và một mốc kiểm chứng.

1.5. Đóng góp của đề tài

1.5.1. Tuyên bố trung thực về mức đóng góp

Đề tài này không tạo ra kết quả state-of-the-art.

Các con số vượt 99% trong tài liệu ALPR quốc tế đến từ nhóm nghiên cứu chuyên nghiệp có hạ tầng GPU lớn và dữ liệu độc quyền. Một đồ án làm trên máy không có GPU CUDA **không đặt mục tiêu đó** — tuyên bố ngược lại là thiếu trung thực học thuật. Đóng góp thực sự nằm ở sáu chỗ, cụ thể và kiểm chứng được.

Sáu đóng góp.

(a) Hệ thống hoàn chỉnh, có kiến trúc phần mềm — không phải tập script rời rạc: pipeline AI tách hoàn toàn khỏi tầng API (NFR-M1), interface trừu tượng cho phép thay engine OCR mà không sửa tầng API (NFR-M5), REST API có tài liệu tự sinh, giao diện web ba màn hình, cơ sở dữ liệu có migration, Docker một lệnh. Trạng thái đã đo: bao phủ kiểm thử tầng nghiệp vụ **87,7%**, **1.001 test thu thập / 1.000 đạt / 1 xfail / 0 thất bại**. Khảo sát cho thấy mã nguồn mở ALPR Việt Nam chủ yếu là script rời rạc **không công bố số liệu độ chính xác** — đây là **khoảng trống kỹ nghệ**, không phải khoảng trống thuật toán, nhưng vẫn có thật.

(b) Bộ luật hậu xử lý ràng buộc theo VỊ TRÍ cho biển số Việt Nam, khai thác ba ràng buộc đặc thù: tập hợp lệ **khác nhau theo từng vị trí** — mã địa phương thuộc **81 giá trị** chứ không phải $\{2\}$ [14], seri **thứ nhất** thuộc 20 chữ cái có G không có R [13], seri **thứ hai** của biển xe mô tô thuộc **20 chữ cái KHÁC** có R không có G; cấu trúc chuỗi và độ dài theo quy chuẩn; và bảng ánh xạ nhầm lẫn ký tự **không đối xứng**.

⚠ **Nói thẳng về độ lớn của đóng góp này.** Luận điểm dự kiến ban đầu — "*khai thác bộ 20 chữ cái, loại trừ 6 chữ I J O Q R W*" — **sai và đã bị bác bỏ**: tập loại trừ toàn hệ thống chỉ có **5 chữ**, còn R **hợp lệ** ở vị trí seri thứ hai của biển xe mô tô. Sửa lại **làm yếu đi** phần đóng góp nếu tính theo "số ký tự loại trừ được". Đổi lại, phần có giá trị nằm ở ràng buộc **phụ thuộc vị trí**: một hệ thống dùng danh sách phẳng 20 chữ cái sẽ sai hệ thống trên mọi biển xe máy có R.

(c) Đo được ĐỊNH LƯỢNG đóng góp của bước hậu xử lý. Phần lớn công trình mô tả bước này ở mức định tính, không trả lời được *nó đóng góp bao nhiêu*. Đề tài giải quyết ở tầng dữ liệu — **lưu đồng thời chuỗi OCR thô và chuỗi đã sửa** — nên hiệu số giữa **NFR-A5** (trước) và **NFR-A6** (sau) là một **con số đo được**.

(d) Đánh giá tách riêng biển một dòng và biển hai dòng. NFR-A8 biến phép tách này thành **nghĩa vụ báo cáo bắt buộc** chứ không phải phân tích tùy chọn, kèm **NFR-A9** — báo cáo theo điều kiện ảnh, nếu bộ dữ liệu có nhãn phù hợp.

(e) Công bố hiệu năng kèm cấu hình phần cứng CPU cụ thể — mọi số liệu kèm model CPU, số luồng, kích thước ảnh đầu vào, backend suy luận và cỡ mẫu đo. FPS không kèm phần cứng thì không tái lập được, cũng không so sánh được.

(f) Đo trên chính ảnh biển số Việt Nam. Khảo sát xác định **không tồn tại benchmark công khai nào so sánh các engine OCR trên riêng ảnh biển số xe máy Việt Nam hai dòng**, và hai số liệu thường được viện dẫn để chứng minh ưu thế của một engine đã **bị bác bỏ khi truy ngược về nguồn gốc** (mục 3.3). Đề tài đã chạy ba phép so sánh trên chính ảnh biển số Việt Nam, cùng máy và cùng ngữ liệu: **PP-OCRv5_mobile ↔ PP-OCRv6_medium** (mục 3.3.2), **bộ nhận dạng gốc ↔ bản tinh chỉnh** (mục 4.5.3), và **PaddleOCR ↔ EasyOCR ↔ Tesseract trên toàn bộ 2.801 biển** (mục 3.3.3) — kết quả PaddleOCR **68,87%**, hơn EasyOCR 54,59 điểm và hơn Tesseract 58,59 điểm, **bác bỏ** tài liệu công khai vốn nghiêng về EasyOCR. Một kết quả **trái kỳ vọng**: bước tách-rời-ghép-ngang mua **34,92 điểm** cho PaddleOCR nhưng chỉ **0,03 điểm** cho Tesseract, nên nó **không** phải kỹ thuật độc lập engine như đã kỳ vọng.

1.5.2. Những gì đề tài KHÔNG tuyên bố

Bốn điều loại trừ. **Không** tuyên bố vượt các con số độ chính xác cao nhất trong nước — chúng đo trên tập dữ liệu riêng không công khai, **không có cơ sở so sánh công bằng**. **Không** đề xuất kiến trúc mạng nơ-ron mới; đề tài **tích hợp và tinh chỉnh**. **Không** giải quyết các thách thức mở — độ phân giải rất thấp, tổng quát hoá xuyên tập dữ liệu, che khuất nặng (**Hướng phát triển, Chương 6**). Mọi số liệu hiệu năng là **số liệu CPU**, **không so sánh trực tiếp được** với FPS đo trên GPU.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương đặt nền lý thuyết và tư liệu cho phần thiết kế. Nguyên tắc xuyên suốt: **mọi con số gắn nguồn tại chỗ, mọi cảnh báo về phạm vi áp dụng giữ nguyên** — lĩnh vực này hay công bố số trên 99% nhưng đo trên tập dữ liệu và giao thức rất khác nhau.

2.1. Phạm vi và bố cục cơ sở lý thuyết

Một hệ thống ALPR gồm bốn khối nối tiếp — **phát hiện vùng biển, nắn chỉnh và tiền xử lý, nhận dạng ký tự, hậu xử lý theo quy chuẩn** — và độ chính xác cuối cùng là **tích** của độ chính xác từng khối, nên một khối yếu kéo cả chuỗi xuống. Đề án đi theo hướng **two-stage** (phát hiện rồi nhận dạng riêng) kết hợp bộ nhận dạng **segmentation-free**; căn cứ của lựa chọn đó trình bày ở Chương 3.

Chương này chỉ giữ phần lý thuyết **ràng buộc trực tiếp một quyết định của hệ thống**: quy chuẩn biển số Việt Nam (2.2) — cơ sở của bộ luật hậu xử lý; kiến trúc YOLO11 và các chỉ số đánh giá khối phát hiện (2.3); kiến trúc CRNN/CTC cùng **điểm gãy của nó trên văn bản nhiều dòng** (2.4) — nền tảng lý thuyết của rủi ro R-04 và của đóng góp kỹ thuật lõi; và khảo sát công trình liên quan cùng sáu khoảng trống nghiên cứu (2.5).

2.2. Quy chuẩn biển số xe Việt Nam

2.2.1. Căn cứ pháp lý hiện hành

Cảnh báo văn bản hết hiệu lực. Nhiều tài liệu, kể cả bài báo 2023 – 2024, vẫn viện dẫn **Thông tư 24/2023/TT-BCA** — đã hết hiệu lực từ **01/01/2025** [12]; đề án chỉ nhắc như bối cảnh lịch sử.

Bốn văn bản căn cứ: **TT 79/2024/TT-BCA** hiệu lực 01/01/2025, thay TT 24/2023, quy định cấu trúc biển, seri, màu sắc [8]; **TT 13/2025/TT-BCA** sửa đổi TT 79/2024 [9]; **TT 51/2025/TT-BCA** hiệu lực 01/7/2025, **thay toàn bộ Phụ lục mã tỉnh** sau sáp nhập còn 34 tỉnh/thành [10]; **QCVN 08:2024/BCA** kèm TT 81/2024/TT-BCA, hiệu lực 01/01/2025, quy chuẩn quốc gia về kết cấu, kích thước, vật liệu [11]. Biển quân đội thuộc TT 169/2021/TT-BQP [15], ngoài phạm vi TT 79/2024.

TT 79/2024 quy định **nội dung** biển — cơ sở biểu thức chính quy; QCVN 08:2024/BCA quy định **hình thức vật lý** — cơ sở ngưỡng tỷ lệ khung hình; module chuẩn hoá cần cả hai. Khung pháp lý đổi **ba lần trong hai năm** là rủi ro kỹ thuật trực tiếp [19]; hệ quả ở mục 2.2.7.

2.2.2. Cấu trúc biển số ô tô và xe máy

a) Biển số ô tô trong nước: **8 ký tự chữ–số**, ba thành phần — **mã địa phương** 2 chữ số trong 81 mã hợp lệ thuộc dải 11 – 99 [10], [14]; **seri** 1 chữ cái trong 20 chữ với biển trắng và vàng, 11 chữ với biển xanh [13]; **số thứ tự** 5 chữ số, 000.01 – 999.99 [8] — ví dụ 30A-123.45, 51K-999.99, 80B-123.45 (Cục CSGT). Trên đường vẫn còn **biển 4 chữ số kiểu cũ** (29A-

1234); xe đã đăng ký **không bắt buộc đổi biển** [26] nên biểu thức chính quy phải chấp nhận nhóm thứ tự **4 hoặc 5 chữ số**.

2.2.3. Mã tỉnh, thành phố

Từ 01/7/2025 cả nước còn **34 tỉnh, thành phố**; ký hiệu sau hợp nhất **bao gồm toàn bộ ký hiệu của các địa phương được hợp nhất** [26], biển cũ không mất giá trị pháp lý. Dải 11 – 99 có **89 số**; theo Phụ lục TT 51/2025 có **81 mã đang dùng** (80 mã địa phương + mã 80 của Cục CSGT) và **8 mã không dùng: 13, 42, 44, 45, 46, 87, 91, 96** [14]. TP. Hồ Chí Minh 13 mã (41; 50 – 59; 61; 72); Hà Nội 6 mã (29; 30 – 33; 40) [14].

Kiểm tra mã tỉnh loại khoảng 9,0% không gian tìm kiếm ở hai ký tự đầu, và quan trọng hơn: biển lỗi OCR hai vị trí đầu từ **sai âm thầm** thành **sai phát hiện được** — đọc ra 46A-123.45 thì biết ngay mã 46 không tồn tại và hạ cờ hợp lệ. Giả thuyết mã 13 là mã cũ của Hà Bắc **chưa kiểm chứng được nguồn chính thức**, chỉ nêu tham khảo.

2.2.4. Tập ký tự seri và các chữ cái bị loại trừ

Mục dễ bị trình bày sai nhất của chương. Biển trắng và vàng chữ đen dùng seri **một trong 20 chữ cái** [13]; đối chiếu 26 chữ Latin thì vắng I J O Q R W — nhưng **suy diễn "26 – 20 = 6 chữ bị loại trừ" là SAI**: danh sách 20 chữ **chỉ áp dụng cho chữ cái thứ nhất**; ở vị trí thứ hai của seri xe máy là tập khác — **có R, không có G**. Hợp hai vị trí, tập chữ không bao giờ xuất hiện trên biển Việt Nam chỉ gồm **5 chữ: I, J, O, Q, W**; chữ R còn ở ký hiệu đặc biệt R, RM của rơ moóc [29].

Bảng 2.1. Tổng hợp các tập ký tự seri

Tập	Số lượng	Nội dung
Chữ cái thứ nhất của seri	20	A B C D E F G H K L M N P S T U V X Y Z
Chữ cái thứ hai của seri xe máy	20	A B C D E F H K L M N P R S T U V X Y Z
Seri biển nền xanh	11	A B C D E F G H K L M
Bị loại trừ khỏi toàn hệ thống	5	I, J, O, Q, W
Tập ký tự an toàn tối thiểu cho OCR	21	20 chữ ở vị trí thứ nhất, hợp thêm R

2.2.5. Màu nền và ý nghĩa

Bảng 2.2. Màu nền biển số và đối tượng áp dụng [13]

Màu nền / màu chữ	Đối tượng	Ghi chú cho ALPR
Trắng / đen	Tổ chức, cá nhân trong nước, xe không kinh doanh vận tải	Phổ biến nhất
Vàng / đen	Xe kinh doanh vận tải	Tín hiệu phân loại duy nhất còn hợp lệ
Xanh dương /	Cơ quan Đảng, Nhà nước, công an	Seri chỉ dùng 11 chữ cái

Màu nền / màu chữ	Đối tượng	Ghi chú cho ALPR
trắng		
Trắng / đỏ	Xe ngoại giao (NG) và tổ chức quốc tế (QT)	Cấu trúc chuỗi khác hẳn [30]
Đỏ / trắng	Xe quân đội và doanh nghiệp quân đội	Ngoài phạm vi TT 79/2024, thuộc TT 169/2021/TT-BQP [15]

QCVN 08:2024/BCA chỉ quy định **4 tổ hợp màu, không có nền đỏ** [11] — biển quân đội do Bộ Quốc phòng quản lý riêng [15]. **Xe điện không có biển riêng**: xe năng lượng sạch **không được cấp biển xanh lá**, dùng biển thường kèm biểu tượng [31] — không phát hiện được xe điện qua màu biển. Màu nền là tín hiệu phân loại duy nhất còn hợp lệ [32]; nhưng module chuẩn hoá làm việc trên chuỗi ký tự, phân loại theo màu ngoài phạm vi của nó.

2.2.6. Kích thước vật lý và tỷ lệ khung hình

Cơ sở định lượng phân biệt biển một dòng với hai dòng — then chốt với rủi ro R-04 (mục 2.4.3). Ô tô được cấp **02** biển: 01 ngắn (**2 dòng**), 01 dài (**1 dòng**); xe mô tô, xe gắn máy, rơ moóc được cấp **01** biển **2 dòng** [32] — **một ô tô mang cùng chuỗi ký tự trên hai biển hình dạng hoàn toàn khác nhau**.

Bảng 2.3. Kích thước và tỷ lệ khung hình của các loại biển số [11]

Loại biển	Kích thước (dài × cao)	Tỷ lệ khung hình	Số dòng
Ô tô — biển dài	520 × 110 mm	4,727	1 dòng
Ô tô — biển ngắn	330 × 165 mm	2,000	2 dòng
Xe mô tô, xe gắn máy	190 × 140 mm	1,357	2 dòng

⚠ Cảnh báo về mốc hiệu lực của bộ số liệu kích thước. Bộ số liệu trên **chỉ đúng từ 01/01/2025**; tiêu chuẩn trước đó quy định biển ô tô ngắn **200 × 280 mm**, biển dài **110 × 470 mm**, và rất nhiều tài liệu thứ cấp — kể cả bài báo năm 2023 — vẫn dùng bộ số cũ. Mọi trích dẫn kích thước biển số **bắt buộc ghi kèm mốc hiệu lực**; nếu không, người phản biện đối chiếu văn bản hiện hành sẽ kết luận là sai.

2.2.7. Ý nghĩa đối với thiết kế hệ thống nhận dạng

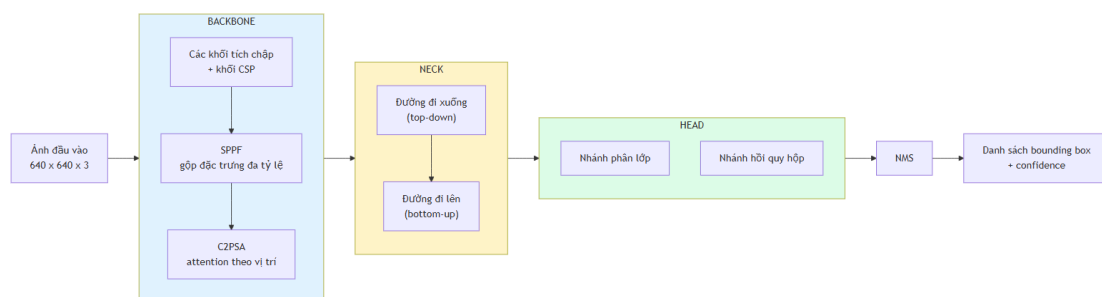
Bảy dữ kiện kéo theo bảy quyết định thiết kế: **81 mã tỉnh trong dài 89 số** biển lỗi OCR hai ký tự đầu thành sai phát hiện được; **tập seri khác theo vị trí** buộc ràng buộc **theo vị trí** và tập huấn luyện OCR đủ 36 ký tự; **hai kiểu seri xe máy, nhóm thứ tự 4 hoặc 5 chữ số** buộc biểu thức chính quy đa nhánh; **chuỗi 8 ký tự khớp hai loại biển** nên phải lưu số dòng độc lập; **seri không còn cho biết loại xe** nên cấm heuristic suy loại phương tiện; **khoảng trống tỷ lệ khung hình 2,727** là cơ sở ngưỡng phân loại bố cục; **khung pháp lý đổi ba lần trong hai năm** buộc hậu xử lý tách rời mô hình để cập nhật độc lập [19].

Về mức đóng góp: luận điểm dự kiến ban đầu — "loại trừ 6 chữ I J O Q R W" — là **sai**, và việc sửa làm **yếu đi** đóng góp theo tiêu chí thu hẹp không gian tìm kiếm; đổi lại phần giá trị chuyển sang ràng buộc **phụ thuộc vị trí trong chuỗi**: danh sách phẳng sẽ sai hệ thống trên toàn bộ lớp biển xe máy có R ở vị trí thứ hai — lỗi mà bộ luật của đồ án ngăn được. Đóng góp là **đúng dẫn về pháp lý và cấu trúc**, không phải cải thiện lớn về không gian tìm kiếm.

2.3. Cơ sở lý thuyết về phát hiện đối tượng

2.3.1. Kiến trúc YOLO: nguyên lý one-stage và anchor-free

Họ **two-stage** (Faster R-CNN) sinh vùng đề xuất rồi phân loại từng đề xuất — độ trễ cao; họ **one-stage** (YOLO, SSD, RetinaNet) hồi quy trực tiếp trong một lần lan truyền xuôi — thời gian thực. Ràng buộc CPU loại họ two-stage từ đầu; một nghiên cứu ALPR trên 50.000 ảnh và 10.000 video clip kết luận nhóm YOLO (v5–v10) vượt trội Faster R-CNN và SSD cả độ chính xác lẫn thời gian suy luận [48].



Hình 2.1. Kiến trúc tổng quát backbone – neck – head của YOLO11 (theo [49], [16])

Ba phần: **backbone** trích đặc trưng, kết thúc bằng SPPF gộp đa tỷ lệ; **neck** hợp nhất đặc trưng nhiều tầng; **head** sinh dự đoán — từ YOLOv8 dùng **anchor-free split head** [49]. Anchor-free có ý nghĩa riêng với biển số: anchor-based hồi quy theo tập hợp mẫu thiết kế theo phân bố COCO — biển số nằm ngoài phân bố đó (một dòng $\approx 4,7:1$, hai dòng $\approx 1,4:1$); anchor-free hồi quy **trực tiếp khoảng cách tâm đến bốn cạnh**, xử lý cả hai chế độ tỷ lệ bằng một cơ chế [16].

2.3.2. YOLO11: các cải tiến kiến trúc

Bài tổng quan độc lập xác định ba thành phần chính của YOLO11: **C3k2**, **SPPF**, **C2PSA** [50]; phần dưới đối chiếu trực tiếp mã nguồn Ultralytics [51]. **a) C3k2** — là **C2f có thể hoán đổi khối con**: C3k2 kế thừa trực tiếp từ C2f của YOLOv8; khác biệt duy nhất là một cờ — tắt thì giống hệt C2f, bật thì dùng khối C3k tùy chỉnh kích thước nhân [51]. YOLO11 giảm tham số mà giữ độ chính xác vì không đổi triết lý CSP, chỉ cấu hình linh hoạt hơn. **b) C2PSA** — thành phần YOLOv8 hoàn toàn không có, khác biệt kiến trúc thực sự; đặt ngay sau SPPF để attention tái phân bố trọng số theo vị trí không gian. Ultralytics khẳng định cơ chế này cải thiện phát hiện **đối tượng nhỏ** và **che khuất phức tạp** so với YOLOv8 [52].

Lưu ý về mức độ chứng minh. Phát biểu về đối tượng nhỏ là **định tính**: Ultralytics không công bố AP_small/AP_medium/AP_large theo chuẩn COCO cho từng biến

thể, nên không thể chứng minh định lượng YOLO11 hơn YOLOv8 bao nhiêu trên đối tượng nhỏ [16]. Đề án phải **tự đo trên dữ liệu của mình**; kết quả ở Chương 5.

Bảng 2.4. So sánh khác biệt kiến trúc giữa các phiên bản YOLO gần đây

Phiên bản	Khối backbone	Cơ chế attention	Đầu dự đoán	NMS	Điểm mới đáng chú ý nhất
YOLOv8 [49]	C2f	Không có	Anchor-free, tách nhánh	Có	Chuyển sang anchor-free
YOLOv9 [53]	GELAN	Không có	Anchor-free	Có	PGI chống mất mát thông tin
YOLOv10 [46]	Rank-guided blocks	Partial self-attention	Hai đầu song song	Không	Consistent dual assignments
YOLO11 [16]	C3k2 (kế thừa C2f)	C2PSA sau SPPF	Anchor-free, tách nhánh	Có	C2PSA cải thiện đối tượng nhỏ
YOLOv12 [54] / YOLOv13 [55]	R-ELAN / DS-C3k2	Area Attention / HyperACE (hypergraph)	Anchor-free	Có	Attention làm trung tâm; tương quan bậc cao, FullPAD
YOLO26 [47]	Kế thừa dòng Ultralytics	Kế thừa dòng Ultralytics	Bỏ DFL	Không (mặc định)	Bỏ DFL, dễ xuất và lượng tử hoá

Luận cứ chọn YOLO11 trình bày đầy đủ ở mục 3.2.

2.3.3. Các chỉ số đánh giá khối phát hiện

a) Precision, Recall và F1. Với TP dự đoán đúng, FP dự đoán sai, FN đối tượng bỏ sót:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Với ALPR, **recall của detection quan trọng hơn precision**: biển bỏ sót là mất vĩnh viễn, vùng báo nhầm bị hậu xử lý loại vì chuỗi không khớp cú pháp.

b) AP và mAP. AP là diện tích dưới đường cong Precision–Recall; mAP là trung bình AP trên N lớp — đề án có $N = 1$ nên mAP trùng AP:

$$\text{AP} = \int_0^1 p(r) dr, \quad \text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

c) **mAP@0.5** và **mAP@0.5:0.95** — hai chỉ số khác nhau, không được so sánh chéo. **mAP@0.5** tính tại một ngưỡng IoU cố định 0,5; **mAP@0.5:0.95** lấy trung bình trên 10 ngưỡng từ 0,5 đến 0,95 bước 0,05:

$$\text{mAP@0.5:0.95} = \frac{1}{10} \sum_{t \in \{0,50; 0,55; \dots; 0,95\}} \text{mAP}@t$$

Trên cùng một mô hình và cùng một tập dữ liệu, **mAP@0.5:0.95** LUÔN nhỏ hơn hoặc bằng **mAP@0.5** — vì **mAP@0.5** là một trong mười số hạng của phép trung bình ở (2.4), và là số hạng lớn nhất.

Khoảng cách giữa hai chỉ số với biến số thường rất lớn do hộp bao bọc, ba minh chứng: **87,2%** so với **46,5%** trên biển Ấn Độ [56]; **0,906** so với **0,631** ở một nghiên cứu YOLOv11 [57]; **99,5%** so với **80,7%** trên biển xe máy Indonesia [58]. Cả ba xác nhận: **biển số dễ phát hiện nhưng khó khớp hộp bao chính xác**.

⚠ **Cảnh báo phương pháp luận bắt buộc giữ nguyên.** Một cách trình bày phổ biến và **sai** là đặt **mAP@0.5** của một nghiên cứu ALPR (khoảng 0,90 – 0,99) cạnh **mAP@0.5:0.95** trên COCO của cùng lớp mô hình (khoảng 0,395 ở phân khúc nano [16]) rồi kết luận "bài toán biển số dễ hơn bài toán COCO". Đây là **so sánh giữa hai chỉ số có định nghĩa khác nhau**, và theo hệ quả toán học nêu trên, chênh lệch giữa chúng **không mang bất kỳ thông tin nào** về độ khó tương đối. Phép đối chiếu hợp lệ duy nhất là **mAP@0.5** với **mAP@0.5**, hoặc **mAP@0.5:0.95** với **mAP@0.5:0.95**, **và trên cùng một tập dữ liệu**; ngay cả khi cùng định nghĩa nhưng khác tập dữ liệu, so sánh cũng chỉ để cảm nhận độ khó chứ không làm luận cứ cho quyết định kỹ thuật. Lỗi này đã được phát hiện và sửa trong quá trình khảo sát tài liệu của đồ án, nêu tường minh ở đây vì hội đồng phản biện phát hiện rất nhanh.

d) **Chỉ tiêu của đồ án.** Vì mục tiêu detection là cắt vùng crop đủ tốt để OCR đọc, đồ án dùng **mAP@0.5** làm **chỉ tiêu chính**, **mAP@0.5:0.95** vẫn **báo cáo** nhưng không đặt ngưỡng chấp nhận; giá trị ở Chương 5. e) **mIoU**. Một số công trình dùng IoU trung bình toàn tập — nhóm Học viện Kỹ thuật Quân sự báo cáo mIoU 95,01% trên biển Việt Nam [59] — chỉ số khác mAP, không so sánh chéo được.

2.4. Cơ sở lý thuyết về nhận dạng ký tự

2.4.1. Bài toán OCR và đặc thù khi áp dụng cho biển số

OCR (*Optical Character Recognition*) chuyển văn bản trong ảnh thành chuỗi, thường gồm **text detection** khoanh vùng rồi **text recognition** đọc từng vùng. Sai lầm phổ biến: lấy thẳng bảng xếp hạng OCR phổ thông làm căn cứ chọn engine cho ALPR.

Bảng 2.5. So sánh OCR văn bản tài liệu và OCR biển số xe

Chiều so sánh

OCR văn bản tài liệu

OCR biển số xe

Chiều so sánh	OCR văn bản tài liệu	OCR biển số xe
Nguồn ảnh	Ảnh quét hoặc chụp tài liệu, gần chính diện	Ảnh cảnh ngoài trời, phối cảnh nghiêng, chói sáng, nhoè do chuyển động
Độ dài chuỗi và tập ký tự	Hàng trăm đến hàng nghìn ký tự; tập lớn, mở, kèm dấu	7 – 9 ký tự; tập đóng — chỉ A–Z và 0–9
Bố cục	Nhiều dòng, đa cột, ngắt dòng tùy ý	Cố định: 1 hoặc 2 dòng theo quy chuẩn nhà nước
Ràng buộc cú pháp và vai trò hậu xử lý	Gần như không có; hậu xử lý phụ trợ	Rất chặt, kiểm tra được bằng biểu thức chính quy; hậu xử lý bắt buộc , là lớp sửa lỗi chính
Tiêu chí đánh giá	CER / WER — chấp nhận sai lẻ tẻ	Khớp chuỗi tuyệt đối — sai 1 ký tự là hỏng cả bản ghi
Giá trị của thông tin ngôn ngữ	Cao — mô hình ngôn ngữ sửa lỗi hiệu quả	Thấp — không có từ vựng để dựa vào

2.4.2. Kiến trúc CRNN và hàm mất mát CTC

CRNN gồm ba tầng: **tầng tích chập** trích đặc trưng và — điểm mấu chốt — downsample chiều cao về **1**, biến bản đồ đặc trưng thành **chuỗi vector theo chiều rộng**; **tầng hồi quy** (Bi-LSTM) mô hình hoá ngữ cảnh hai chiều; **tầng phiên mã** giải mã thành chuỗi, thường bằng CTC. EasyOCR dùng đúng kiến trúc này (ResNet, Bi-LSTM, CTC) [60]; PaddleOCR dùng SVTR-LCNet kết hợp GTC [17], vẫn thuộc họ CTC.

Hàm mất mát CTC giải vấn đề: biết chuỗi nhãn đúng nhưng **không biết mỗi ký tự nằm ở cột đặc trưng nào**. CTC thêm ký hiệu trống ε , định nghĩa ánh xạ $\mathcal{B}$ gộp ký tự lặp rồi xoá ε — ví dụ $\mathcal{B}(3\varepsilon 00\varepsilon A) = 30A$ — và tính xác suất chuỗi nhãn $\mathbf{l}$ bằng tổng xác suất **mọi** đường đi thô $\boldsymbol{\pi}$ ánh xạ về nó:

$$p(\mathbf{l} \mid \mathbf{x}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{l})} \prod_{t=1}^T y_{\pi_t}^t, \quad \mathcal{L}_{\text{CTC}} = -\log p(\mathbf{l} \mid \mathbf{x})$$

Tổng ở (2.3) tính hiệu quả bằng quy hoạch động tiến–lùi. Ưu điểm quyết định: **không cần nhãn vị trí từng ký tự** — lý do CTC là mặc định của hầu hết engine OCR mã nguồn mở, và lý do LPRNet đạt 3 ms/biển trên GPU GTX 1080, 1,3 ms trên CPU i7-6700K mà vẫn 95% accuracy trên biển Trung Quốc [40].

2.4.3. Vì sao kiến trúc CTC gặp khó với văn bản nhiều dòng

Mục kỹ thuật quan trọng nhất của chương: nền tảng lý thuyết cho rủi ro **R-04** ("khả năng Cao, ảnh hưởng Cao") — ở Việt Nam nơi xe máy áp đảo, biển hai dòng là dạng phổ biến chứ không phải ngoại lệ.



Hình 2.2. Cơ chế sục đồ của CTC trên ảnh văn bản hai dòng (theo [61])

⚠ **Cảnh báo phạm vi áp dụng — bắt buộc giữ nguyên.** Cặp số 94,3% / 45,7% được đo trên bộ **RodoSol-ALPR của Brazil, không phải trên dữ liệu Việt Nam**. Nó được dẫn ở đây như một *analogue* định lượng về độ khó vượt trội của biển hai dòng xe máy tại một quốc gia cũng có tỷ lệ xe máy cao. Trích dẫn nhằm cặp số này thành số liệu Việt Nam là lỗi trích dẫn nghiêm trọng.

Bảng 2.6. Các phương pháp phân biệt biển một dòng và biển hai dòng

Phương án	Cơ chế	Ưu điểm và hạn chế
PA-1. Lấy lớp từ chính detector	YOLO xuất thêm một lớp: 0 = một dòng, 1 = hai dòng [64]	Chính xác nhất, chi phí gần 0 khi tự gán nhãn; phải gán nhãn hai lớp từ đầu
PA-2. Ngưỡng tỷ lệ khung hình	So ngưỡng suy từ quy chuẩn (mục 2.2.6)	Rẻ nhất, không cần huấn luyện; sai khi biển nghiêng nếu đo trên hộp bao thô
PA-3. Chiều ngang tìm điểm trung	Tổng cường độ pixel theo hàng; biển hai dòng có điểm trung sâu ở giữa [35]	Vị trí cắt thích nghi từng ảnh; điểm trung biển mất khi biển nghiêng
PA-4. Phân cụm hộp bao theo tọa độ dọc	Dùng text detection của engine OCR, gom nhóm theo tâm dọc [65]	Tái dùng kết quả sẵn có; phụ thuộc chất lượng text detection trên crop nhỏ
PA-5. Kiểm tra tính thẳng hàng của ký tự	Nối tâm ký tự trái nhất và phải nhất, đo độ lệch các ký tự còn lại [66]	Trực quan, dễ gỡ lỗi; cần phát hiện từng ký tự, ngưỡng pixel phụ thuộc độ phân giải

Lưu ý cách đọc số liệu này. Tài liệu gốc ghi recognition pre-trained 0,00%, nhưng con số đó **không** nghĩa là PaddleOCR không đọc được biển: mô hình pre-trained sinh thêm một ký tự đặc biệt khiến chuỗi trượt tiêu chí khớp tuyệt đối; hậu xử lý loại ký tự đó là đạt 90,97% [67]. Luận điểm đúng là **tinh chỉnh nâng 90,97% → 94,54%**; số liệu đo trên **biển Trung Quốc một dòng**, không chứng minh điều gì về biển hai dòng Việt Nam.

2.4.4. Chỉ số CER và độ chính xác mức chuỗi

a) CER (Character Error Rate) dựa trên khoảng cách Levenshtein, với S thay thế, D xóa, I chèn, N tổng ký tự chuỗi thực; độ chính xác mức ký tự là $1 - \text{CER}$:

$$\text{CER} = \frac{S + D + I}{N}$$

CER **có thể vượt 1** khi chuỗi dự đoán dài hơn chuỗi thực rất nhiều — đúng tình huống CTC gặp ảnh hai dòng. **b) WER** tương tự nhưng đơn vị là từ; một công trình biển Việt Nam báo cáo WER 0,014 trên bãi đỗ xe trong nhà [70]. **c) Độ chính xác mức chuỗi** (*plate-level accuracy, exact match*) là chỉ số nghiêm ngặt nhất:

$$Acc_{plate} = \frac{\#\{\text{biển số có TOÀN BỘ chuỗi ký tự khớp chính xác}\}}{\#\{\text{tổng số biển số trong tập kiểm thử}\}}$$

Sai một ký tự vẫn tính sai hoàn toàn — phản ánh đúng giá trị sử dụng: chuỗi sai hoặc không khớp bản ghi nào, hoặc khớp nhầm sang phương tiện khác. Quan hệ CER – mức chuỗi **không tuyến tính và bất lợi**: biển 8 ký tự với xác suất đúng mỗi ký tự p , xác suất đúng cả chuỗi là p^8 — $p = 0,99$ chỉ còn $\approx 0,923$, $p = 0,95$ tụt xuống $\approx 0,663$ — lý do engine có CER rất tốt trên văn bản vẫn thất bại trên biển số. **d) End-to-end Recognition Rate** — tỷ lệ biển đọc đúng hoàn toàn trên **toàn bộ pipeline** — là chỉ số duy nhất phản ánh lỗi tích lũy, chỉ tiêu quan trọng nhất của đề án; cuộc thi ICPR 2026 về biển độ phân giải thấp dùng chỉ số này làm chính, đội vô địch đạt 82,13% [71]. Kèm theo là chỉ số vận hành: **độ trễ** p50, p95, p99 (bắt buộc kèm cấu hình phần cứng), **kích thước mô hình**, **bộ nhớ thường trú**, **số tham số**; giá trị ở Chương 5.

2.5. Các công trình liên quan

2.5.1. Công trình quốc tế tiêu biểu

Khảo sát lập danh mục **21 công trình quốc tế tiêu biểu** từ 2018 đến 2026, kèm phương pháp, bộ dữ liệu đánh giá và kết quả công bố của từng công trình. Sáu mốc kiến trúc còn lại — số 4, 5, 12, 15, 18, 20 — **không kèm số liệu đối chứng công bố được**: Li–Wang–Shen 2019, mạng thống nhất một lần lan truyền xuôi [39]; Zhang và cộng sự 2020, attention 2D, công bố **CLPD** [41]; Nascimento và cộng sự 2024, **LCDNet** với hàm mất mát **LCOFL**, GAN có bộ phân biệt là OCR [78]; Meyer và cộng sự 2025, **SaLT** giảm phụ thuộc cú pháp [19]; Shabaninia và cộng sự 2025, nhận dạng **không phụ thuộc layout** trên IR-LPR, UFPR-ALPR, AOLP [42]; Gong–Liu 2026, **LP-LLM** trên Qwen3-VL với Character Slot Queries và LoRA [44].

2.5.2. Công trình về biển số Việt Nam

Nghiên cứu ALPR cho biển Việt Nam chủ yếu công bố tại hội nghị, tạp chí khu vực, **không xuất hiện trên các benchmark quốc tế lớn**, phần lớn đánh giá trên tập tự thu thập không công khai — so sánh công bằng gần như bất khả thi.

2.5.3. Các bộ dữ liệu chuẩn trong lĩnh vực

Khảo sát đối chiếu **chín bộ dữ liệu chuẩn** của lĩnh vực theo quy mô, đặc điểm và **giấy phép sử dụng** — cột giấy phép quyết định bộ nào dùng được cho đề án này.

2.5.4. Khoảng trống nghiên cứu và định vị đề tài

Bảng 2.7. Sáu khoảng trống nghiên cứu và cách đề án lấp

#	Khoảng trống được xác định từ khảo sát	Cách đề án lắp
1	Chưa có nghiên cứu Việt Nam nào công bố bảng so sánh tách riêng độ chính xác biển một dòng và biển hai dòng trên cùng một hệ thống (mục 2.5.2)	Đề án báo cáo tách bạch hai con số này
2	Chưa có nghiên cứu Việt Nam nào mô tả có hệ thống bộ luật hậu xử lý ràng buộc theo VỊ TRÍ trong chuỗi — các mô tả hiện có dừng ở danh sách phẳng, phần lớn dùng nhầm con số 20 chữ cái cho toàn chuỗi (mục 2.2.4)	Thiết kế hậu xử lý theo từng vị trí, đo tách bạch trước và sau hậu xử lý ; hiệu số là đóng góp định lượng
3	Hầu hết công trình trong nước chỉ báo cáo mAP của detection , không báo cáo end-to-end mức chuỗi (mục 2.5.2)	Báo cáo cả hai, end-to-end là chỉ tiêu quan trọng nhất
4	Không tồn tại benchmark công khai nào so sánh các engine OCR trên riêng ảnh biển số xe máy Việt Nam hai dòng (mục 3.3)	✅ Đã lắp 03/08/2026 — đo ba engine trên 2.801 biển, cùng tăng bao quanh: PaddleOCR 68,87% · EasyOCR 14,28% · Tesseract 10,28% (mục 3.3.3)
5	Số liệu hiệu năng thường công bố không kèm phần cứng (mục 2.5.1)	Mọi số liệu hiệu năng kèm: model CPU, số luồng, kích thước ảnh vào, backend suy luận, cỡ mẫu đo
6	Hầu hết kho mã nguồn mở Việt Nam không công bố số liệu và không có kiến trúc phần mềm (mục 2.5.2)	Công bố đầy đủ giao thức đo, tập kiểm thử, toàn bộ chỉ số; bàn giao hệ thống có API, giao diện, cơ sở dữ liệu, kiểm thử, đóng gói

Sáu khoảng trống đều thuộc loại **kỹ nghệ và báo cáo**, không phải thuật toán: đề án không đặt mục tiêu vượt các con số trên 99% đã khảo sát — trong đó 99,28% của nhóm Học viện Kỹ thuật Quân sự đo trên tập riêng không công khai — và mọi số liệu hiệu năng của đề án là **số liệu CPU**, không so trực tiếp với FPS đo trên GPU.

Tuyên bố trung thực đầy đủ về mức đóng góp, cùng bốn điều đề án **không** tuyên bố, đặt ở **mục 1.5.1 và 1.5.2** (Chương 1) — nơi chính danh để tuyên bố đóng góp.

CHƯƠNG 3. KHẢO SÁT CÔNG NGHỆ VÀ LỰA CHỌN MÔ HÌNH

3.1. Phương pháp khảo sát và tiêu chí lựa chọn

Mỗi lựa chọn trình bày theo cùng một khuôn: phương án đã xét, tiêu chí, kết luận, **đánh đổi phải chấp nhận**. Khi bằng chứng không đủ phân định, mục này nói rõ là không đủ.

3.1.1. Bốn ràng buộc chi phối mọi lựa chọn

Bốn ràng buộc sau thu hẹp không gian phương án **trước khi** so sánh — vì sao một số ứng viên mạnh bị loại sớm.

#	Ràng buộc	Hệ quả trực tiếp lên việc chọn
1	Suy luận trên CPU, không có GPU CUDA (CON-02, mục 4.3.1)	Phương án không công bố tốc độ CPU đều không có căn cứ để đánh giá ; mô hình hàng trăm triệu tham số loại từ đầu
2	Biển số Việt Nam có biển hai dòng	Engine giả định vẫn bản một dòng gây ở đây; tiêu chí phân loại, không phải điểm cộng
3	Phải đóng gói và bàn giao được	Giấy phép, dung lượng mô hình, số phụ thuộc là tiêu chí thật
4	Ngân sách thời gian CPU hữu hạn	Một số phép so sánh đã thiết kế nhưng không chạy được ; mục 3.1.2 nói rõ là những phép nào

3.1.2. Ranh giới giữa cái đã đo và cái mới chỉ khảo sát tài liệu

Không phải mọi lựa chọn đều qua thực nghiệm: có phép đồ án tự chạy trên chính máy và dữ liệu của mình, có phép chỉ dựa số liệu nhà phát hành, và có phép **chưa bao giờ chạy được**.

Bảng 3.1. Mức bằng chứng của từng phép so sánh trong chương

Phép so sánh	Mức bằng chứng	Trình bày ở
PP-OCRv5_mobile ↔ PP-OCRv6_medium	✅ Tự đo — 200 vùng cắt biển số, cùng máy, cùng thứ tự ảnh	3.3.2
Bộ nhận dạng gốc ↔ bản tinh chỉnh	✅ Tự đo — 2.801 biển có nhãn chuỗi, bốn cấu hình	5.4
YOLO11n ↔ YOLOv8n và năm thế hệ YOLO khác	📄 Khảo sát tài liệu — theo benchmark chính thức của nhà phát hành, đồ án không tự chạy lại	3.2
PaddleOCR ↔ EasyOCR ↔ Tesseract	✅ Tự đo 03/08/2026 — 2.801 biển có nhãn	3.3.3

Phép so sánh	Mức bằng chứng	Trình bày ở
	chuỗi, ba nhánh, cùng tầng bao quanh	
PyTorch ↔ ONNX Runtime ↔ OpenVINO	✗ Chưa đo — chọn theo benchmark bên thứ ba, chưa có số tự đo	3.4 · 5.6.3
Độ phân giải 416 ↔ 640	⚠ Có số đo nhưng không quy kết được — ba biến đổi đồng thời và ngược chiều nhau	3.6

Dòng **✗** còn lại là khoản nợ thực sự, ghi nhận nhất quán ở mục 5.9.2 và Chương 6:

- **So sánh runtime chưa chạy** ⇒ chọn ONNX Runtime đứng vững nhờ **lý do vận hành** (một runtime duy nhất cho cả hai mô hình, tránh xung đột hai framework học sâu), không nhờ số liệu tốc độ tự đo.

3.2. Mô hình phát hiện: YOLO11

Các phương án đã xét. Bỏ thế hệ YOLO từ YOLOv8 trở về sau — mốc chuyển sang anchor-free, có ý nghĩa trực tiếp với bài toán biến số (mục 2.3.1): YOLOv8 [49], YOLOv9 [53], YOLOv10 [46], YOLO11 [16], YOLOv12 [54], YOLOv13 [55], YOLO26 [47]. Họ two-stage (Faster R-CNN, Mask R-CNN) loại từ đầu vì chi phí tính toán không hợp ràng buộc CPU.

3.3. Engine nhận dạng ký tự

Đây là lựa chọn trình bày **trung thực nhất về mức độ chắc chắn**: chọn *họ engine* theo khảo sát tài liệu (3.3.1), rồi chọn *bậc mô hình* theo phép đo tự chạy (3.3.2).

3.3.1. PaddleOCR, EasyOCR, Tesseract — khảo sát tài liệu

Các phương án đã xét: tám engine — PaddleOCR, EasyOCR, Tesseract, TrOCR, docTR, MMOCR, fast-plate-ocr và RapidOCR/OnnxTR.

3.3.2. PP-OCRv5 mobile so với PP-OCRv6 — đo trên máy đồ án

Mục này chọn **bậc mô hình** bên trong họ đã chọn, dựa trên số liệu **tự đo**. Quy trình đầy đủ ở docs/reports/35-ppocrv6-evaluation.md.

Bảng 3.2. PP-OCRv6_medium_rec so với PP-OCRv5_mobile_rec, đo trên 200 vùng cắt biến số của đồ án

Mô hình	Đúng chuỗi	Trung vị	p95
PP-OCRv5_mobile_rec — đang dùng	134/200 = 67,0%	23,0 ms	31,9 ms
PP-OCRv6_medium_rec	145/200 = 72,5%	386,9 ms	429,0 ms

⚠ Đây là phép chiếu, không phải phép đo. Cộng 364 ms vào p95 hiện hành cho khoảng **1.507 ms**, tức **vượt sà 1.500 ms** và đẩy NFR-P1 từ **🟡** xuống **✗**. Con số này suy ra từ độ trễ nhánh nhận dạng đo cô lập, **chưa chạy lại toàn đường ống** —

muốn công bố phải đo thật. Nhưng ngay cả với sai số rộng, hướng của kết luận không đổi: NFR-P2 vốn đã trượt sà thì chắc chắn trượt sâu hơn.

Lưu ý bắt buộc khi trích bài PP-OCrv6. Cặp "+5,1 / +4,6 điểm" mà bài v6 công bố được tính trên **baseline của chính nó** (v5_server 78,1% / 81,6%), không phải trên baseline trong tài liệu PaddleX (86,38% / 83,8%). Ghép hai nguồn sẽ **đảo chiều kết luận**. Trích thì phải trích kèm baseline gốc.

3.3.3. Benchmark ba engine trên 2.801 biển số Việt Nam — đo 03/08/2026

Mục 3.3.1 kết thúc bằng một khoản nợ: giữ PaddleOCR dựa trên lý do kỹ thuật, **không** dựa trên bằng chứng độ chính xác, trong khi tài liệu công khai nghiêng về EasyOCR. Mục này trả nợ đó. Số liệu đầy đủ ở docs/reports/36-engine-benchmark.md.

a) Vì sao phép đo này khó làm đúng. Bốn lượt chạy đầu đều cho số vô nghĩa; mỗi lượt hòng lộ ra một điều kiện bắt buộc — truyền tên model tường minh, tắt backend oneDNN (mục 4.6.3), khôi phục tỷ lệ khung hình, lọc mảnh vụn ở mép dải ghép. Bài học chung: **phần lớn năng lực đọc biển số không nằm trong engine** mà ở tầng bao quanh nó. So sánh ba engine với ba tầng bao quanh khác nhau là đo tầng bao quanh chứ không đo engine.

b) Thiết kế. Cả ba engine chạy trong **đúng một tầng bao quanh** — sao đúng chuỗi bước của bộ nhận dạng bản giao hàng — trên **cùng một mảng NumPy đã chuẩn bị xong**; khác biệt duy nhất còn lại là engine. Đây cũng là bằng chứng thực nghiệm cho NFR-M5. Một chỗ cố ý không cào bằng: Tesseract chạy kèm whitelist A-Z0-9, vì giới hạn tập ký tự là **năng lực gốc** của nó; cắt bỏ "cho công bằng" mới là làm sai.

Kiểm chứng harness: nhánh có-split của PaddleOCR đo được **63,73%**, khớp **chính xác** NFR-A5 = 0,6373 công bố từ trước bằng một đường đo hoàn toàn khác — cùng một số tới bốn chữ số.

Bảng 3.3. So sánh ba engine OCR trên 2.801 biển số Việt Nam (567 một dòng, 2.234 hai dòng)

Engine	Nhánh	Toàn bộ	1 dòng	2 dòng	CER	Rỗng	p50
PaddleOCR	tắt split	28,81%	94,2%	12,2%	0,588	6	295 ms
PaddleOCR	có split	63,73%	94,2%	56,0%	0,094	11	405 ms
PaddleOCR	+ hậu xử lý	68,87%	94,9%	62,3%	0,089	11	402 ms
EasyOCR	tắt split	6,53%	15,3%	4,3%	0,647	4	84 ms
EasyOCR	có split	10,35%	15,3%	9,1%	0,282	1	248 ms
EasyOCR	+ hậu xử lý	14,28%	28,6%	10,7%	0,269	1	249 ms
Tesseract	tắt split	9,57%	47,3%	0,0%	0,777	960	101 ms
Tesseract	có split	9,60%	47,3%	0,0%	0,565	700	108 ms
Tesseract	+ hậu xử lý	10,28%	50,4%	0,1%	0,567	700	106 ms

c) PaddleOCR thắng dứt khoát — và điều này bác bỏ tài liệu công khai. Ở cấu hình bản giao hàng, PaddleOCR đạt **68,87%**, hơn EasyOCR **54,59 điểm** và hơn Tesseract **58,59 điểm** — khoảng cách quá lớn để quy cho nhiễu. Kết luận "*tài liệu công khai không cho thấy PaddleOCR vượt EasyOCR trên ảnh biển số*" ở mục 3.3.1 **vẫn đúng về tài liệu công khai**, nhưng nay đồ án có số liệu của chính mình trên biển số Việt Nam và nó nói ngược lại. Quyết định giữ PaddleOCR nay **có thêm căn cứ độ chính xác**.

d) Tách-rời-ghép-ngang **KHÔNG** độc lập engine — kết quả bất ngờ nhất.

Engine	tắt split → có split	Mức tăng
PaddleOCR	28,81% → 63,73%	+34,92 điểm
EasyOCR	6,53% → 10,35%	+3,82 điểm
Tesseract	9,57% → 9,60%	+0,03 điểm

Nếu cả ba cùng tăng mạnh, đóng góp kỹ thuật của đồ án sẽ độc lập với engine — một khẳng định mạnh. **Dữ liệu không cho phép nói thế**. Phát biểu đúng: tách-rời-ghép-ngang là điều kiện **cần** để đọc biển hai dòng — nó biến bài toán đa dòng thành bài toán một dòng — nhưng **không đủ**; engine phải đủ mạnh để tận dụng dải ảnh đã ghép.

Ngược lại, **bộ luật hậu xử lý giúp cả ba** (+5,14 · +3,93 · +0,68 điểm). Đóng góp (b) của đồ án vì vậy **là** đóng góp độc lập engine, khác với tách-rời-ghép-ngang.

e) Tesseract không đọc được biển hai dòng. **0,0% trên 2.234 biển hai dòng**, kể cả sau khi đã ghép thành một dòng, trong khi đọc được 47,3% biển một dòng. Đã kiểm bằng mắt để loại khả năng lỗi công cụ: nó **có** đọc ra chữ nhưng luôn kèm ký tự rác, và 700/2.801 lần trả chuỗi rỗng. Dự đoán "*Tesseract vỡ khi crop nhiều dòng*" ở mục 3.3.1 được xác nhận, ở mức nghiêm trọng hơn.

⚠ Hai điều phép đo này không trả lời. Thứ nhất, nó đo trên **vùng biển đã cắt sẵn**; báo cáo 31 cho thấy thứ tự xếp hạng có thể **đảo ngược** trên ảnh toàn cảnh qua bộ phát hiện thật, nên kết luận chỉ áp cho tầng nhận dạng. Thứ hai, nó **không** kết luận engine nào tốt hơn nói chung — chỉ kết luận engine nào đọc biển số Việt Nam tốt hơn *bên trong tầng bao quanh của đồ án*; một hệ thống thiết kế quanh EasyOCR, với tiền xử lý riêng của nó, có thể cho số khác.

3.4. Runtime suy luận trên CPU: ONNX Runtime

Các phương án đã xét: chạy trực tiếp tệp trọng số PyTorch, ONNX Runtime, OpenVINO.

Tiêu chí: tốc độ CPU, mức đa nền tảng, độ nặng phụ thuộc khi đóng gói, khả năng cùng tồn tại với framework khác.

Cảnh báo trích dẫn bắt buộc. Chỉ được dùng **phần số liệu tốc độ** của bảng benchmark này. Các con số mAP đi kèm được đo trên tập coco8 chỉ gồm **8 ảnh**, nên **vô nghĩa về mặt thống kê** và không được trích dẫn dưới bất kỳ hình thức nào.

3.5. Các lựa chọn công nghệ nền tảng khác

Phần lớn quyết định còn lại là **ràng buộc của đề bài**; ghi lại kèm lý do và đánh đổi để Chương 4 tham chiếu.

Bảng 3.4. Tổng hợp quyết định công nghệ nền tảng

#	Hạng mục	Lựa chọn (phương án thay thế)	Lý do chính	Đánh đổi phải chấp nhận
1	Web framework backend	FastAPI (Django, Flask)	Tự sinh đặc tả OpenAPI — tạo sẵn một sản phẩm bàn giao; có sẵn WebSocket và tác vụ nền	Phải biết khi nào không dùng hàm bất đồng bộ; nguy cơ tranh chấp với luồng suy luận
2	ORM và migration	SQLAlchemy 2.0 + Alembic (Tortoise ORM, Peewee)	Tích hợp sâu kiểu tĩnh; lược đồ đã thay đổi nên nhu cầu migration là có thật	Đường cong học dốc nhất trong nhóm
3	Cơ sở dữ liệu	SQLite (PostgreSQL, MySQL)	Ghi có thể xếp hàng vì suy luận CPU mới là nút cổ chai; không thêm dịch vụ khi đóng gói	Chỉ một tiến trình ghi tại một thời điểm ; vượt ngưỡng tải phải chuyển PostgreSQL
4	Frontend	React + TypeScript + Vite + TailwindCSS (Vue, Angular, Svelte)	Hệ sinh thái lớn nhất; công cụ build tiền nhiệm ngừng bảo trì; kiểu tĩnh nối tiếp từ backend	Tự lắp ghép routing, quản lý trạng thái, thành phần giao diện
5	Framework học sâu	PyTorch (TensorFlow)	Ultralytics khai báo PyTorch là phụ thuộc lỗi — chọn YOLO11 là chọn PyTorch	Kéo theo framework học sâu thứ hai (PaddlePaddle) do lựa chọn OCR — giải bằng mục 3.4
6	Đóng gói	Docker + Compose	Yêu cầu tái lập và khởi động bằng một lệnh	Kích thước image là rủi ro do có framework học sâu

3.6. Độ phân giải đầu vào: 640 thay vì 416

Đề án có sẵn hai mô hình để đối chiếu — baseline-416-v1.pt và best.pt — nhưng **phép so sánh giữa chúng không quy kết được nguyên nhân**: giữa hai lượt huấn luyện có **ba biến thay đổi đồng thời và ngược chiều nhau** (độ phân giải 416 → 640, bộ dữ liệu v1 → v3 đã khử rò rỉ, số epoch), nên chênh lệch chỉ số **không gán được cho riêng biến nào**. Điều

này phải nói rõ vì bản nháp trước từng trình bày best .pt như mô hình "tệ hơn baseline", trong khi ở tầng phát hiện nó **vượt mọi ngưỡng NFR** (mục 5.4.1).

Lựa chọn **640** vì vậy đứng trên căn cứ khác: đó là độ phân giải mà chỉ tiêu NFR-A1/A2 đặt ra và là độ phân giải mọi số liệu tốc độ CPU chính thức của Ultralytics được đo. Muốn quy kết nguyên nhân cần một ma trận thí nghiệm cô lập từng biến (E1 – E3), ước tính **≈ 33 giờ CPU** — vượt ngân sách còn lại, ghi nhận là **chưa thực hiện** ở mục 5.9.2.

CHƯƠNG 4. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

Trên cơ sở lý thuyết ở Chương 2 và lựa chọn công nghệ ở Chương 3, chương này trình bày thiết kế và hiện thực của hệ thống trong cùng một mạch. Nguyên tắc: mọi mô tả tương ứng với mã nguồn có thật; chức năng chưa hoàn thiện ghi rõ mức độ; số đo chưa có thì nói thẳng là chưa có. Chương gồm phân tích yêu cầu (4.1), kiến trúc (4.2), môi trường phát triển (4.3), bộ dữ liệu (4.4), huấn luyện mô hình (4.5), tầng AI (4.6), backend và CSDL (4.7), giao diện (4.8), Docker (4.9) và bảng đối chiếu cài đặt lịch thiết kế (4.10).

Trạng thái bản này: hệ thống chạy ALPRPipeline với mô hình chính thức `models/best.pt` (/health báo `model_loaded: true`, engine `yolo:best.pt+paddleocr-PP-OCRV5-mobile`, `mAP@0.5 = 0,9829`); StubPipeline đã ra khỏi đường chạy chính. Mọi kết quả thực nghiệm trình bày ở Chương 5.

4.1. Phân tích yêu cầu

4.1.1. Khảo sát nhu cầu và các tác nhân

Nhận dạng biển số là bài toán nền tảng của bãi đỗ tự động, thu phí không dừng, kiểm soát ra vào và giám sát giao thông. Áp mô hình ALPR huấn luyện trên dữ liệu nước ngoài vào Việt Nam gặp bốn trở ngại. **Thứ nhất, biển hai dòng chiếm tỉ trọng lớn** (toàn bộ xe máy và một phần ô tô) trong khi đa số bộ dữ liệu quốc tế giả định biển một dòng; điểm gây này đã đo được: trên **bộ RodoSol-ALPR của Brazil**, OpenALPR nhận đúng 3.772/4.000 ô tô biển một dòng (94,3%) nhưng chỉ 1.827/4.000 xe máy biển hai dòng (45,7%), chênh **48,6 điểm phần trăm** [7][88]. **Thứ hai, quy chuẩn biển số có tính pháp lý và cấu trúc chặt**: Thông tư 79/2024/TT-BCA [8], sửa đổi bởi TT 13/2025 [9] và TT 51/2025 [10], thông số vật lý theo QCVN 08:2024/BCA [11] — cấu trúc chặt vừa là ràng buộc vừa là cơ hội thiết kế cho khối hậu xử lý dựa trên luật. **Thứ ba, điều kiện thu nhận ảnh khắc nghiệt**: che khuất, bụi bẩn, nghiêng, ngược sáng, ban đêm. **Thứ tư, không có phần cứng tăng tốc**: máy thực hiện không có GPU CUDA, mọi suy luận và trình diễn chạy trên CPU (mục 4.1.4a, 4.3.1).

Lưu ý phạm vi số liệu. Cặp 94,3% / 45,7% đo trên **bộ RodoSol-ALPR của Brazil**, **không phải dữ liệu Việt Nam**; đồ án chỉ dùng nó làm dẫn chứng định lượng rằng "biển hai dòng khó hơn" là sự kiện đo được, không phải cảm nhận.

Hệ thống có bốn tác nhân: **người vận hành** (đưa ảnh/video, xem kết quả, tra cứu), **người phân tích** (thống kê, lọc, xuất báo cáo), **nhà phát triển** (tích hợp REST API), **hội đồng đánh giá** (quan sát, phản biện). Do hệ thống chạy nội bộ/localhost (giả định A-04), đồ án **không xây dựng xác thực và phân quyền**; ba tác nhân đầu là các *vai trò* trên cùng một giao diện, không phải các *tài khoản*.

4.1.2. Sơ đồ use case và ba use case chính

Ba use case chính — nhận dạng từ ảnh (UC-01), từ video (UC-02) và tra cứu lịch sử (UC-05) — đều được đặc tả theo cùng một khuôn: tác nhân, tiền điều kiện, luồng chính, luồng thay thế và hậu điều kiện.

4.1.3. Yêu cầu chức năng

Hệ thống có **34 yêu cầu chức năng** chia sáu nhóm, phân mức theo MoSCoW: 21 *Must*, 6 *Should*, 3 *Could*, 4 *Won't*. Bốn yêu cầu mức *Won't* đều là yêu cầu thuần giao diện và đều chuyển mức trong hai đợt thu gọn phạm vi ngày 20/07/2026 — trong đó **FR-4.1 là yêu cầu mức *Must* duy nhất bị đưa ra khỏi phạm vi**, ghi ở mục 6.2. Bảng đầy đủ từng mã yêu cầu ở **Phụ lục H.2**.

4.1.4. Yêu cầu phi chức năng

Các chỉ tiêu phi chức năng chia bảy nhóm — độ chính xác (NFR-A), hiệu năng (NFR-P), độ tin cậy (NFR-R), khả năng chịu tải (NFR-SC), khả năng bảo trì (NFR-M), bảo mật (NFR-S) và khả dụng (NFR-U) — mỗi chỉ tiêu kèm **ngưỡng tối thiểu, mục tiêu và phương pháp đo**. Hai ràng buộc chi phối toàn bộ nhóm hiệu năng: suy luận **chỉ trên CPU** (CON-02) và ngân sách độ trễ đầu-cuối. Bảng đầy đủ ở **Phụ lục H.3**; kết quả đối chiếu từng chỉ tiêu ở mục 5.7.

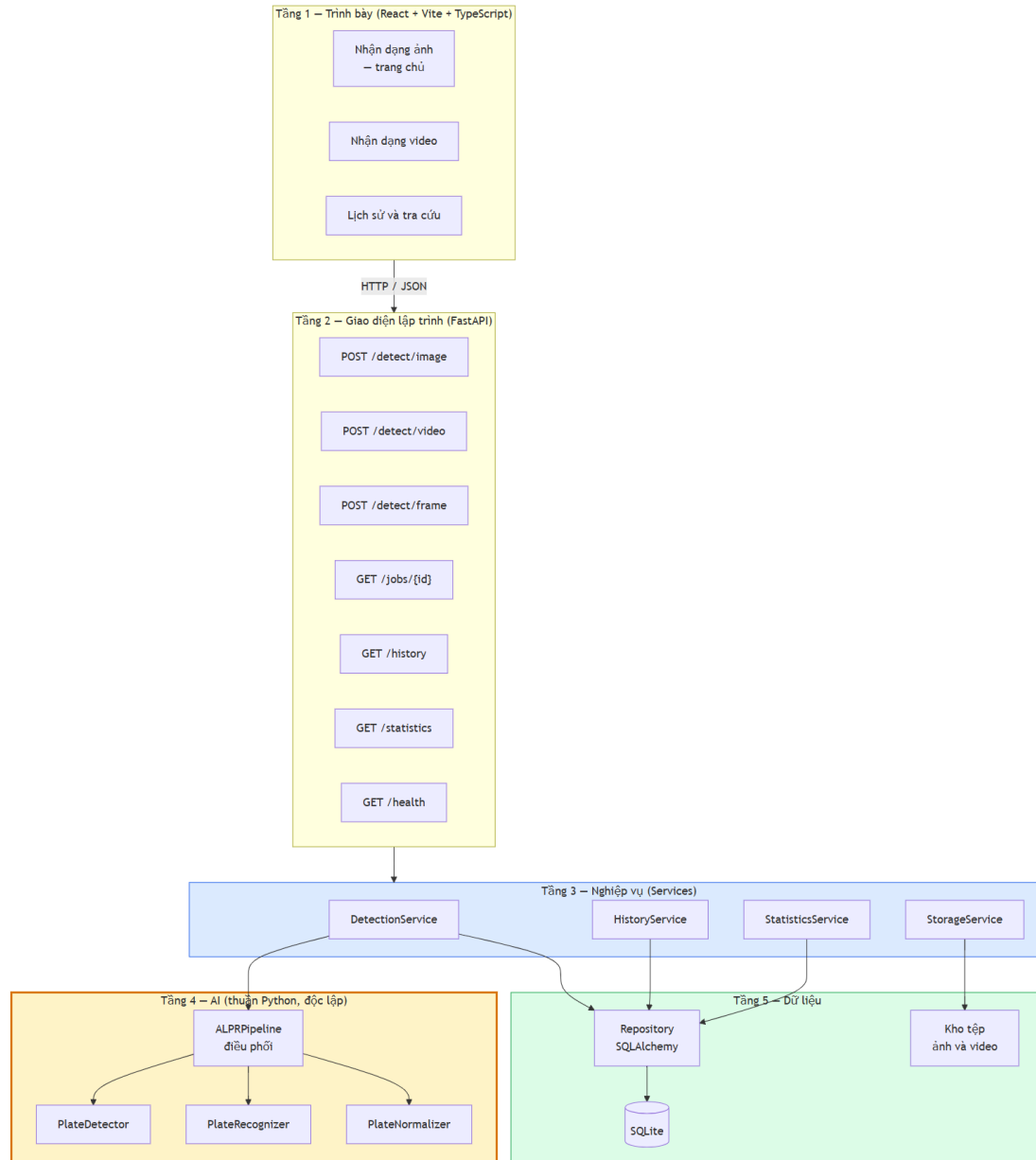
4.2. Kiến trúc hệ thống

4.2.1. Nguyên tắc kiến trúc

Kiến trúc sạch và quy tắc phụ thuộc: phụ thuộc chỉ hướng vào trong, từ chi tiết dễ thay đổi (framework web, CSDL, giao diện) về quy tắc nghiệp vụ ổn định. Ở bài toán này, thứ ổn định là thuật toán nhận dạng biến số — phát hiện, cắt, đọc, chuẩn hoá theo quy chuẩn Việt Nam; thứ dễ đổi là FastAPI hay Flask, SQLite hay PostgreSQL. Do đó **pipeline AI nằm ở vòng trong cùng**, tầng web phụ thuộc nó chứ không ngược lại. SOLID vận dụng: *trách nhiệm đơn nhất* — detector chỉ trả bounding box, recognizer chỉ trả chuỗi, normalizer chỉ chuẩn hoá — cho phép đo từng khối riêng; *thay thế Liskov* — dùng theo nghĩa đen khi hệ thống chạy pipeline giả lập đúng hợp đồng pipeline thật; *đảo ngược phụ thuộc* — tầng nghiệp vụ phụ thuộc hợp đồng trừu tượng, cài đặt tiềm từ ngoài.

Bốn ràng buộc kiến trúc: (1) **không trộn mã AI với mã API** (NFR-M1) ⇒ pipeline AI là package Python độc lập, không import framework web; (2) **mọi thành phần AI thay thế được** (NFR-M5) ⇒ đều đứng sau lớp trừu tượng; (3) **không hard-code đường dẫn** (NFR-M4) ⇒ mọi đường dẫn qua đối tượng cấu hình đọc từ biến môi trường; (4) **chạy được không cần GPU** (CON-02, NFR-C2) ⇒ thiết bị suy luận là tham số cấu hình, mặc định cpu — phát biểu là *cấu hình mặc định* chứ không phải "chế độ dự phòng", nên đường chạy CPU là đường được kiểm thử thường xuyên nhất.

4.2.2. Kiến trúc phân tầng



Hình 4.1. Kiến trúc phân tầng năm tầng và chiều phụ thuộc

Năm tầng: **1 — Trình bày** (giao diện, chỉ biết hợp đồng HTTP của tầng 2); **2 — API** (định tuyến, kiểm tra hợp lệ, ánh xạ ngoại lệ thành mã HTTP, sinh OpenAPI); **3 — Nghiệp vụ** (điều phối: tạo tác vụ, gọi pipeline, lưu tệp, ghi CSDL, gộp trùng, thống kê); **4 — AI** (phát hiện, nhận dạng, chuẩn hoá — **chỉ biết NumPy, OpenCV và thư viện học sâu**); **5 — Dữ liệu** (CSDL, kho tệp). **Điểm mấu chốt:** khối tầng AI **không có mũi tên nào đi lên**. Sau hai đợt thu gọn 2026-07-20, tầng trình bày còn **ba trang** nhưng **tầng 2–5 không đổi một dòng**: POST /detect/frame, GET /statistics, GET /health vẫn phục vụ và vẫn có kiểm thử tích hợp — phép thử ngoài dự kiến cho nguyên tắc phụ thuộc một chiều.

4.2.3. Tách tầng AI khỏi tầng API và cách kiểm chứng ràng buộc

Ba lợi ích. Kiểm thử độc lập: test chỉ cần nạp mảng NumPy, không phải dựng ứng dụng web. **Tái sử dụng trong script huấn luyện và đánh giá:** nếu logic tiền xử lý nằm lẫn trong hàm HTTP thì script đánh giá phải sao chép, hai bản sẽ lệch nhau — dẫn tới tình huống tệ nhất: **con số công bố không phải con số hệ thống thực sự tạo ra** (đúng loại sự cố đã xảy ra thật, mục 4.6.4g và 4.10). **Thay engine không sửa tầng API — đã kiểm chứng trên thực tế:** suốt Phase 5–7 hệ thống chạy StubPipeline, toàn bộ tầng API, nghiệp vụ, CSDL, giao diện được xây và kiểm chứng **trước khi mô hình được huấn luyện**; khi trọng số sẵn sàng, chuyển sang ALPRPipeline chỉ là đổi thành phần được tiêm, **không sửa dòng nào** ở router, service, schema. Để trạng thái mô phỏng không bị nhầm với vận hành thật, /health báo degraded chừng nào stub còn được dùng.

4.2.4. Luồng xử lý của pipeline AI và nhánh biến hai dòng



Hình 4.2. Luồng xử lý của pipeline AI, các khối tô đỏ là nhánh biến hai dòng

Nhánh biến hai dòng là phần khó nhất của đề án và là rủi ro đã xác định từ khâu lập kế hoạch (R-04). Bộ OCR dựng sẵn giả định văn bản một dòng ngang, nên với biến hai dòng chúng đọc theo thứ tự không xác định, ghép lẫn hoặc bỏ sót một dòng — cơ chế đứng sau chênh lệch 48,6 điểm phần trăm **do trên bộ RodoSol-ALPR của Brazil** đã dẫn ở 4.1.1 [7]. Giải pháp: **tách vùng biến thành hai nửa, nhận dạng từng nửa, ghép theo thứ tự trên trước dưới sau** — mỗi nửa lúc này là một dòng ngang đúng giả định của bộ OCR (cài đặt ở 4.6.4).

4.2.5. Bảng tổng hợp các quyết định kiến trúc

Mỗi quyết định ghi kèm lý do và **đánh đổi phải chấp nhận** — một quyết định trình bày như không có nhược điểm là quyết định chưa cân nhắc đủ.

Tám quyết định kiến trúc được ghi thành hồ sơ AD-01 ... AD-08, mỗi hồ sơ nêu **bối cảnh, phương án đã cân nhắc, quyết định và hệ quả phải chấp nhận** — dạng ghi chép này khiến một quyết định về sau có thể bị lật lại mà người lật hiểu được vì sao nó từng đúng. Các mục 4.2.1 – 4.2.4 trình bày bốn quyết định có ảnh hưởng rộng nhất.

Ghi chú: AD-03 không đổi sau khi gỡ trang Webcam vì ở ~5 FPS trên CPU, nút thắt là suy luận chứ không phải giao thức. AD-04 cố ý **không** chọn tracking vì phức tạp hơn đáng kể và thêm một họ siêu tham số. AD-05 là quyết định duy nhất **đã thay đổi** so với phác thảo ("PyTorch trước, ONNX nếu cần") — ghi nhận tường minh thay vì lặng lẽ sửa bảng. AD-06 kéo theo hai quyết định phái sinh đã cài đặt: yolo11n và **PP-OCRv5 mobile** — ràng buộc CPU thay đổi *lựa chọn mô hình*, không chỉ tốc độ.

4.3. Môi trường và công cụ phát triển

4.3.1. Cấu hình máy thực hiện và hệ quả của ràng buộc CPU

Toàn bộ cài đặt, kiểm thử, đo đạc chạy trên một máy trạm duy nhất (docs/00-requirements/environment.md): Windows 11 Pro; Intel Core i5-14600K, 14 nhân / 20 luồng; RAM 31,77 GiB; Intel UHD 770 — **không có GPU CUDA**; Python 3.13.12; Node.js 18.20.8; Docker 29.4.3. Cấu hình này **mâu thuẫn với mô tả môi trường ban đầu** (macOS Apple Silicon, Python 3.12); ghi nhận sai lệch thành văn bản là bước đầu của Phase 0. Ràng buộc CPU để lại dấu vết cụ thể: NFR-P1 phát biểu thẳng cho CPU; AD-06 kéo theo yolo11n và PP-OCRv5 mobile; và vì huấn luyện trên CPU mất 1–3 ngày/lượt, ai/training/ chạy được cả local lẫn Colab/Kaggle với siêu tham số trong tệp cấu hình (ai/training/config.py, 586 dòng). Hệ quả đo được: p95 đầu-cuối trên models/best.pt (máy rảnh) là **731 ms** (client-side) / **780 ms** (in-process), OCR chiếm **~64,3%**, phát hiện **~34,2%** (đối chiếu NFR-P1 ở 4.10).

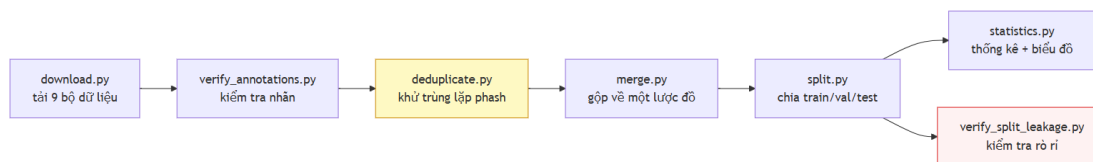
4.3.2. Ba môi trường ảo Python tách biệt và bộ công cụ

Đồ án dùng **ba môi trường ảo tách biệt**: .venv-ai/ (huấn luyện, xuất mô hình — NumPy 2.3.3, OpenCV 5.0, torch 2.13.0+cpu), .venv-ocr/ (thử nghiệm OCR — paddlepaddle 3.3.1, paddleocr 3.7.0), backend/.venv/ (dịch vụ — torch, ultralytics 8.4.101, paddleocr). Bắt buộc tách vì paddleocr kéo theo paddlex, **hạ cấp NumPy và thay opencv-python bằng opencv-contrib-python 4.10** — lùi một phiên bản lớn so với OpenCV 5.0 của nhánh huấn luyện; cài chung thì mỗi lần cài lại một nhánh âm thầm đổi phiên bản nhánh kia — lỗi không làm sập chương trình mà làm **kết quả đo không tái lập được**. Phân tách phản ánh ở requirements.txt và requirements-inference.txt, được Dockerfile.backend cài theo hai lớp riêng (4.9).

Bộ công cụ: FastAPI + Uvicorn; SQLAlchemy 2.x + Alembic; Pydantic v2; Ultralytics 8.4.101 chạy YOLO11 [16]; PaddleOCR 3.7.0 cho PP-OCRv5 [17]; Vite + React + TypeScript; pytest + pytest-cov; Docker Compose. Docker dùng Python 3.12 trong khi local dùng 3.13 là **chủ ý**: container là nơi lấy lại phiên bản mục tiêu (NFR-C1).

4.4. Xây dựng bộ dữ liệu

4.4.1. Đường ống sáu bước và thành phần bộ dữ liệu



Hình 4.3. Đường ống sáu bước xây dựng bộ dữ liệu

Mỗi bước là một script độc lập trong scripts/dataset/ có CLI riêng, sinh báo cáo JSON/CSV; run_pipeline.py chạy cả chuỗi một lệnh. **Kết quả: 15.133 ảnh** hợp nhất từ **7 bộ công khai** (Roboflow, HuggingFace, Kaggle), còn **6 nguồn nguyên tố** sau khi loại **11.978 ảnh (44,2%)** bản sao từ **27.111 ảnh**; tổng 9 bộ tải về, 2 bộ nhãn mức ký tự tách riêng cho đánh giá OCR. Chia 70/20/10 thành **10.592 / 3.027 / 1.514 ảnh**. Bảng dưới **phải trích khi nói về dữ liệu của đồ án**; nguồn và giấy phép từng bộ ở **Phụ lục C.1**.

Bảng 4.1. Đóng góp của từng bộ dữ liệu trước và sau khử trùng lặp

#	Bộ (slug)	Vào gộp	Còn lại	Bị loại
1	roboflow_school_fuhih	8.357	6.868 (45,38%)	17,8%
2	hf_vn_plates_segment	4.578	4.375 (28,91%)	4,4%
3	roboflow_traffic_camera	3.843	3.162 (20,89%)	17,7%
4	roboflow_eric_nguyen	840	353 (2,33%)	58,0%
5	roboflow_demo_tracking	236	235 (1,55%)	0,4%
6	roboflow_cuong_ta	8.254	140 (0,93%)	98,3%
7	roboflow_tran_ngoc_xuan_tin	1.005	0	100%
Tổng		27.113	15.133	44,2%

Cảnh báo phạm vi bắt buộc kèm mọi số liệu OCR. Phân loại màu nền trên 2.801 ảnh cho: **2.736 biển trắng (97,68%)**, 20 vàng, 4 xanh, **0 đỏ, 0 ngoại giao**. Phát biểu đúng là *"1 – CER = 0,9454 trên một tập gồm 97,7% biển trắng"*, **không phải "trên biển số Việt Nam"** (docs/reports/17-plate-type-audit.json).

4.4.2. Khử trùng lặp chéo bộ và con số 44,2%

Các bộ công khai fork lẫn nhau, nên một ảnh nằm ở train dưới tên bộ này và test dưới tên bộ khác thì **độ chính xác test đang đo khả năng ghi nhớ**; con số tiêu đề vì vậy là số nhóm trùng **chéo bộ**. Vết cặn ~690 triệu cặp là bất khả thi nên script dùng **multi-index hashing**: cắt mã băm 64 bit thành threshold + 1 dải — theo nguyên lý chuồng bồ câu, hai mã khác nhau tối đa threshold bit bắt buộc trùng khớp trên ít nhất một dải — nên tập ứng viên chứa mọi cặp thật rồi được xác minh chính xác: **thuật toán chính xác, không xấp xỉ**.

Có **hai phép đo trên hai mẫu số khác nhau**, trích một con số trần không nêu mẫu số là gây hiểu nhầm: **(a)** trên 7 bộ vào hợp nhất — mẫu số 27.111, ngưỡng Hamming 5, loại **11.978 = 44,2%, đã xoá thật**; **(b)** trên corpus còn lại — mẫu số 15.133, ngưỡng 10, chỉ ra **47,8% có thể loại** nhưng **chưa xoá**. 47,8% không mâu thuẫn 44,2%: ngưỡng lỏng hơn, và chỉ đo chứ chưa xoá. Hai hệ quả của tỷ lệ 44,2%: quy mô thật khác hẳn danh nghĩa (ca cực đoan tran_ngoc_xuan_tin vào 1.005 ra **0** — lý do **không được cộng dồn expected_images** của các bộ Roboflow), và phân bố huấn luyện lệch vì bản sao tập trung ở các bộ được chép nhiều nhất. split.py giữ **mọi thành viên của một nhóm trùng trong cùng split** nên bản trùng không bị xoá cũng không rò rỉ được.

4.4.3. Giới hạn của perceptual hash: nó tóm tắt bố cục khung ảnh, không tóm tắt chiếc xe

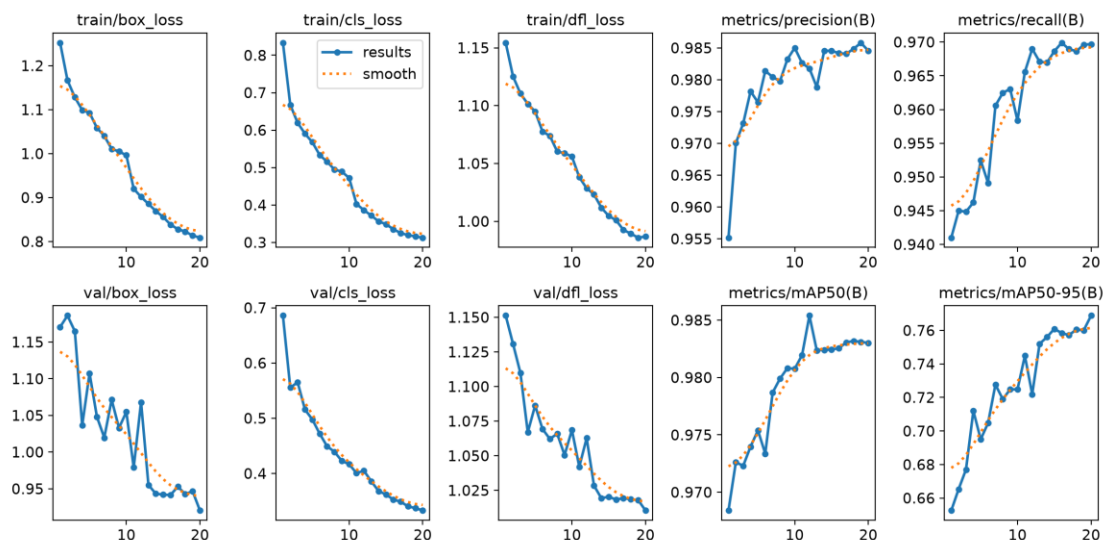
Đây là **giới hạn không khắc phục được** bằng công cụ hiện có. Một lần kiểm tra độc lập ở ngưỡng Hamming 10 trên bộ v1 tìm thấy **619 cặp gần trùng giữa train và test**, toàn bộ ở dải $d = 6-10$; kiểm bằng mắt cho thấy **cùng một chiếc xe, cùng chuỗi biển số, ở cả hai split**. Pipeline không bắt được vì bộ chia gom nhóm ở ngưỡng 5 rồi lần kiểm đầu cũng đo lại ở ngưỡng 5 và báo "0 cặp" — **lập luận vòng tròn**. Nâng ngưỡng cũng không giải quyết: phash rút ảnh thành 64 bit mô tả **cấu trúc tần số thấp của toàn khung**, nên hai xe khác nhau qua cùng một camera có khoảng cách phash rất nhỏ vì 90% khung hình giống hệt. Đánh đổi không thoát được: ngưỡng thấp bỏ sót cặp cùng xe khác ngày; ngưỡng cao gộp nhầm hàng nghìn ảnh xe khác nhau cùng camera. Bộ v3 chia lại với gom nhóm ngưỡng cao hơn và kiểm độc lập ở ngưỡng 10, nhưng đồ án ghi nhận thẳng thắn: **vẫn còn rò rỉ tồn dư không khử được bằng phash** — khắc phục đòi hỏi so khớp mức chuỗi biển số hoặc đặc trưng phương tiện. Hệ quả: baseline-416-v1.pt đạt $mAP@0.5 = 0,9933$ trên bộ v1 nhưng con số đó **không được báo cáo là "đạt"** vì tập test có rò rỉ đã đo được (4.10).

4.5. Huấn luyện mô hình

4.5.1. Siêu tham số và chi phí huấn luyện bộ phát hiện

Cấu hình lượt chính thức trích từ runs/final-640-v3/args.yaml — tệp Ultralytics tự sinh, là bản ghi *đã thực thi* chứ không phải *dự định*; bảng đầy đủ ở **Phụ lục B.1**. Giá trị chịu lực: model = yolo11n.pt (tiền huấn luyện COCO, **2.590.035** tham số — biến thể nano do ràng buộc CPU); imgsz = **640** (đúng độ phân giải NFR-A1/A2); epochs = **20**, batch = 8; optimizer = AdamW, lr0 = 0.001, cos_lr; close_mosaic = 10; device = cpu; seed / deterministic = 42 / true. **flip1r = 0.0** lệch có chủ ý so với mặc định 0.5: lật ngang tạo ký tự gương hoá — phân bố không bao giờ có trong thực tế. Vì giới hạn thời gian CPU chỉ chạy được **một lượt huấn luyện duy nhất**, không có nhiều seed để ước lượng phương sai; cố định seed ít nhất bảo đảm lượt này tái lập được — mọi chỉ số là kết quả **một lần chạy**, không có khoảng tin cậy (hạn chế ghi ở 5.9.3). **Chi phí**: baseline baseline-416-v1.pt 40 epoch, imgsz 416, bộ v1 — **156 phút**; best.pt 20 epoch, imgsz 640, bộ v3 — **≈ 35,6 phút/epoch, tổng ≈ 712 phút (≈ 11,9 giờ)** trên CPU. Ba yếu tố cùng thay đổi (số ảnh $\times 3,3$, diện tích $\times 2,37$, epoch giảm nửa) khiến so sánh ở mục 3.6 **không quy kết được nguyên nhân cho từng biến**.

4.5.2. Tiến triển của quá trình huấn luyện



Hình 4.4. Đường cong huấn luyện theo epoch — ba hàm mất mát và bốn chỉ số trên tập validation (nguồn: `runs/final-640-v3/results.csv`)

Ba hàm mất mát giảm đơn điệu và **không có dấu hiệu quá khớp**: box_loss 1,252 → 0,809, cls_loss 0,833 → 0,313, df_l_loss 1,154 → 0,987; đường validation bám sát đường train suốt 20 epoch. Chỉ số trên tập validation đi lên rồi bão hoà sớm: mAP@0,5 đạt **0,9684 ngay ở epoch 1** và chỉ nhích lên **0,9830** ở epoch 20, trong khi mAP@0,5:0,95 — chỉ số nhạy với độ khít của hộp — tăng đáng kể hơn, **0,6526 → 0,7688**.

Bảng 4.2. Tiến triển chỉ số trên tập validation theo mốc epoch

Epoch	box_loss (val)	cls_loss (val)	df_l_loss (val)	mAP@0.5	mAP@0.5:0.95	Precision	Recall
1	1,1705	0,6858	1,1513	0,9684	0,6526	0,9552	0,9410
2	1,1861	0,5554	1,1307	0,9726	0,6653	0,9700	0,9450
3	1,1647	0,5651	1,1098	0,9723	0,6770	0,9731	0,9449
5	1,1074	0,4977	1,0863	0,9754	0,6950	0,9765	0,9525
10	1,0548	0,4168	1,0686	0,9808	0,7248	0,9850	0,9584
15	0,9420	0,3619	1,0205	0,9824	0,7609	0,9846	0,9686
20	0,9204	0,3331	1,0105	0,9830	0,7688	0,9846	0,9697
Epoch tốt nhất (= 20)	0,9204	0,3331	1,0105	0,9830	0,7688	0,9846	0,9697

4.5.3. Tinh chỉnh bộ nhận dạng ký tự và lý do không đưa vào bản giao hàng

PP-OCRV5 mobile huấn luyện trên chữ cảnh tổng quát; mục này trả lời bằng số câu hỏi fine-tune trên đúng miền dữ liệu thì được gì. **Cấu hình:** 30 epoch trên 6.672 mẫu (2.801 ảnh gốc + hai biến thể tăng cường mỗi ảnh), kiểm định 571 mẫu, charset đủ 36, khởi đầu từ en_PP-OCRV5_mobile_rec_pretrained; kết thúc train acc **0,9449**, val acc **0,8809**.

Bảng 4.3. So sánh bộ nhận dạng gốc và bản tinh chỉnh trên cùng ngữ liệu

Cấu hình	A5 (chuỗi thô)	A6 (sau hậu xử lý)	Đúng định dạng	ms/ảnh
Model gốc, det + rec — bản giao hàng	0,6373	0,7512	0,9443	328,8
Model fine-tune, det + rec	0,5998	0,6762	0,9018	—
Model gốc, chỉ rec	0,6776	0,7508	0,9568	35,7
Model fine-tune, chỉ rec	0,8618	0,8758	0,9886	38,5

4.6. Tầng AI — thiết kế và cài đặt

4.6.1. Tổ chức gói ai/inference và ba lớp trừu tượng

Gói gồm mười một mô-đun cùng `__init__.py`, tổng **4.852 dòng**: `types.py`, `interfaces.py`, `config.py`, `exceptions.py`, `plate_rules.py`, `normalizer.py`, `detector.py`, `recognizer.py`, `two_line.py`, `plate_color.py`, `pipeline.py`. Ràng buộc "không import FastAPI" kiểm chứng tự động ở 4.2.3; lý do nền tảng: gói phải chạy được trong Jupyter, script benchmark và Colab.

4.6.2. Bộ phát hiện — YoloPlateDetector

YoloPlateDetector và PaddleOcrRecognizer (4.6.3) đều là **adapter mỏng**: không nơi nào ngoài hai mô-đun này chạm vào Results của Ultralytics hay máy OCR của PaddleOCR. Cả hai **ghim phiên bản mô hình tường minh** — name của detector trả `yolo:{stem}{suffix}`, recognizer ghim `OCR_VERSION = "PP-OCRV5"` — vì một con số benchmark chỉ tái lập được khi nêu đúng bộ trọng số; nâng cấp thư viện không được âm thầm đổi mô hình đứng sau một kết quả đã công bố.

Detector **nạp trọng số ngay trong hàm khởi tạo** để tệp thiếu làm hệ thống thất bại lúc khởi động kèm hướng dẫn khắc phục, thay vì thất bại lúc có yêu cầu đầu tiên. Nó nhận `.pt/.onnx/.torchscript` và **cả thư mục OpenVINO** — từ chối thư mục sẽ khiến cấu hình nhanh nhất trên CPU Intel không cấu hình được. Mọi hộp bao đều được **kẹp về biên ảnh** và hộp suy biến trả None kèm log, nên tầng trên không bao giờ nhận toạ độ nằm ngoài ảnh.

4.6.3. Bộ nhận dạng ký tự — PaddleOcrRecognizer

Một phát hiện kỹ thuật phải nêu ở thân bài: backend oneDNN làm sập suy luận trên PP-OCRv5. Trên đúng nền tảng mục tiêu (paddlepaddle 3.3.1, Windows, CPU), chạy mô hình phát hiện văn bản qua oneDNN kết thúc bằng `NotImplementedError: (Unimplemented) ConvertPirAttribute2RuntimeAttribute...` — khiếm khuyết phía thư viện, không phải lỗi cấu hình. Xử lý: hằng số có tài liệu `DEFAULT_ENABLE_MKLDNN: Final[bool] = False`. Đây là **núm hiệu năng, không phải núm độ chính xác** — khi lỗi thượng nguồn được sửa chỉ cần lật giá trị và đo lại. Nó cũng giải thích một phần NFR-P1: **một con đường tăng tốc CPU hiển nhiên đang bị chặn bởi lỗi thư viện chứ không phải chưa được thử.**

4.6.4. Mô-đun xử lý biến hai dòng — `two_line.py`

a) Vì sao bài toán tồn tại. Bộ nhận dạng hiện đại là CRNN/CTC với giả định **căn chỉnh đơn điệu** giữa cột ảnh và ký tự — chỉ đúng với văn bản một dòng; chồng lên đó, PP-OCR **resize mọi ảnh cắt về chiều cao 48 px** [103]. Biển xe máy 140×190 mm (QCVN 08:2024/BCA [11]) có tỷ lệ $\approx 1,36$, nên sau khi ép về 48 px mỗi hàng ký tự chỉ còn ~ 24 px — dưới mức nét chữ còn tách rời. Hệ quả định lượng: trên bộ **RodoSol-ALPR của Brazil**, OpenALPR đạt **94,3% trên biển ô tô một dòng** nhưng chỉ **45,7% trên biển xe máy hai dòng** [7][88] — đồ án trích cập số này thuần túy làm dẫn chứng tương đương định lượng, không phải số liệu Việt Nam.

b) Ước lượng số dòng bằng tỷ lệ khung. `estimate_line_count` dùng `DEFAULT_TWO_LINE_AR_THRESHOLD = 2.3: line_count = 2 if aspect_ratio < threshold else 1`. Đây là **heuristic do đồ án đề xuất, không phải quy tắc pháp lý** — quy chuẩn chỉ cung cấp kích thước vật lý ($4,727 / 2,000 / 1,357$); 2,5 chọn **lệch về phía hai dòng** vì đường xử lý hai dòng suy giảm êm khi gặp đầu vào một dòng, chiều ngược lại thì không. Hạn chế ghi trong mã: dải 2,5–3,0 là vùng xám thật vì biến một dòng chụp nghiêng gắt có tỷ lệ hộp bao tụt vào đó; định lượng tần suất thuộc Chương 5.

c) Cắt trên/dưới có chồng lấn. `split_two_line` dùng `UPPER_HALF_END_RATIO = 5/12`, `LOWER_HALF_START_RATIO = 1/3` — hai nửa **chồng lấn 1/12 chiều cao biển**, chủ ý do bất đối xứng chi phí: cắt cụt chân/dỉnh chữ phá huỷ thông tin **vĩnh viễn**, còn lọt vài điểm ảnh hàng bên cạnh thì bộ nhận dạng bỏ qua như nền. Hàm ép hai nửa không rỗng và cảnh báo nếu tham số làm mất chồng lấn.

d) Ghép ngang bằng `np.hstack`. Chiều cao chung `max(upper.h, lower.h, MIN_MERGE_HEIGHT)` với `MIN_MERGE_HEIGHT = 48` — bằng đúng chiều cao đầu vào cố định của PP-OCR. Nửa trên đặt bên trái nên thứ tự đọc bảo toàn, và sau khi ghép, **một hàng ký tự duy nhất nhận trọn ngân sách 48 px** thay vì hai hàng chia nhau; `_match_channels` nâng cả hai nửa về BGR khi số kênh lệch.

e) Tiền xử lý ảnh biển — `preprocess_plate`. Ba bước, **mỗi bước bật/tắt độc lập** để ablation được: chuyển xám (ký tự không mang thông tin màu); CLAHE (`clipLimit=2.0`, ô 8×8) vì biến phản quang tạo mảng chói cục bộ mà cân bằng toàn cục không xử lý được [95]; khử nhiễu `bilateralFilter(5, 50, 50)` vì lọc song phương **bảo toàn biên** — làm mờ Gauss đủ mạnh sẽ bo tròn đầu nét, thứ phân biệt 8 với B. Kết quả luôn là BGR ba kênh;

recognizer truyền `upscale_to_height = 64` vì ảnh biến ra khỏi detector thường chỉ cao 20–40 px.

f) Bước cứu dòng trên — một giả thuyết hợp lý bị chính dữ liệu bác bỏ. Mục có giá trị phương pháp luận cao nhất của khối: quan sát chế độ hồng — đề xuất giả thuyết *nghe rất hợp lý — đo và bác bỏ* — và chính phép bác bỏ dẫn tới thiết kế đúng. Ảnh biến 29E-015.66 trả về 015.66: sau khi ghép, bộ phát hiện văn bản chỉ tìm thấy **một** vùng chữ và bỏ hẳn cụm 29E. Giả thuyết đầu tiên — bỏ phép ghép, đọc riêng từng nửa rồi nối chuỗi — được **đo** trên 200 biến hai dòng có nhãn và **sự đổ**: 64,5% xuống **3,5%**, thắng ở **0/200** ảnh (số liệu đầy đủ ở 5.5.6). Nguyên nhân nằm đúng ở chi tiết đã biện minh ở (c): hai nửa cắt **chồng lấn** đi vào OCR riêng rẽ thì dải chồng lấn **bị đọc hai lần**, sinh ký tự rác nối giữa chuỗi. Kết luận đảo ngược cách hiểu về phép ghép: trên dải liền mạch, vùng lặp nằm giữa hai cụm chữ và **bị bộ phát hiện văn bản gạt đi** — hai ảnh rời thì không có ngữ cảnh để gạt.

Bản sửa vì vậy **giữ nguyên chiến lược ghép**, chỉ thêm một bước phục hồi hẹp qua vị từ `should_rescue_two_line(recognition)`: chỉ True khi đồng thời `line_count == 2`, `not is_valid_format`, và `raw_text` khác rỗng — khi đó tốn thêm **một** lần OCR trên riêng nửa trên, ghép `upper + raw`, chuẩn hoá lại; kết quả mới **chỉ được nhận nếu qua kiểm tra định dạng**, mọi trường hợp khác trả nguyên kết quả cũ. **Tính chất "không thể làm tệ đi"** là **tính chất cấu trúc**: cổng chỉ mở khi kết quả **đã hồng sẵn**, nên tập bị ảnh hưởng và tập đang đúng là hai tập rời nhau — hai phép đo A/B ở 5.5.6 vì vậy là *kiểm chứng*, không phải *căn cứ*. Mức cải thiện **khiêm tốn** (+1,86 và +0,50 điểm trên hai mẫu đọc lặp, 0 ca bị làm hồng), không trình bày như đột phá: nó vá một điểm mù cụ thể với chi phí ~15–21 ms mỗi biến hai dòng, không đụng tới nút thắt chính.

g) Một lỗi phương pháp đo, quan trọng hơn chính bản vá.

`ai/evaluation/ocr_accuracy.py` — nơi sinh các con số NFR-A4–A7 công bố ở Chương 5 — **không đi qua ALPRPipeline** mà gọi thẳng recognizer và normalizer, nên mọi logic ở tầng điều phối **vô hình với các con số công bố**. Cách sửa: **tách bước cứu thành hai hàm tự do cấp mô-đun** (`should_rescue_two_line`, `rescue_two_line_upper`) để cả pipeline lẫn bộ đo cùng gọi. Bài học vượt ra ngoài biến hai dòng — **một bộ đo đi tắt qua tầng điều phối sẽ đo một hệ thống khác với hệ thống được giao** — và biện pháp này về sau **vẫn không đủ**: cùng loại lỗi tái diễn lần thứ ba, phân tích ở mục 5.5.6.

4.6.5. Bộ luật hậu xử lý — đóng góp kỹ thuật chính

Bộ phát hiện và bộ nhận dạng đều là mô hình có sẵn; khối hậu xử lý thì không. `plate_rules.py` tuân ba quy tắc: **thuần khiết** (không I/O, không trạng thái toàn cục khả biến); **regex sinh từ tập hợp, không viết tay** — mẫu không thể trôi khỏi bảng nó mã hoá; **lớp ký tự là hằng có tên**.

a) PROVINCE_CODES — 81 mã tỉnh đang dùng theo phụ lục TT 51/2025/TT-BCA [10] (80 mã địa phương + mã 80), song song UNUSED_PROVINCE_CODES gồm **8 mã không bao giờ được cấp**: 13, 42, 44, 45, 46, 87, 91, 96. Giá trị so với `\d{2}`: nó **bác bỏ** 13A–123.45; giữ tường minh tập không dùng cho phép test khẳng định hai tập phủ đúng dải 11–99. Sáp nhập

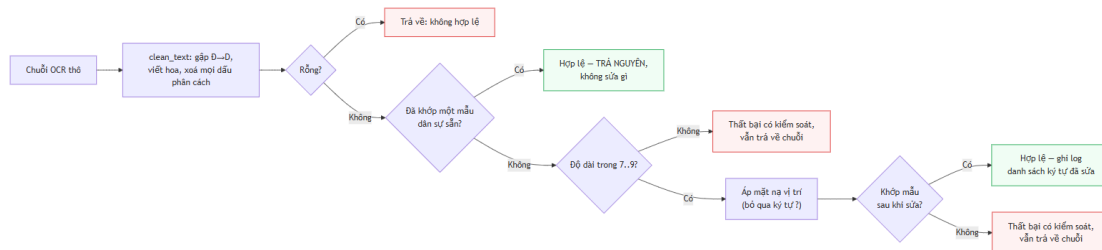
hành chính 2025 không làm mất hiệu lực biển đã cấp — mối quan tâm của tầng báo cáo, không phải của định dạng.

b) Các lớp ký tự sê-ri. Bốn hằng: L20 (chữ sê-ri ô tô; chữ **thứ nhất** sê-ri xe máy), L20B (chữ **thứ hai** sê-ri xe máy), L11 (sê-ri biển xanh), L21 (20 chữ chuẩn **cộng R**). **L20 và L20B là ảnh gương của nhau ở đúng hai ký tự** (L20 có G không R, L20B có R không G): 29-AR 123.45 hợp lệ còn 29-AG 123.45 thì không. L21 tồn tại vì một mô hình charset "20 chữ" **không bao giờ dự đoán ra R** nên sai **có hệ thống** trên mọi biển xe máy mang R ở sê-ri thứ hai — loại sai không hậu xử lý nào cứu được vì thông tin đã huỷ ở tầng mô hình. Cùng logic: OCR_SAFE_CHARSET = 31 ký tự, OCR_TRAINING_CHARSET = 36; huấn luyện trên 36 rồi ràng buộc về 31 là chủ ý — mô hình **được phép** dự đoán ký tự bất hợp pháp tạo sai lầm *quan sát được, sửa được*, mô hình *không thể về mặt kiến trúc* dự đoán nó tạo sai lầm vô hình. EXCLUDED_LETTERS = {I, J, O, Q, W} — 5 chữ bị loại toàn quốc; chính việc loại I, O, Q làm sửa lỗi OCR khả thi. (Ghi chú hiệu chỉnh: con số "6 chữ bị loại" từng gồm cả R đã được sửa.)

c) POSITION_MASKS và ký tự đại diện ? ở chỉ số 3 — chi tiết cài đặt quan trọng nhất của khối. Ba mặt nạ: car_5 = "DDLDDDD", car_4 = "DDLDDDD", motorcycle_9 = "DDL?DDDD"; MASK_BY_LENGTH ánh xạ độ dài 7/8/9. D = bắt buộc chữ số, L = bắt buộc chữ cái, ? = **đại diện, tuyệt đối không ép kiểu**. Hai kiểu biển xe máy cùng 9 ký tự khác nhau ở đúng vị trí này — kiểu mới sê-ri hai chữ (29AA12345), kiểu cũ sê-ri chữ + số (29B112345, vẫn lưu hành) — nếu tách thành DDLDDDD và DDLDDDD thì ép kiểu tại chỉ số 3 là bắt buộc, và kết quả kiểm chứng bằng chạy thật: **một trong hai kiểu bị phá huỷ** (29AA12345 → 29A412345, hoặc 29B112345 → 29BL12345), trong khi DDL?DDDD trả lại đúng cả hai. Chỉ số 3 của chuỗi 9 ký tự là **vị trí duy nhất trong toàn hệ thống biển số Việt Nam mà cả chữ lẫn số đều hợp lệ**; nhánh ? viết tường minh trong apply_position_rules chứ không rơi vào else. Chuỗi 8 ký tự không cần mặt nạ thay thế vì cả hai cách đọc áp cùng DDLDDDD.

d) Hai bảng ánh xạ nhầm lẫn và tính không đối xứng. TO_DIGIT 12 mục (0→0, Q→0, D→0, I→1, J→1, L→1, Z→2, A→4, S→5, G→6, T→7, B→8), TO_LETTER 9 mục (0→D, 1→L, 2→Z, 3→B, 4→A, 5→S, 6→G, 7→T, 8→B); áp riêng tại vị trí D và L. **Ánh xạ không đối xứng, và đó là phát hiện trung tâm:** 0 → 0 đúng, nhưng 0 → 0 **không bao giờ đúng** vì 0 không phải chữ sê-ri hợp lệ — với cả 0 và Q bị loại, D là ứng viên đồng hình duy nhất còn lại, nên chiều đúng là 0 → D tại vị trí chữ; chữ R **tuyệt đối không được ánh xạ đi** vì hợp lệ ở vị trí thứ hai sê-ri xe máy (2.2.4). Quy tắc an toàn: **ký tự không có mục trong bảng thì giữ nguyên. Ghi nhận trung thực về nguồn gốc:** hai bảng suy từ lập luận hình dạng ký tự, **không phải từ đo đạc**; vài cặp — đáng chú ý L → 1 — là phỏng đoán yếu; thay bằng bảng trích từ ma trận nhầm lẫn 36×36 đo được thuộc Chương 5, và trình bày bảng hiện tại như **giả thuyết cần kiểm chứng** vừa trung thực vừa mạnh hơn về học thuật.

e) Thuật toán chuẩn hoá. VietnamesePlateNormalizer.normalize_detailed thực hiện luồng sau:



Hình 4.5. Thuật toán chuẩn hoá chuỗi biến số theo bộ luật ràng buộc vị trí

Ba nguyên tắc chịu lực: **thử regex trước khi sửa** (chuỗi đã hợp lệ thì mọi chỉnh sửa chỉ có thể làm hỏng); **không bao giờ vứt bỏ** — chuỗi không sửa được vẫn trả về với `is_valid_format=False` và được lưu; **giữ chuỗi thô** vào `raw_ocr_text`. Kết quả là `NormalizationOutcome` bất biến mang chuỗi thô, chuỗi cuối, cờ hợp lệ, kết quả phân loại và **danh sách bộ ba (chỉ số, trước, sau) cho từng ký tự đã sửa** — dấu vết kiểm toán mà chương đánh giá dựa vào.

f) Xử lý nhập bằng số dòng. `line_count == 1` **chứng minh** chuỗi là biển ô tô (xe máy luôn hai dòng), còn `line_count == 2` **không chứng minh gì** vì biển ô tô ngắn cũng hai dòng — cờ `is_ambiguous` được giữ, `KindDecision` trả *tập ứng viên* kèm cờ thay vì bịa thông tin đầu vào không chứa. Thứ tự `PATTERNS_BY_KIND`: `DIPLOMATIC` đầu (hình dạng không thể nhầm); `SPECIAL` trước các mẫu xe máy (danh sách mã đóng và hiểm); `MILITARY` cuối vì là trường hợp **nhận-ra-để-loại-trừ** — khớp `RE_MILITARY` nhưng không thuộc `CIVIL_KINDS` nên không bao giờ được báo là biển dân sự hợp lệ.

4.6.6. Ghép thành `ALPRPipeline` bằng *tiêm phụ thuộc*

`ALPRPipeline` là **đối tượng tổ hợp**: không giữ mô hình, chỉ sắp thứ tự giai đoạn, cắt ảnh, **đo thời gian từng giai đoạn** và **cô lập lỗi mức từng biển** — không chứa logic học sâu nên kiểm thử được bằng thành phần giả lập; `build_default_pipeline()` import ba lớp cụ thể **trong thân hàm**. `stage_times` luôn đủ năm khoá ("`detect`", "`crop`", "`ocr`", "`normalize`", "`total`") — giai đoạn không chạy báo 0.0 thay vì vắng mặt; đây là thứ cho phép phân rã độ trễ (trên `best.pt`: `OCR ~64,3%`, `detect ~34,2%`) và cũng chính nó giúp phát hiện con số cũ "`93,3%`" là tạo tác của một lần đo trên hệ thống có lỗi `crop`. **Chính sách thất bại phân tầng**: ảnh không biển trả kết quả rỗng; OCR hỏng trên một biển thì biển đó `recognition=None`, biển khác vẫn xử lý; detector hỏng ném `DetectionError`; chuẩn hoá hỏng giữ nguyên kết quả thô và log. Cắt ảnh **kẹp lại lần hai** dù `BaseDetector` đã hứa — cắt là nơi duy nhất sai một đơn vị tạo mảng rỗng âm thầm; ảnh cắt là **bản sao**, không phải view, vì view sẽ ghim cả khung video trong bộ nhớ. `normalize_detailed(raw, line_count=...)` không thuộc `BaseNormalizer` nên pipeline dò bằng `getattr` và lùi về `normalize(raw)` nếu không có.

4.6.7. Nhận dạng họ biển và màu nền — `plate_color.py`

a) Vấn đề đã quan sát: thông tin được tính ra rồi bị vứt đi. Ảnh biển đỏ quân đội KV6938 được OCR đọc **đúng** ở độ tin cậy 0,999 nhưng giao diện hiển thị "**Sai định dạng biển số**" — sai về phát biểu chữ không sai về tính toán: biển quân đội là biển hợp lệ nằm ngoài hệ dân

sự (4.6.5f). Hai thông tin bị vớt trước khi tới CSDL: **họ biển** (PlateKind, chín giá trị từ car tới military/unknown) và **chuỗi hiển thị** do format_for_display dựng lại dấu phân cách (29E01566 → 29E-015.66). Bản sửa: **giữ lại** những gì đã tính (mục d, 4.7.2), và bổ sung nguồn bằng chứng mà chuỗi ký tự về nguyên tắc không thể mang — màu nền.

Bảng 4.4. Độ chính xác bộ nhận màu nền trên bộ dữ liệu ngoài hiệu chỉnh

Lớp nhãn người gán	Số ảnh	Đúng	Độ chính xác
Biển vàng	694	684	98,56%
Biển trắng	808	787	97,40%
Biển xanh	63	61	96,83%
Tổng	1.565	1.532	97,89%

4.7. Backend và cơ sở dữ liệu

4.7.1. Cấu trúc phân tầng backend và tầng nghiệp vụ

Backend gồm 21 mô-đun Python (19 ứng dụng + 2 Alembic) trong năm tầng với **luồng phụ thuộc một chiều nghiêm ngặt**; core/ được mọi tầng dùng nhưng không phụ thuộc tầng nào. Ba quy tắc: **router không viết truy vấn** — mọi truy cập qua repository, thay đổi lược đồ có bán kính ảnh hưởng một tệp; **repository flush, không bao giờ commit** — lưu một tác vụ cùng sáu biển là **một** thao tác logic, commit giữa chừng để lại trạng thái hỏng mà từng dòng riêng lẻ đều hợp lệ, ranh giới giao dịch thuộc tầng service; **khoá sắp xếp qua danh sách cho phép tường minh** — getattr(model, name) biến ?sort_by=metadata thành lỗi 500, ánh xạ tường minh biển khoá lạ thành 400 sạch sẽ; MAX_PAGE_SIZE = 200.

4.7.2. Cơ sở dữ liệu: lược đồ, di trú và các quyết định thiết kế dữ liệu

Cơ sở dữ liệu gồm hai bảng quan hệ một-nhiều: detection_job (một lần sử dụng hệ thống) và detection_history (mỗi biển số phát hiện được một bản ghi), nối bằng khoá source_job_id. Tách hai bảng là điều kiện để thống kê đếm đúng — *lượt nhận dạng* và *biển số phát hiện* là hai đại lượng khác nhau. detection_history hiện có **21 cột** sau ba lần di trú Alembic, trong đó hai quyết định đáng chú ý: lưu **song song** raw_ocr_text và plate_number để đo được đóng góp của khối hậu xử lý, và cột upper_char_count để giải nhập nhằng cách nhóm chữ số của biển hai dòng.

4.7.3. REST API

API kiểu REST, tự sinh OpenAPI 3.x và Swagger UI. Endpoint nghiệp vụ dưới tiền tố /api; health check đặt ở gốc để giám sát và Docker healthcheck không phụ thuộc phiên bản API. Đếm từ backend/api/routes/ đối chiếu OpenAPI: **10 thao tác HTTP trên 9 đường dẫn** (/api/history/{detection_id} mang cả GET và DELETE); /docs, /redoc, /openapi.json do FastAPI tự sinh, không tính. Bảng đặc tả đầy đủ ở **Phụ lục F.1**. Tóm tắt: GET /health (200); POST /api/detect/image (200); POST /api/detect/video (**202**); POST /api/detect/frame (200, kèm job_id tùy chọn); GET /api/jobs/{job_id} (200/404); GET

/api/history (200, các tham số lọc – sắp xếp – phân trang); GET /api/history/export (200 text/csv, UTF-8 **có BOM**, không phân trang); GET /api/history/{detection_id} (200/404/422); DELETE /api/history/{detection_id} (**204**); GET /api/statistics (200, tham số days). Lỗi chung: 400, 413, 422, 500.

4.7.4. UnavailablePipeline — một phương án lùi phải thất bại theo cách quan sát được

Giai đoạn chưa có mô hình, hệ thống chạy StubPipeline — bịa kết quả có cấu trúc hợp lệ, chính đáng lúc đó để xây API/CSDL/frontend. Vấn đề: **stub từng được cài làm phương án lùi khi không nạp được mô hình** — một triển khai cấu hình sai sẽ trả lời mọi lần tải lên bằng một biến số thuyết phục nhưng hư cấu: chế độ hỏng **trông giống thành công**, loại nguy hiểm nhất trong hệ thống có ghi CSDL. Ba lớp nay phân vai rõ: ALPRPipeline — nhận dạng thật, is_ready = True, /health ok; UnavailablePipeline — **mặc định khi hỏng**, ném ALPRError và **không bịa gì**, /health degraded; StubPipeline — **chỉ chạy khi ALPR_USE_STUB đặt tường minh**, /health degraded. Trọng số thiếu thì **dịch vụ vẫn khởi động** (tiến trình từ chối khởi động không nói được vì sao), mỗi yêu cầu trả lỗi sạch sẽ; build_pipeline log WARNING: *"Every result this process returns is invented."*

4.7.5. Xử lý lỗi, log có cấu trúc và request_id

Mỗi ngoại lệ mang **hai mô tả cho hai độc giả**: user_message tiếng Việt ngắn gọn có hành động, đi vào thân HTTP; internal_detail tiếng Anh kỹ thuật, chỉ đi vào log. Cây ngoại lệ APIError ánh xạ thẳng sang mã HTTP (400/404/413/415/500), và **bốn bộ xử lý được đăng ký** — trong đó một bộ *bắt tất cả* cho Exception, không có nó thì ngoại lệ ngoài dự kiến ở cấu hình debug sẽ hiển thị cả stack trace (NFR-S4). Log ghi **mỗi dòng một đối tượng JSON** kèm request_id truyền ngầm qua ContextVar — log video xen kẽ log tải lên đồng thời, văn bản thuần không tách lại được.

4.7.6. Ba lỗi thực tế đã gặp và sửa trong quá trình cài đặt

Ba lỗi đáng ghi nhận đã gặp khi cài đặt: pydantic-settings JSON-decode trường danh sách **trước** validator khiến dịch vụ sập lúc khởi động; SQLite âm thầm nuốt tzinfo khiến mọi phân tích theo thời gian sai lệch mà không gì trông sai; và log tiếng Việt làm sập console cp1252 trên Windows — sự cố xảy ra *bên trong* cỗ máy logging, đúng lúc log quan trọng nhất. **Cả ba đều đi qua được kiểm thử đơn vị**, vì cả ba nằm ở ranh giới mã–môi trường (nguồn cấu hình, tầng lưu trữ, bảng mã đầu ra) — lập luận cụ thể cho việc bộ kiểm thử phải gồm kiểm thử tích hợp chạy trên đường dẫn thật.

4.8. Giao diện người dùng

4.8.1. Cấu trúc, điều hướng và các màn hình

Giao diện là SPA React + TypeScript dựng bằng Vite, gồm **3 trang** và 48 mô-đun .tsx/.ts: pages/ (ImageDetection ở trang chủ /, VideoDetection /video, History /history); components/ui/ 15 component nguyên thủy; các nhóm component detection/history;

services/api.ts, types/index.ts, hooks/, lib/. Điều hướng cố ý **phẳng**: ba màn hình truy cập trực tiếp từ thanh điều hướng; chi tiết bản ghi và xác nhận xoá là hộp thoại chồng lên trang lịch sử để không mất ngữ cảnh bộ lọc.

4.8.2. Tầng gọi API và ánh xạ kiểu dữ liệu

services/api.ts là **nơi duy nhất frontend biết về axios hoặc mã HTTP**: component nhận dữ liệu đã có kiểu hoặc ApiError chuẩn hoá. Sáu hàm gọi API ứng một-một với sáu endpoint, cộng hai hàm dựng URL. **Ba endpoint còn lại không còn hàm gọi phía giao diện nhưng vẫn hoạt động ở backend** — cần phân biệt *hàm gọi bị xoá* với *endpoint thì không*. Không hostname nào viết cứng: origin đọc từ biến môi trường lúc build, mặc định rỗng (cùng-origin).

4.8.3. Nguyên tắc trải nghiệm người dùng

Bốn trạng thái bắt buộc của mọi thành phần hiển thị dữ liệu: *đang tải* (thiếu thì giao diện trông như treo, người dùng bấm lại tạo thêm tải); *có dữ liệu*; *rỗng* (thiếu thì màn hình trắng không phân biệt được với lỗi); *lỗi* (tiếng Việt, nêu nguyên nhân và cách khắc phục, có thử lại). Trạng thái rỗng xuất hiện với **ba nghĩa cần ba thông điệp**: chưa có lượt nhận dạng nào; bộ lọc không khớp; ảnh không chứa biển số — nghĩa thứ ba là biểu hiện giao diện của cùng quyết định ở tầng API (200 danh sách rỗng) và tầng pipeline: **không tìm thấy không phải là lỗi**.

4.8.4. Hàng đợi một khe ở client thời gian thực (trang webcam đã gỡ 2026-07-20)

Trang webcam đã gỡ khỏi frontend, nhưng lập luận thiết kế của nó vẫn đúng và trở thành **khuyến nghị bắt buộc cho bất kỳ client nào** gọi POST /api/detect/frame. Suy luận CPU chỉ ~5 FPS, nên một bộ đếm giờ ngây thơ sẽ khởi động yêu cầu thứ hai trước khi yêu cầu thứ nhất trở về — tồn đọng chỉ tăng và tab đứng hình. Giải pháp là **giữ đúng một yêu cầu đang bay**; khung tới trong lúc khe bận thì bị **bỏ qua chứ không xếp hàng**: bỏ một khung không tốn gì vì khung sau cập nhật hơn, xếp hàng thì tốn tất cả.

4.9. Triển khai bằng Docker

Đóng gói phục vụ NFR-C1: môi trường chạy tái lập được, không phụ thuộc máy cá nhân. Dockerfile.backend build hai giai đoạn, cài **hai tệp requirements thành hai lớp riêng** để thay đổi một tầng không mất bộ đệm tầng kia (hệ quả trực tiếp của 4.3.2), chạy dưới người dùng không đặc quyền, đặt OMP_NUM_THREADS tường minh để hai container không cạnh tranh nhân CPU đến mức cùng chậm, và HEALTHCHECK có start-period đủ dài cho việc nạp trọng số. Dockerfile.frontend build rồi phục vụ tĩnh bằng nginx:alpine — ảnh chạy không chứa Node hay mã nguồn. **Trọng số mô hình không nằm trong ảnh Docker** mà gán từ ngoài, cùng một volume riêng cho bộ đệm mô hình PaddleOCR — không có volume này thì mỗi lần down && up phải tải lại vài trăm MB và không có mạng thì container không khởi động được. Bảng biến môi trường ở **Phụ lục F.2. Trạng thái kiểm chứng**: docker compose config hợp lệ; đo hiệu năng trong container thuộc Chương 5.

4.10. Những chỗ cài đặt lệch khỏi thiết kế, và lý do

Nguyên tắc: **mọi điểm lệch đều được nêu, kể cả những điểm chưa được giải quyết.**

Bảng 4.5. Tổng hợp chín điểm lệch giữa thiết kế và cài đặt

#	Thiết kế	Cài đặt thực tế	Loại lệch	Trạng thái
1	StubPipeline là phương án lùi khi thiếu mô hình	UnavailablePipeline là phương án lùi; stub chỉ chạy khi opt-in tường minh	Cải tiến so với thiết kế	Đã giải quyết
2	Một môi trường ảo Python	Ba môi trường ảo tách biệt	Bắt buộc bởi xung đột phụ thuộc	Đã giải quyết
3	FR-2.4: có nút huỷ tác vụ video	Nút hiện diện nhưng bị vô hiệu hoá ; không có endpoint huỷ	Đạt một phần	⚠ Chưa xong
4	Mô hình chính thức <code>imgsz=640</code> trên split sạch	Đã có <code>models/best.pt</code> (<code>imgsz=640</code> , <code>split v3</code> , mAP@0.5 0,9829)	Đúng thiết kế	✅ Đã giải quyết
5	NFR-P1: độ trễ E2E $p95 \leq 800$ ms	Đo được 1.143,10 ms — dưới sàn 1.500 ms nhưng vượt mục tiêu 800 ms	🟡 Chỉ đạt sàn	⚠ Chưa đạt mục tiêu
6	Video job xuất video đã chú thích (<code>output_path</code>)	Chưa cài đặt; chỉ trả về các dòng lịch sử	Hoãn có lý do	⚠ Chưa xong
7	Bật oneDNN để tăng tốc CPU	Buộc phải tắt do lỗi thư viện	Bắt buộc bởi lỗi thượng nguồn	Đã ghi nhận
8	Khử rò rỉ bằng phash	Còn rò rỉ tồn dư không khử được bằng phash	Giới hạn phương pháp	Đã ghi nhận
9	Bộ đo độ chính xác OCR đo hệ thống đang giao	Bộ đo gọi thẳng recognizer + normalizer, bỏ qua tầng điều phối	Lỗi phương pháp đo	✅ Đã phát hiện và sửa

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương 5 trình bày hệ thống đã được xây dựng như thế nào. Chương này trả lời hai câu hỏi trọng số ngang nhau: hệ thống đó **hoạt động tốt đến mức nào**, và **những con số đó có đáng tin không** — nên mỗi con số đều đi kèm ngữ cảnh đo của nó.

5.1. Mục tiêu và phương pháp đánh giá

5.1.1. Các câu hỏi nghiên cứu mà chương này trả lời

Sáu câu hỏi cụ thể hoá từ docs/00-requirements/non-functional-requirements.md.

RQ1 — YOLO11n có đạt chỉ tiêu định vị biển số Việt Nam không (5.5; NFR-A1...A3; 5.4)?

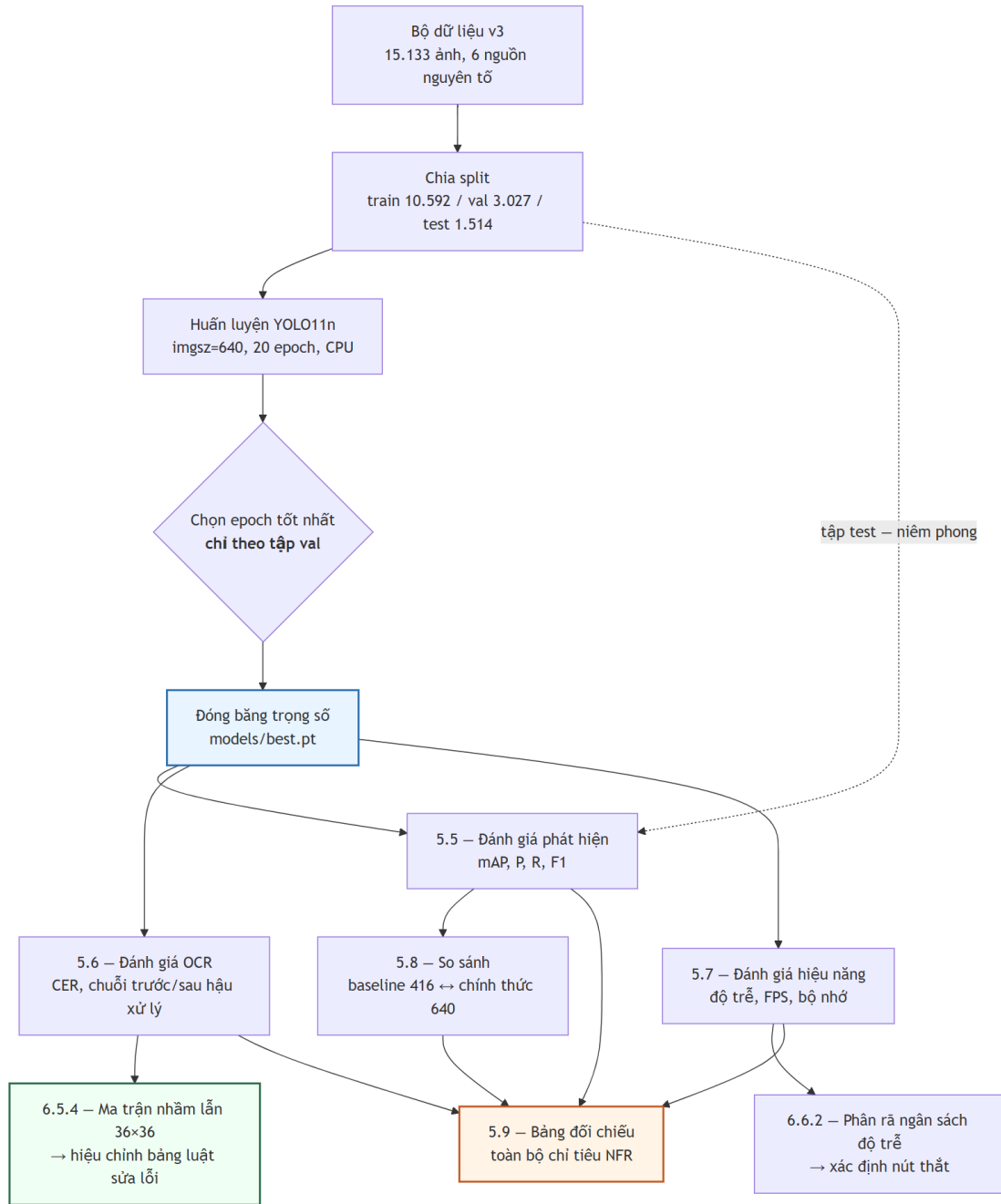
RQ2 — độ chính xác nhận dạng chên bao nhiêu giữa biển một dòng và hai dòng (NFR-A8; 5.4.2, 5.5.3)? **RQ3** — khối hậu xử lý theo luật đóng góp bao nhiêu điểm phần trăm vào độ chính xác chuỗi đầy đủ (NFR-A5 ↔ A6; 5.5.2)? **RQ4** — có đạt chỉ tiêu độ trễ trên phần cứng CPU-only không, nút thắt ở đâu (5.7; NFR-P1...P7; 5.6)? **RQ5** — bảng luật sửa lỗi ký tự, vốn suy từ hình dạng chữ chứ không từ đo đạc, có khớp các cặp thực sự bị nhầm không (5.5.4; VNPLATE §9.8)? **RQ6** — số liệu chịu mỗi đe dọa nào đến tính hợp lệ (5.9.3)? RQ3 và RQ5 mang đóng góp học thuật riêng — một lượng hoá khối chức năng mà tài liệu ALPR chỉ mô tả định tính, một thay tri thức suy đoán bằng tri thức đo được; RQ6 quyết định giá trị của năm câu còn lại.

5.1.2. Hai nguyên tắc trình bày bắt buộc

Một — mọi số hiệu năng phải kèm cấu hình phần cứng: đề án suy luận hoàn toàn trên CPU nên so với các con số FPS đo trên GPU là không hợp lệ nếu không ghi rõ; cấu hình ở 5.2 là điều kiện diễn giải cho toàn mục 5.6. **Hai** — mọi số độ chính xác phải kèm tên tập dữ liệu và số mẫu, vì độ chính xác là thuộc tính của cặp (mô hình, tập đánh giá); hệ quả: NFR-A4...A7 chỉ đo được trên tập con có nhãn chuỗi, nhỏ hơn nhiều tập test phát hiện, mẫu số đó không được giấu. **Ký hiệu:**  đạt mục tiêu ·  chỉ đạt ngưỡng tối thiểu ·  không đạt ·  chưa đo ·  không áp dụng.

Cảnh báo trích dẫn. Cặp số **94,3%** (biển một dòng) ↔ **45,7%** (biển hai dòng), chên **48,6 điểm phần trăm**, đo trên bộ dữ liệu **RodoSol-ALPR của Brazil** [7]. Đây **không phải** số liệu Việt Nam; mỗi lần dẫn, tên bộ dữ liệu và quốc gia phải xuất hiện **ngay trong câu**, và nó chỉ dùng như *analogue định lượng* về độ khó tương đối của biển hai dòng, không bao giờ như mốc chuẩn phải vượt.

5.1.3. Giao thức đo



Hình 5.1. Giao thức đo và ràng buộc phụ thuộc giữa các bước đánh giá.

Không bước đo nào chạy trước khi trọng số được đóng băng; tập test không được chạm vào trong huấn luyện lẫn chọn epoch — chọn epoch chỉ dựa vào validation. Ba quy ước: **kích thước lô = 1 khi đo độ trễ** (riêng mAP dùng lô lớn hơn vì không phụ thuộc kích thước lô); **bỏ 3 lượt khởi động nóng**; **báo cáo p50/p95/p99, không báo cáo trung bình**, vì trung bình che đuôi phân bố còn NFR-P1 phát biểu ở p95.

5.2. Môi trường thực nghiệm

Toàn bộ số liệu đo trên **một máy trạm cá nhân duy nhất** (docs/00-requirements/environment.md): **Windows 11 Pro 10.0.26200, Python 3.13.12**, CPU Intel **Raptor Lake** (Family 6, Model 183) — **14 nhân vật lý / 20 nhân logic, không có GPU CUDA** nên mọi suy luận và huấn luyện chạy trên CPU (device=cpu); chế độ đo **lô = 1, bỏ 3 lượt khởi động nóng**. Đây là **tiền tố ngầm định của mọi con số hiệu năng ở 5.6**.

Phiên bản thư viện trích từ pip freeze đúng thời điểm chạy phép đo cuối cùng (2026-07-20), không chép từ requirements.txt (tệp yêu cầu ghi *ràng buộc*, không ghi *phiên bản đã cài*); một môi trường ảo hợp nhất backend/.venv: ultralytics 8.4.101 · torch 2.13.0+cpu · torchvision 0.28.0+cpu · paddleocr 3.7.0 (PP-OCRv5 mobile) · paddlepaddle 3.3.1 · onnxruntime 1.27.0 · opencv-python 4.10.0.84 · numpy 2.4.5 · fastapi 0.139.2 · uvicorn 0.51.0 · sqlalchemy 2.0.51 · imagehash 4.7.2 · pytest 9.1.1.

5.2.1. Vì sao ràng buộc CPU-only là ràng buộc thiết kế, không phải hạn chế tạm thời

Lập luận đầy đủ ở **4.3.1**. NFR-P1 phát biểu *kèm* ràng buộc CPU, nên kết luận "đạt sàn, không đạt mục tiêu" ở 5.6 là kết luận về hệ thống trong đúng bối cảnh vận hành thật, không phải con số chờ nâng cấp phần cứng. Cấu hình mô hình cũng do ràng buộc phần cứng quyết định — **YOLO11n** (2.590.035 tham số) [16] và **PP-OCRv5 mobile** [118]. Về quy mô: **35,6 phút mỗi epoch**, một lượt 20 epoch mất khoảng **12 giờ** liên tục, khiến **tìm kiếm siêu tham số bất khả thi**; chương này báo cáo *một* cấu hình huấn luyện, không phải kết quả của một quá trình tối ưu — giới hạn thật, ghi ở 5.9.3.

5.3. Bộ dữ liệu thực nghiệm

Lưu ý về cách đặt tên. Thư mục yolo_v2/ và yolo_v3/ chỉ **phiên bản của bộ dữ liệu**, **không** liên quan đến kiến trúc YOLOv2 hay YOLOv3. Mô hình dùng trong toàn đề án là **YOLO11n**.

Bảng 5.1. Ba phiên bản bộ dữ liệu và số cặp ảnh gần trùng xuyên split theo ngưỡng Hamming

Thuộc tính / ngưỡng	v1	v2	v3
Tổng số ảnh	4.578	15.133	15.133
Ngưỡng Hamming gộp trùng lặp	5	5	10
Số ảnh train / val / test	—	—	10.592 / 3.027 / 1.514
Dùng cho	baseline-416-	bị loại	best.pt

Thuộc tính / ngưỡng	v1	v2	v3
	v1.pt	bỏ	
Cặp gần trùng xuyên split, Hamming 0	—	—	0 (thông tin mới)
Hamming 5	—	—	0 (= ngưỡng gộp v1, v2 — không mang thông tin mới)
Hamming 10	619	2.699	0 (= ngưỡng gộp v3 — không mang thông tin mới)
Hamming 12 · 15	—	—	791 · 3.529 (thông tin mới)
Hamming 20	—	—	137.506 (ngưỡng đã lỏng tới mức nhiều cặp là dương tính giả)

v1 quá nhỏ và chỉ một nguồn (1 bộ vào hợp nhất, 1 nguồn nguyên tố) — động cơ tải thêm **tám bộ** (tổng **9 bộ**), trong đó **sáu bộ** vào hợp nhất detection cùng bộ gốc (v2, v3: **7 bộ vào hợp nhất, 6 nguồn nguyên tố**), hai bộ nhãn mức ký tự tách riêng cho OCR. **v2 sửa được quy mô nhưng không sửa được rò rỉ**: lên 15.133 ảnh lại *tăng* cặp gần trùng xuyên split lên 2.699 vì các nguồn chứa ảnh có nguồn gốc chung. **v3 giữ nguyên corpus** (cùng 15.133 ảnh), khác biệt duy nhất là ngưỡng khử trùng lặp 5 → 10 và split sinh lại — cô lập biến có chủ ý, cho phép quy kết mọi thay đổi kết quả cho *chất lượng split*, không cho *lượng dữ liệu*.

5.3.1. Khử trùng lặp: hai tỉ lệ, hai mẫu số khác nhau

Có **hai tỉ lệ khử trùng lặp trên hai mẫu số khác nhau**: **44,2%** (11.978/27.111, trước hợp nhất, trên 7 bộ vào hợp nhất detection) và **47,8%** (7.227/15.133, sau hợp nhất). Hai số **không cộng dồn và không thay thế nhau**; cơ chế và cách đọc trình bày ở mục 4.4.2. Điểm phải nhớ khi trích: mẫu số 27.111 là tổng ảnh của **7 bộ vào hợp nhất detection, không phải 9 bộ đã tải** — hai bộ còn lại mang **nhãn mức ký tự**, tách riêng cho tăng OCR.

5.3.2. Kiểm chứng rò rỉ dữ liệu — và vì sao con số "0 cặp rò rỉ" không chứng minh được điều gì

Rò rỉ xảy ra khi tập test chứa ảnh gần trùng ảnh train: mô hình *ghi nhớ* thay vì *tổng quát hoá*, mọi chỉ số bị thổi phồng — với corpus ghép từ nhiều nguồn công khai đây là rủi ro hệ thống [7]. Công cụ đo là **băm tri giác** (imagehash.phash, 64 bit).

Lập luận vòng tròn. v3 được khử trùng lặp ở ngưỡng Hamming 10; đo rò rỉ ở đúng ngưỡng đã dùng để gộp trùng lặp là **kiểm tra lại chính định nghĩa của mình**, không phải kiểm chứng độc lập. Kết quả bằng 0 ở đó chứng minh bước khử trùng lặp *đã chạy đúng đặc tả*, và **không chứng minh gì thêm** về mức độ "sạch" của tập test.

Chỉ các ngưỡng **12 (791 cặp), 15 (3.529 cặp), 20 (137.506 cặp)** mang thông tin mới, và ngay cả chúng cũng **không** chứng minh tập test sạch. **Ba giới hạn của phash**: (1) phash chỉ bắt tương đồng ở mức **bố cục sáng-tối tổng thể** — hai ảnh cùng *một chiếc xe* ở hai góc khác

n nhau, hay hai khung hình cách nhau vài giây trong cùng video, vẫn mang **cùng một biển số** dù Hamming lớn; loại rò rỉ ngữ nghĩa này **không khử được bằng bất kỳ ngưỡng phash nào**; (2) **không có định danh phương tiện hay chuỗi biển cho toàn corpus** nên không chia split theo **nhóm biển số** được — chính hạn chế dẫn tới mẫu số nhỏ của các bảng OCR ở 5.5; (3) **ngưỡng cao sinh dương tính giả**, nên 137.506 là **cận trên bi quan**. **Kết luận trung thực**: khẳng định được *bước khử trùng lặp ở ngưỡng 10 đã chạy đúng đặc tả*; **không** khẳng định được *tập test độc lập với tập train*. Rò rỉ tồn dư ở mức ngữ nghĩa **không đo được bằng công cụ hiện có** — mối đe dọa đầu tiên ở 5.9.3; mọi chỉ số ở 5.4 phải đọc kèm ghi chú này.

5.3.3. Phân bố nguồn dữ liệu giữa các split

Toàn tập chia **15.133 = 10.592 / 3.027 / 1.514**, tức **70,0% / 20,0% / 10,0%**. Hai điểm phải nêu khi đọc mọi kết quả của chương. **Thứ nhất, tập test nghiêng về ảnh camera giao thông** — nguồn roboflow_traffic_camera có **20,3%** số ảnh rơi vào test, gấp đôi tỉ lệ tổng thể 10,0% — nên khi đọc mAP theo dải kích thước (5.4.3) phải nhớ rằng đối tượng nhỏ trong tập test tập trung ở một nguồn. **Thứ hai, một bộ dữ liệu dư thừa hoàn toàn**: roboflow_tran_ngoc_xuan_tin vào hợp nhất với 1.005 ảnh và ra khỏi khử trùng lặp chéo bộ với **0 ảnh** — **loại 100,0%**, bằng chứng định lượng cho việc các bộ Roboflow tái sử dụng ảnh của nhau rất nặng và là lý do **không được cộng dồn expected_images để suy ra quy mô thật**.

5.4. Đánh giá bộ phát hiện biển số

Toàn bộ 5.4 đo trên **tập test v3: 1.514 ảnh, 1.611 đối tượng nhân thật**, imgsz=640, device=cpu; tập test chưa từng dùng trong huấn luyện hay chọn epoch.

5.4.1. Chỉ số tổng thể

Cả bốn chỉ tiêu bắt buộc đều đạt mục tiêu, đo trên ultralytics_val: **mAP@0.5 = 0,9829** (NFR-A1; sần 0,85, mục tiêu 0,90 ) , **mAP@0.5:0.95 = 0,7834** (NFR-A2; sần 0,55, mục tiêu 0,65 ) , **Precision = 0,9837** và **Recall = 0,9714** (NFR-A3; sần 0,88 / 0,85, mục tiêu 0,92 / 0,90 ) , **F1 = 0,9775** tại ngưỡng confidence 0,25; đối chiếu ở Bảng 5.10. Ba lưu ý: (1) **bài toán chỉ có một lớp** (plate), mAP một lớp không so trực tiếp được với mAP nhiều lớp trên COCO nên giá trị cao là bình thường, **không phải bằng chứng về độ khó đã vượt qua**; (2) chỉ số quyết định là **mAP@0.5:0.95** vì độ khớp box ảnh hưởng trực tiếp đến chất lượng vùng cắt đưa sang OCR; (3) **chỉ số tổng thể che giấu phân bố**.

5.4.2. Tách theo biển một dòng và biển hai dòng (NFR-A8)

Layout xác định theo nhãn lớp khi bộ dữ liệu có khai báo, và theo **ngưỡng tỉ lệ khung hình 2,5** khi không có — nằm giữa tỉ lệ chuẩn của biển một dòng (4,727) và biển hai dòng (2,000 / 1,357) theo QCVN 08:2024/BCA.

Bảng 5.2. Kết quả phát hiện tách theo biển một dòng và hai dòng (NFR-A8)

Chỉ số	Biển một dòng	Biển hai dòng	Chênh (điểm %)
Số đối tượng nhận thật (<i>tổng 1.611</i>)	286	1.325	n/a
mAP@0.5	0,9884	0,9675	2,09
mAP@0.5:0.95	0,7526	0,7649	-1,23
Precision · Recall · F1	0,9861 · 0,9895 · 0,9878	0,9735 · 0,9691 · 0,9713	1,26 · 2,04 · 1,65

5.4.3. Tách theo dải kích thước hộp giới hạn

Mục này tồn tại vì **bộ dữ liệu không đạt tiêu chí chất lượng Q6: 10,91% số hộp có diện tích dưới 0,5% diện tích ảnh**, vượt ngưỡng 10%. Đối tượng nhỏ là chế độ thất bại đã ghi nhận rộng rãi của bộ phát hiện một giai đoạn [119], biến số độ phân giải thấp đã thành hướng nghiên cứu riêng [71]; một con số mAP tổng sẽ **giấu chế độ thất bại sau giá trị trung bình**.

Bảng 5.3. Kết quả phát hiện tách theo dải kích thước hộp giới hạn

Dải (diện tích box / diện tích ảnh)	Số đối tượng	Tỉ lệ tập test	mAP@0.5	mAP@0.5:0.95	Recall
Rất nhỏ — dưới 0,5%	262	16,26%	0,8553	0,5249	0,8740
Nhỏ — 0,5% đến 1%	124	7,70%	0,9755	0,7055	0,9758
Trung bình — 1% đến 5%	900	55,87%	0,9913	0,8005	0,9922
Lớn — 5% đến 15%	297	18,44%	0,9966	0,8394	0,9966
Rất lớn — trên 15%	28 $\triangle$	1,74%	1,0000	0,8562	1,0000
Toàn tập test	1.611	100%	0,9711	0,7625	0,9727

Dòng $\triangle$ (28 đối tượng < 30) **không có ý nghĩa thống kê**, không đưa vào so sánh.
Dải tính từ ($w \times h$) của hộp nhận thật chia diện tích ảnh gốc; số box không gán được dải: 0.

5.5. Đánh giá khối OCR và hậu xử lý

Ràng buộc mẫu số. NFR-A4...A7 chỉ đo được trên **tập con có nhãn chuỗi ký tự** (datasets/annotations/plate_labels.csv) — **2.801 biển**, không phải trên 1.514 ảnh test. Thiếu nhãn chuỗi cho phần lớn corpus là hạn chế thật, ghi ở 5.9.3.

5.5.1. Độ chính xác mức ký tự (NFR-A4)

Chỉ số là **CER** theo khoảng cách Levenshtein, chuẩn hoá theo độ dài nhãn thật, với S ký tự thay thế, D bị xoá, I chèn thừa, N tổng ký tự nhãn thật; NFR-A4 phát biểu theo **1 – CER**.

5.5.2. Chuỗi đầy đủ: trước và sau hậu xử lý (NFR-A5 ↔ NFR-A6)

Đây là mục quan trọng nhất của cả chương. Khối hậu xử lý theo luật — chuẩn hoá ký tự, áp mặt nạ vị trí chữ số / chữ cái theo định dạng biển số Việt Nam, sửa cặp ký tự đồng hình, kiểm tra mã tỉnh — là **đóng góp kỹ thuật riêng** của đồ án, không có sẵn trong thư viện nào. Câu hỏi *khối đó đóng góp bao nhiêu?* chỉ trả lời được nếu **đo hai lần trên cùng một tập mẫu**: chuỗi thô PaddleOCR trả về (A5) và chuỗi sau khi áp toàn bộ luật (A6). Đây cũng là lý do kỹ thuật khiến trường `raw_ocr_text` tồn tại bên cạnh `plate_text` trong lược đồ cơ sở dữ liệu — **điều kiện cần để phép đo này thực hiện được**, phải có từ giai đoạn thiết kế.

Bảng 5.4. Độ chính xác OCR trước và sau hậu xử lý, trên 2.801 biển có nhãn chuỗi

Chỉ số	Sàn	Mục tiêu	Trước hậu xử lý	Sau hậu xử lý	Chênh (điểm %)
1 – CER (NFR-A4)	0,92	0,95	0,9061	0,9454	n/a
CER	≤ 0,08	≤ 0,05	0,0939	0,0546	n/a
Chuỗi đầy đủ đúng (A5 → A6)	0,80 → 0,85	0,85 → 0,90	0,6373 ✖	0,7512 ✖	+11,39
<i>N / S / D / I</i> trên chuỗi thô	—	—	23.855 / 862 / 1.272 / 107	(không đổi)	n/a
Biển sửa đúng / bị làm hỏng / sai cả trước lẫn sau	—	—	—	319 / 0 / 697	n/a

5.5.3. Tách theo biển một dòng và hai dòng cho OCR

Bảng 5.5. Độ chính xác nhận dạng tách theo biển một dòng và hai dòng

Chỉ số	Biển một dòng	Biển hai dòng	Chênh (điểm %)
Số mẫu có nhãn chuỗi (<i>tổng 2.801</i>)	567	2.234	n/a
1 – CER (NFR-A4)	0,9925	0,9344	5,81
Chuỗi đúng trước hậu xử lý (A5)	0,9418	0,5600	38,18
Chuỗi đúng sau hậu xử lý (A6)	0,9541	0,6996	25,45
Cải thiện do hậu xử lý (A6 – A5)	+1,23	+13,97	n/a
Độ chính xác E2E (A7)	0,6861	0,5219	—

Đây là kết quả khoa học quan trọng nhất của chương. Chênh lệch mà tăng phát hiện gần như che khuất (2,09 điểm, Bảng 5.2) nay lộ ra ở tầng OCR với **biên độ khác hẳn cấp**: 5,81 điểm ở mức ký tự, **25,45 điểm** ở A6, **38,18 điểm** ở A5. **Biển một dòng về cơ bản đã giải xong** (A6 = 0,9541 vượt cả mục tiêu 0,90), toàn bộ việc "OCR không đạt" là do **biển hai dòng kéo xuống**; vì biển hai dòng chiếm **2.234 / 2.801 = 79,8%** tập có nhãn chuỗi (phản ánh tỉ lệ xe máy rất cao ở Việt Nam), con số tổng bị quần thể khó này chi phối. **25,45 điểm là con số sau khi đã áp cả hai bậc cứu chữa** (5.5.6, 5.5.7): ở lượt 20/07, A6 biển hai dòng

là 0,5810 và khoảng cách là 36,79 điểm — chuỗi biện pháp đã thu hẹp **11,34 điểm**, một dịch chuyển thật nhưng vẫn để lại một phần tư khoảng cách; phần còn lại nằm ở **năng lực nhận dạng ký tự**, không ở khâu cắt/ghép hay hình học, vì cả hai khâu sau đã xử lý và đo tách bạch.

5.5.4. Ma trận nhầm lẫn ký tự 36×36

Bảng 5.6. Các cặp ký tự bị nhầm nhiều nhất, đối chiếu bảng luật hiện hành

Hạng	Ký tự thật → bị đọc thành	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ không?
1	L → 1	90	10,44%	có (TO_DIGIT) — đúng chiều
2	E → F	73	8,47%	không
3	4 → L	53	6,15%	không
4	U → 1	38	4,41%	không
5	D → 0	34	3,94%	có (TO_DIGIT) — đúng chiều
6	Z → 7	32	3,71%	không
7	2 → 7	26	3,02%	không
8	X → Y	21	2,44%	không
9	B → R	20	2,32%	không
10	9 → 0	19	2,20%	không

5.5.5. Độ chính xác đầu-cuối toàn trình (NFR-A7)

NFR-A7 đo **ảnh đầu vào → phát hiện → cắt → OCR → hậu xử lý → chuỗi cuối**; khác A6 ở chỗ A6 đo trên **vùng biển cắt chuẩn theo nhãn thật** còn A7 đo trên vùng biển do **chính bộ phát hiện** tìm ra, nên A7 tích lũy cả hai nguồn sai số và theo lý thuyết luôn $\leq$ A6. Trên 2.801 mẫu: **A7 = 0,5552** (sàn 0,82, mục tiêu 0,88 — **✗**); **E2E với điều kiện đã phát hiện được biển = 0,6306**; tỉ lệ biển **bỏ sót** ở tầng phát hiện **0,1196**; phát hiện đúng nhưng **đọc sai chuỗi 0,3694**; chênh **A6 – A7 = 19,60 điểm**.

⚠ **Cảnh báo hiệu lực — phải đọc trước khi diễn giải A7.** Con số 0,5552 đo trên **ảnh crop biển số**, không phải ảnh hiện trường, vì không bộ dữ liệu nào trong đề án có đồng thời ảnh toàn cảnh và chuỗi biển số nhãn thật. Bộ phát hiện được huấn luyện trên ảnh giao thông đầy đủ nên ảnh chỉ có biển số chiếm gần hết khung là **ngoài phân bố huấn luyện**: phần lớn thất bại ở đây do bộ phát hiện không bắt được box trên ảnh crop (bỏ sót 11,96%), **không phải do OCR đọc sai**. Bằng chứng: trên ảnh hiện trường thật, Phase 7 đo **mAP@0.5 = 0,9829**, tương thích với 5.4.1. Muốn đo NFR-A7 đúng cách cần gán nhãn chuỗi cho một phân bố test có ảnh hiện trường (ví dụ một phần yolo_v2) — **việc này chưa làm**.

5.5.6. Bước "cứu dòng trên" cho biển hai dòng — thiết kế, kiểm chứng A/B và kết quả trên toàn tập

Hồ sơ lỗi thiên về *xoá* ($D = 1.272 > S = 862$) trên biển hai dòng có cách giải thích tự nhiên: **mất hẳn một dòng**. Hệ thống đọc biển hai dòng bằng **ghép-rời-đọc** (*split-then-hstack*): cắt vùng biển thành hai nửa chồng lấn, xếp cạnh nhau thành dải ngang, chạy OCR **một lần**. Phương án thay thế — đọc riêng từng nửa rồi nối chuỗi — đã được đo A/B chứ không bị loại bằng lập luận, trên 200 biển hai dòng (seed = 20260720, docs/reports/15-two-line-ab.json): **A, ghép rời OCR một lần** (*đang dùng*) đúng $129/200 = 64,50\%$, 2 ca OCR trả chuỗi rỗng, 340,11 ms; **B, OCR từng nửa rồi nối** đúng $7/200 = 3,50\%$, 9 ca chuỗi rỗng, 391,35 ms — B kém A **61,00 điểm phần trăm** và đắt hơn **51,24 ms**; **122** ca A thắng B, **0** ca B thắng A. **Giả thuyết "đọc riêng từng dòng thì chính xác hơn" bị bác bỏ dứt khoát**. Nguyên nhân đọc được ngay trong dữ liệu: hai nửa **cố ý chồng lấn** để không cắt cụt ký tự, nên đọc riêng thì dải chồng lấn bị đọc **hai lần** và ký tự bị nhân đôi — 84G122593 thành 84-G124E009.01225.93. Một kết quả âm có giá trị: lựa chọn kiến trúc ở Chương 5 không tùy tiện.

⚠ **Số liệu toàn tập của riêng bước cứu là số lịch sử 20/07, giữ nguyên mốc.**

Trên 2.801 ảnh có nhãn chuỗi (16-ocr-accuracy-rescued.json, best.pt, imgs = 640; cột "trước" là lượt đo trước đó trên đúng cùng tập mẫu và cùng mô hình):
A4 0,8734 → **0,8848 (+1,14; sần 0,92 ✗)**; A5 0,6098 → **0,6098 (0,00; sần 0,80 ✗)**;
A6 0,6555 → **0,6730 (+1,75; sần 0,85 ✗)**; A7 0,5227 → **0,5295 (+0,68; sần 0,82 ✗)**;
A6 riêng biển **một dòng** 0,9489 → **0,9489 (0,00)**; A6 riêng biển **hai dòng** 0,5810 → **0,6030 (+2,20)**; biển bị can thiệp / thành đúng hoàn toàn / bị làm hỏng = **89 / 2.801 · 49 · 0**. Bảng số này trả lời đúng một câu hỏi — *riêng bước cứu dòng trên đóng góp bao nhiêu* — và câu trả lời ấy không đổi theo thời gian; nhưng **các giá trị tuyệt đối đã bị vượt qua** (A6 hiện là 0,7512 chứ không phải 0,6730) nên **không được trích cột "sau bước cứu" như số hiện hành**. Trên lượt 28/07, bước cứu dòng trên cho câu trả lời cuối ở **209 biển**.

5.5.7. Bậc thang thử-lại cho biển nghiêng/méo — chi phí, lợi ích và một quyết định tắt tính năng

5.6. Đánh giá hiệu năng

Mọi số trong 5.6 phải đọc cùng cấu hình ở 5.2: **Intel Raptor Lake 14 nhân / 20 luồng, Windows 11, CPU-only, kích thước lô = 1**.

5.6.1. Độ trễ đầu-cuối (NFR-P1)

Bảng 5.7. Độ trễ đầu-cuối một ảnh, đối chiếu NFR-P1

Chỉ số	Sần	Mục tiêu	Trước bậc thang (20/07)	Cấu hình giao hàng (28/07)	Kết quả
p50 (ms)	—	—	414,67	405,77	n/a

Chỉ số	Sàn	Mục tiêu	Trước bậc thang (20/07)	Cấu hình giao hàng (28/07)	Kết quả
p95 (ms)	≤ 1500	≤ 800	731,15	1.143,10	
p99 (ms)	—	—	947,83	1.420,07	n/a
Trung bình (ms)	—	—	400,74	447,38	n/a
Số ảnh đo	—	—	100	100	n/a
Bội số so với sàn / mục tiêu	—	—	0,49× / 0,91×	0,76× / 1,43×	n/a

NFR-P1 đạt ngưỡng tối thiểu nhưng không đạt mục tiêu, và đây là **thoái lui có chủ ý và đã định lượng**: tắt hẳn bậc thang thử-lại đưa p95 về **866,3 ms**, tức toàn bộ **+277 ms** là của nó; nhưng vì bậc thang chỉ chạy **sau khi lần đọc đầu thất bại** nên nó không chạm vào trường hợp thường — **trung vị thậm chí giảm nhẹ** (414,67 → 405,77 ms), chi phí dồn hết vào đuôi. Với hệ thống mà 95% ảnh xong dưới 1,15 giây và một nửa xong dưới 0,41 giây, p50 mô tả trải nghiệm điển hình còn p95 mô tả trường hợp xấu; NFR-P1 phát biểu theo p95 nên kết luận chính thức là **đạt sàn, không đạt mục tiêu**. Ở lượt đo đầu với cả ba biến thể bật, p95 là **1.514,26 ms** — **vượt cả ngưỡng tối thiểu**; bóc tách chỉ ra bậc siêu phân giải chiếm hơn nửa chi phí đó mà không mua được biến nào đo được nên nó bị **tắt mặc định**, đưa p95 về 1.143,10 ms.

5.6.2. Phân rã ngân sách độ trễ theo từng bước

Bảng 5.8. Phân rã ngân sách độ trễ theo từng bước

Bước xử lý	Ước lượng Phase 0 (ms)	Đo thật (ms)	Chênh (lần)	% tổng
Giải mã ảnh + tiền xử lý	50	2,83	0,06	1,7%
Suy luận YOLO11n @ 640px (CPU)	150	57,27	0,38	34,0%
Cắt + tiền xử lý vùng biên số	30	0,00	0,00	0,0%
PaddleOCR (mỗi biến)	120	108,28	0,90	64,3%
Hậu xử lý regex + kiểm tra hợp lệ	5	0,03	0,01	0,0%
Ghi CSDL + lưu ảnh	50	—	—	—
Tổng (một biến số)	405	168,41	0,47	100%

Ba phát hiện. **Một, ước lượng Phase 0 sát bất ngờ ở tổng nhưng lệch ở phân bố**: tổng suy luận thuần **168,41 ms/biến**, nhỏ hơn cả ước lượng Phase 0 — ước lượng ban đầu **không** sai một bậc độ lớn như con số nhiễu 5.857 ms từng khiến người ta tin. **Hai, nút thắt là PaddleOCR nhưng KHÔNG áp đảo như báo cáo cũ**: 64,3% so với 34,0% của bộ phát hiện, **thay thế** con số cũ "OCR 93,3% / detect 6,5%" vốn đo trên hệ thống đang có lỗi crop

(~1.322 ms/ảnh); nguyên nhân OCR đắt vẫn đúng — PaddleOCR là **pipeline nhiều giai đoạn** (phát hiện văn bản → phân loại hướng → nhận dạng) thiết kế cho ảnh tài liệu tổng quát, nên hệ thống trả chi phí cho năng lực mà vùng biển đã cắt không cần. **Ba, chiến lược tối ưu đổi hẳn:** theo định luật Amdahl, tăng tốc detector 2–3× có thể kéo E2E xuống quãng **15–23%**, khác hẳn kết luận cũ "chỉ giảm tối đa 6,7%"; và vì NFR-P1 mới đạt sàn còn NFR-P2 trượt cả sàn (2,379 FPS, sàn 3), tối ưu hiệu năng nằm trên đường tới chỉ tiêu chứ không chỉ là *dư địa cải thiện thêm*.

5.6.3. So sánh backend suy luận: PyTorch, ONNX Runtime và OpenVINO

Phép so sánh này chưa được thực hiện. Lướt `benchmark_cpu --backends pytorch onnx opencvino` **chưa chạy**, nên độ trễ riêng bộ phát hiện, mức tăng tốc so với PyTorch và phần trăm cải thiện đầu-cuối của hai runtime kia đều **chưa có số**. Khi đo, chỉ tiêu **mAP@0.5** sau **khi xuất** phải đo cùng lúc để kiểm tra việc chuyển đổi định dạng **không làm suy giảm độ chính xác** — nếu có suy giảm thì mức tăng tốc phải được đánh giá như một đánh đổi chứ không phải một khoản lãi. Thí nghiệm vẫn đáng làm dù kết luận đoán trước được từ 5.6.2: nó **kiểm chứng** lập luận Amdahl bằng số liệu [18] [117].

Tối ưu backend của bộ phát hiện GIỜ có ý nghĩa, nhưng chưa đủ một mình. Vì NFR-P1 **chỉ đạt sàn** còn NFR-P2 **trượt sàn**, đây không phải dư địa cải thiện thêm mà là đường dẫn tới chỉ tiêu; muốn giảm mạnh hơn thì khối OCR (64,3%) vẫn là mục tiêu lớn nhất.

Bốn hướng tấn công khối OCR theo chi phí tăng dần (chi tiết ở Chương 6): **tắt các giai đoạn không cần thiết của pipeline PaddleOCR; bật MKL-DNN và chỉnh số luồng CPU; xuất mô hình nhận dạng sang ONNX Runtime; thay bằng mô hình nhận dạng chuyên cho biến số** huấn luyện trên tập ký tự hẹp — tiềm năng lớn nhất, tốn công nhất.

5.6.4. Chế độ webcam (tầng API) và xử lý video (NFR-P2, NFR-P3)

NFR-P2 không đạt (2,379 FPS; sàn 3, mục tiêu 5), nguyên nhân không phải tốc độ trung bình mà là đuôi phân bố. Trung vị mỗi khung chỉ **180,05 ms** — tương ứng 5,6 FPS, vượt mục tiêu — nhưng p95 là **1.247,70 ms**, và giao diện thời gian thực dùng **hàng đợi một khe**: chỉ một yêu cầu bay tại một thời điểm, khung sinh ra trong lúc chờ bị bỏ thay vì xếp hàng; ở kỷ luật đó thông lượng bị chi phối bởi những lần chậm nhất. Đuôi ấy chính là bậc thang thử lại (5.5.7) — **đánh đổi đã biết**: cùng cơ chế đó mua thêm 34 biến đọc đúng và đẩy NFR-P1 từ  xuống . Ngược lại NFR-P3 **đạt**: video 14,25 giây xử lý hết **19,1 giây** (sàn ≤ 95 s, mục tiêu ≤ 47,5 s), tức **0,746×** thời gian thực — **0,546 s mỗi khung phân tích**, `vid_stride = 5`.

5.6.5. Chịu tải, bộ nhớ và độ tin cậy (NFR-SC1, NFR-P4...P7, NFR-R4)

Mọi chỉ tiêu hiệu năng ngoài đường xử lý ảnh đều đạt với biên rất rộng (chi tiết ở Bảng 5.10): nạp mô hình **6,41 s**, khởi động tới khi /health sẵn sàng **8,36 s** (sàn 30 s); overhead API p95 **19,01 ms**; truy vấn lịch sử 10.000 bản ghi p95 **18,71 ms** — nhanh hơn mục tiêu **~27 lần**; RSS pipeline / máy chủ backend **0,759 / 0,806 GB** so với sàn 4 GB, tức phẳng ở khoảng

0,8 GB; 10 yêu cầu đồng thời ổn định so với ngưỡng 5; soak 15 phút **100,0% thành công trên 2.028 yêu cầu**, RSS chỉ tăng **+0,094 GB** (0,726 → 0,820) — **không rò rỉ**; cơ sở dữ liệu sống sót qua khởi động lại với **0/9.031 bản ghi mất**. Hình đường cong thông lượng và tỉ lệ lỗi theo mức đồng thời 1 / 2 / 5 / 10 (07-concurrency . png) **đã có nhưng cần vẽ lại**.

Nhưng hai chỉ tiêu trên chính đường xử lý ảnh thì không: NFR-P1 chỉ đạt sàn và **NFR-P2 trượt cả sàn**, cùng nguyên nhân là đuôi độ trễ do bậc thang thử-lại. **Kiến trúc phần mềm không còn là vấn đề** — tầng API, tầng dữ liệu, bộ nhớ, độ ổn định đều dư biên; **nhưng độ trễ suy luận thì vẫn là vấn đề**. Hai nhánh đi tiếp: nâng *độ chính xác* OCR biến hai dòng (5.5), và cắt *đuôi độ trễ* — đặt trần thời gian cho bậc thang, hoặc chỉ chạy nó ở chế độ ảnh tĩnh.

5.6.6. Bỏ bước phát hiện chữ của PaddleOCR: một quyết định suýt sai

Mục 4.5.3 để ngỏ một câu hỏi: chế độ **chỉ nhận dạng** cho model fine-tune thêm **12,46 điểm A6** và rẻ hơn **~290 ms** mỗi ảnh — vì sao không bật? Vì ngữ liệu chứng minh nó **không có thẩm quyền trả lời câu hỏi đó**.

Bảng 5.9. Bỏ bước phát hiện chữ — ba ngữ liệu, hai kết luận ngược nhau

Cấu hình	A6 trên 2.801 ảnh cắt sẵn	Bộ demo ảnh toàn cảnh	Tầng dễ (n=372)	Tầng khó (n=236)	A7 hiện trường	KTC 95%
Model gốc, det + rec — <i>bản giao hàng</i>	0,7512	17 / 22	96,8%	44,1%	56,3%	[52,0 ; 60,7]
Model gốc, chỉ rec	0,7508	13 / 22	96,8%	19,9%	37,8%	[34,3 ; 41,3]
Model fine- tune, det + rec	0,6762	14 / 22	96,8%	30,9%	46,3%	[42,2 ; 50,3]
Model fine- tune, chỉ rec	0,8758	15 / 22	94,1%	44,5%	56,0%	[51,7 ; 60,4]

5.7. Đối chiếu toàn bộ chỉ tiêu phi chức năng

Bảng tổng hợp trình bày khi bảo vệ, liệt kê **đầy đủ mọi mã NFR** đã đặt ra ở Phase 0, không lọc bỏ mã nào — kể cả những mã không đạt.

Bảng 5.10. Đối chiếu chỉ tiêu phi chức năng, gom theo nhóm

Nhóm	Số chỉ tiêu	Kết quả	Con số quyết định
Độ chính xác — phát hiện (A1,	3	 đạt cả ba,	mAP@0,5 = 0,9829 (mục tiêu 0,90); Precision · Recall = 0,9837 · 0,9714

Nhóm	Số chỉ tiêu	Kết quả	Con số quyết định
A2, A3)		biên rộng	
Độ chính xác — nhận dạng chuỗi (A4 – A7)	4	● một, ✗ ba	A4 = 0,9454 (vượt sần 0,92, dưới mục tiêu 0,95); A5 · A6 · A7 = 0,6373 · 0,7512 · 0,5552 , cả ba dưới sần
Đóng góp hậu xử lý (A6 – A5)	1	✓	+11,39 điểm — 319 sửa đúng, 0 làm hỏng , trên 2.801 biên
Báo cáo tách bạch (A8, A9)	2	● A8, □ A9	Chênh lệch layout: 2,09 điểm ở phát hiện so với 25,45 điểm ở nhận dạng. A9 không đo được — bộ dữ liệu không có nhãn điều kiện chụp
Hiệu năng — độ trễ (P1, P2, P3)	3	● P1, ✗ P2, ✓ P3	p95 một ảnh 1.143,10 ms (sần 1.500, mục tiêu 800; p50 chỉ 405,77 ms). FPS thời gian thực 2,379 — dưới sần 3. Video 0,746× — vượt mục tiêu 0,3×
Hiệu năng — tài nguyên (P4 – P7)	5	✓ đạt cả năm	Nạp mô hình 6,41 s ; overhead API 19,01 ms ; truy vấn 10.000 bản ghi 18,71 ms ; RSS 0,806 GB
Độ tin cậy và chịu tải (R1 – R5, SC1 – SC3)	8	✓ đạt cả tám	100,0% thành công qua 2.028 yêu cầu soak 15 phút; 0/9.031 bản ghi mất sau khởi động lại; 10 yêu cầu đồng thời ổn định
Bảo trì, bảo mật, khả dụng, ràng buộc (M, S, U, C)	14	✓ 13 , ⚠ 1	Bao phủ kiểm thử tăng nghiệp vụ 87,7% (sần 70%); chạy không cần GPU; riêng M6 còn 83 cảnh báo E501 của ruff

5.8. Phân tích lỗi

Bảng 5.11. Tần suất từng loại lỗi

Mã	Loại lỗi	Số ca	Tỉ lệ trong tổng ca sai	Tỉ lệ toàn tập đánh giá	Một dòng	Hai dòng
E1	Bỏ sót biên	335	—	—	—	—
E2	Phát hiện nhầm	(chưa đo)	—	—	—	—
E3	Nhầm ký tự	445	63,85%	15,89%	17	428
E4	Thiếu ký tự	73	10,47%	2,61%	0	73
E5	Thừa ký tự	18	2,58%	0,64%	5	13
E6	Sai thứ tự	0	0,00%	0,00%	0	0
	Tổng số ca sai	697	100%	24,88%	—	—
	Tổng ca đánh giá	2.801	n/a	100%	—	—

Mã	Loại lỗi	Số ca	Tỉ lệ trong tổng ca sai	Tỉ lệ toàn tập đánh giá	Một dòng	Hai dòng
(mẫu số)						

5.9. Bàn luận

5.9.1. Đọc kết quả: đạt gì, không đạt gì

Chương 6 tổng hợp đầy đủ kết quả và hạn chế; mục này chỉ nêu **cách đọc** bộ số liệu vừa trình bày. **Vạch ngăn nằm giữa hai tầng, không rải đều:** bộ phát hiện đạt toàn bộ chỉ tiêu với biên rộng và điểm yếu duy nhất — dải "rất nhỏ" ở 5.4.3 — được phơi bày chứ không giấu; khối hậu xử lý đóng góp **thuần dương, không rủi ro** (+11,39 điểm, 0 ca hồi quy); còn ba chỉ tiêu độ chính xác chuỗi thì không đạt.

5.9.2. Sáu hạng mục chưa đo và trạng thái khắc phục

Hạng mục	Trạng thái	Có làm được trong khuôn khổ đề án?
NFR-A9 — độ chính xác theo điều kiện ảnh	Không có nhãn điều kiện chụp trong bộ dữ liệu	 Không — thiếu điều kiện; khắc phục một phần bằng gán nhãn thủ công cho tập con
So sánh backend suy luận PyTorch ↔ ONNX ↔ OpenVINO (5.6.3)	Chưa chạy benchmark_cpu	 Có — chỉ cần thời gian máy
Phân rã đóng góp theo từng nhóm luật (5.5.2)	Chưa có cơ chế bật/tắt luật trong plate_rules.py	 Có — cần viết thêm mã
Thí nghiệm cô lập biến E1 – E3 (3.6.2)	Ước tính ≈ 33 giờ CPU , vượt ngân sách	 Không trong khuôn khổ đề án
Benchmark engine OCR hứa ở mục 3.3	 Đã chạy 03/08/2026 (3.3.3) — PaddleOCR 68,87% so với EasyOCR 14,28% và Tesseract 10,28%	— đã hoàn thành
Huấn luyện YOLO26n làm đối chứng hứa ở mục 3.2	Chưa huấn luyện — ngân sách CPU dồn hết cho lượt best.pt	 Có — chỉ cần thời gian máy

Phân biệt "**chưa đo vì chưa tới lượt**" với "**không đo được vì thiếu điều kiện**" là quan trọng khi đọc bảng này: chỉ nhóm thứ hai — NFR-A9 thiếu nhãn, và A7 thiếu tập ảnh hiện trường có nhãn chuỗi ở thời điểm đo — mới là hạn chế thật của công trình.

5.9.3. Các mối đe dọa đến tính hợp lệ của kết quả

Nguyên tắc: **nêu mối đe dọa, đánh giá mức nghiêm trọng, nói rõ đã làm gì để giảm thiểu — kể cả khi biện pháp giảm thiểu là "không có".**

Bảng 5.12. Tám mối đe dọa đến tính hợp lệ của kết quả

#	Mối đe dọa	Mức	Biện pháp giảm thiểu đã áp dụng
1	Rò rỉ dữ liệu tồn dư không khử được bằng bấm tri giác	Cao	Nâng ngưỡng gộp 5 → 10 và đo ở nhiều ngưỡng cao hơn ngưỡng gộp. Vẫn còn 791 cặp ở Hamming 12; rò rỉ <i>ngữ nghĩa</i> (cùng một xe, góc khác) không ngưỡng phash nào phát hiện được ⇒ mọi chỉ số ở 5.4 và 5.6 phải coi là cận trên lạc quan
2	Tập test không xuyên bộ dữ liệu	Cao	Không có — cách đúng là giữ lại một nguồn hoàn toàn không dùng để huấn luyện; chưa thực hiện
3	Mẫu số nhỏ cho các chỉ số OCR (2.801 / 15.133 ảnh có nhãn chuỗi)	Cao	Công bố mẫu số ở mọi bảng của 5.5; không rút kết luận về chênh lệch nhỏ
4	Đo trên một cấu hình phần cứng duy nhất	Trung bình	Công bố cấu hình đầy đủ ở 5.2; không ngoại suy sang CPU, hệ điều hành hay số nhân khác
5	Một lượt huấn luyện duy nhất, không ước lượng được phương sai	Trung bình	Cố định seed = 42 để tái lập; không phát biểu so sánh dựa trên chênh lệch nhỏ
6	Bộ dữ liệu không đạt tiêu chí Q6 về tỉ lệ đối tượng nhỏ (10,91%)	Trung bình	Báo cáo mAP tách theo dải kích thước ở 5.4.3
7	Nhãn layout suy ra từ tỉ lệ khung hình khi bộ dữ liệu không khai báo	Thấp – TB	Ưu tiên nhãn lớp tường minh khi có; ghi rõ tỉ lệ ô suy bằng heuristic
8	Ma trận nhầm lẫn ký tự phụ thuộc thuật toán căn chỉnh chuỗi	Thấp	Áp ngưỡng tần suất tối thiểu trước khi đưa một cặp vào bảng luật (5.5.4)

5.10. Đối chiếu với các công trình đã công bố

Ba khác biệt khiến việc đặt cạnh nhau hai con số độ chính xác của hai công trình là **không hợp lệ về phương pháp**, trừ khi cả ba đều trùng. **Bộ dữ liệu và quốc gia**: biển Trung Quốc chủ yếu một dòng, biển Brazil có bố cục và phong chữ riêng, biển Việt Nam có tỷ lệ biển hai dòng cao. **Định nghĩa chỉ số**: “*accuracy*” trong các bài được khảo sát khi thì là mức ký tự, khi là mức chuỗi, khi là toàn trình tính cả bước phát hiện. **Điều kiện ảnh**: camera tĩnh ở trạm thu phí khác hẳn ảnh chụp tự do. Bằng chứng mạnh nhất đến từ chính lĩnh vực:

Laroca và cộng sự (2022) chạy **12 mô hình OCR trên 9 tập dữ liệu công khai**, độ chính xác trung bình **sụt từ 82,4% xuống 45,2%** khi đánh giá xuyên tập dữ liệu.

⚠ **Hệ quả bắt buộc cho toàn mục này.** Mọi con số của công trình khác đều **kèm tên bộ dữ liệu và quốc gia ngay trong bảng**, và không con số nào được dùng để kết luận rằng hệ thống của đồ án tốt hơn hay kém hơn. Chúng chỉ trả lời một câu hỏi hẹp hơn nhiều: *kết quả của đồ án có nằm trong vùng giá trị mà lĩnh vực đã ghi nhận hay không.*

Bảng 5.13. Đối chiếu với các công trình đã công bố — mọi dòng kèm bộ dữ liệu và quốc gia

Khối	Công trình	Bộ dữ liệu · quốc gia	Chỉ số công bố	Giá trị
Phát hiện	Batra và cộng sự (2022)	Google Open Images + biển Ấn Độ, 5.991 ảnh	mAP@0.5	87,2%
Phát hiện	Ba nghiên cứu dùng YOLO11 cho ALPR (mục 3.2)	các tập khác nhau	mAP@0.5	90,6% – 99,5%
Phát hiện	Đồ án này	corpus Việt Nam hợp nhất, 1.514 ảnh test	mAP@0.5	98,29%
Nhận dạng	Xu và cộng sự — RPnet (2018)	CCPD · Trung Quốc	accuracy end-to-end	98,5%
Nhận dạng	Laroca và cộng sự (2021)	8 tập từ 5 khu vực	recognition rate trung bình	96,9%
Nhận dạng	Xu và cộng sự — LPTR-AFLNet (2025)	biển Trung Quốc	accuracy riêng biển hai dòng	99,37%
Nhận dạng	Tran và Bui (2024)	biển Việt Nam, chạy trên Raspberry Pi 4	accuracy	95,68%
Nhận dạng	Đồ án này	2.801 biển Việt Nam có nhãn chuỗi	A6 / A7	75,12% / 55,52%

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Đồ án đã bàn giao một hệ thống nhận dạng biển số xe Việt Nam hoàn chỉnh, chạy đầu-cuối trên máy **không có GPU**: bộ phát hiện tự huấn luyện, khối nhận dạng ký tự, bộ luật hậu xử lý theo quy chuẩn Việt Nam, REST API, giao diện web, cơ sở dữ liệu và đóng gói Docker. Trạng thái xác minh bằng HTTP thật — 10 thao tác trên 9 đường dẫn phản hồi đúng, **1.000/1.001** kiểm thử tự động đạt, bao phủ tầng nghiệp vụ 87,7%.

Bảng 6.1. Đối chiếu chỉ tiêu cam kết ở Phase 0 với số đo trên models/best.pt

Mã	Chỉ tiêu	Mục tiêu	Đo được	
A1 · A2	mAP@0,5 · mAP@0,5:0,95 (phát hiện)	0,90 · 0,65	0,9829 · 0,7834	✓
A3	Precision · Recall (phát hiện)	0,92 · 0,90	0,9837 · 0,9714	✓
A4	1 – CER (mức ký tự)	0,95	0,9454	●
A5 · A6	Chuỗi trước · sau hậu xử lý	0,85 · 0,90	0,6373 · 0,7512	✗
A7	Toàn trình từ ảnh gốc	0,88	0,5552	✗ *
A8	Chênh lệch layout ở tầng phát hiện (điểm %)	—	2,09	—
P1	Độ trễ p95 một ảnh (ms)	≤ 800	1.143,10	●
P4 · P5 · P6	Nạp mô hình (s) · Overhead API · Truy vấn 10.000 bản ghi (ms)	≤ 15 · ≤ 50 · ≤ 500	6,41 · 19,01 · 18,71	✓
P7 · R4 · SC1	RSS (GB) · Thành công khi chạy liên tục · Yêu cầu đồng thời	≤ 2 · ≥ 99% · ≥ 5	0,806 · 100% · 10	✓

* A7 phải đọc như **cận dưới bi quan** — đo trên ảnh nằm ngoài phân bố huấn luyện của bộ phát hiện nên tỉ lệ bỏ sót bị thổi phồng.

Vạch ngăn "đạt / không đạt" trùng khít vạch ngăn giữa hai tầng: mọi chỉ tiêu phát hiện, độ tin cậy và chịu tải đều đạt; mọi chỉ tiêu độ chính xác chuỗi đầy đủ đều không đạt. Riêng NFR-P1 chỉ đạt ngưỡng tối thiểu — thoái lui **có chủ ý** đối lấy 34 biến đọc thêm.

Bốn đại lượng đo được mà khảo sát không tìm thấy tương đương trong tài liệu Việt Nam.

- Đóng góp thuần của khối hậu xử lý theo vị trí: +11,39 điểm** — sửa đúng 319 biến, làm hỏng 0 biến trên 2.801 mẫu.
- Chênh lệch giữa hai bố cục biến trên dữ liệu Việt Nam thật: 25,45 điểm** ở khối nhận dạng, so với chỉ 2,09 điểm ở khối phát hiện. Rủi ro R-04 vì vậy nằm trọn ở tầng đọc ký tự.

3. **Benchmark ba engine OCR trên 2.801 biển, cùng một tầng bao quanh:**
PaddleOCR **68,87%** so với EasyOCR 14,28% và Tesseract 10,28%. Phép đo lấp khoảng trống nghiên cứu số 4 và **bác bỏ** tài liệu công khai vốn nghiêng về EasyOCR. Nó cũng cho một kết quả trái kỳ vọng: bước tách-rời-ghép-ngang mua 34,92 điểm cho PaddleOCR nhưng chỉ 0,03 điểm cho Tesseract — **điều kiện cần, không đủ**.
4. **Bộ nhận màu nền biển đạt 97,89%** trên 1.565 ảnh có nhãn, cung cấp nguồn bằng chứng mà chuỗi ký tự không mang được: phân giải nhập nhằng giữa biển xanh nhà nước và biển trắng cá nhân khi hai chuỗi giống hệt nhau.

Ngoài các con số, đồ án để lại **một quy trình đánh giá có kiểm chứng**: mọi số liệu sinh lại được bằng một lệnh, mọi phép so sánh kèm điều kiện đo, và các kết quả âm — hai lượt tinh chỉnh bộ nhận dạng đều không thắng model gốc ở chế độ vận hành — được ghi lại thay vì bỏ đi.

6.2. Hạn chế

Bảng 6.2. Tám hạn chế của đồ án

#	Hạn chế	Mức	Hệ quả cần lưu ý
1	OCR biển hai dòng còn yếu, kéo độ chính xác toàn trình chưa đạt	Cao	A6 = 0,7512 và A7 = 0,5552 cùng dưới ngưỡng — nút thắt lớn nhất
2	Bộ dữ liệu lệch nặng về biển trắng	Cao	97,68% mẫu thuộc một lớp, nên kết luận về độ chính xác OCR chỉ áp cho biển trắng
3	Rò rỉ dữ liệu tồn dư không khử được bằng băm tri giác	Trung bình	phash chỉ bắt tương đồng bố cục sáng-tối, không bắt "cùng xe, khác ngày"
4	Tập test không xuyên bộ dữ liệu	Trung bình	mAP 0,9829 lạc quan hơn mức gặp khi triển khai với nguồn ảnh mới
5	Độ trễ chỉ đạt ngưỡng tối thiểu	Trung bình	p95 = 1.143,10 ms; đánh đổi có chủ ý lấy 34 biển đọc thêm
6	Một yêu cầu mức <i>Must</i> (FR-4.1) bị đưa ra khỏi phạm vi	Trung bình	Trang Tổng quan đã gỡ 20/07/2026 — phải nêu rõ khi bảo vệ
7	SQLite chỉ cho phép một tiến trình ghi tại một thời điểm	Thấp	Đủ cho quy mô đồ án, chặn ở triển khai đa người dùng
8	Xem trực tiếp và xử lý nền tranh chấp CPU với nhau	Thấp	Chạy video nền làm chậm luồng nhận dạng ảnh

6.3. Hướng phát triển

Bảng 6.3. Chín hướng phát triển, xếp theo mức tác động

#	Hướng	Giải hạn chế	Ghi chú
1	Huấn luyện lại module nhận dạng riêng cho biển số Việt Nam	1	Hướng quan trọng nhất. Hai lượt tinh chỉnh đã thực hiện đều chưa thắng model gốc ở chế độ vận hành
2	Thu thập dữ liệu cho các loại biển hiếm	2	Điều kiện để mở rộng kết luận ra ngoài biển trắng
3	Bổ sung nhãn chuỗi cho toàn tập	1, 2	Hiện chỉ 2.801/15.133 ảnh có nhãn chuỗi
4	Xây dựng tập test xuyên bộ dữ liệu	3, 4	Giữ nguyên một nguồn hoàn toàn không dùng để huấn luyện
5	Tăng tốc suy luận: lượng tử hoá OCR, đóng gói ONNX/OpenVINO	5	Phép so sánh runtime chưa chạy — khoản nợ ghi ở mục 5.9.2
6	Thí nghiệm cô lập biến độ phân giải · dữ liệu · số epoch	4	Ma trận E1–E3, ước tính ≈ 33 giờ CPU
7	Bám vết đối tượng qua khung hình cho video (SORT/DeepSORT)	—	Gộp nhiều lần đọc cùng một biển thành một kết quả
8	Tách lịch chạy giữa xem trực tiếp và xử lý nền	8	Hàng đợi ưu tiên hoặc giới hạn luồng cho tác vụ nền
9	Chuyển sang PostgreSQL nếu triển khai đa người dùng	7	Chỉ cần khi vượt quy mô một tiến trình ghi

6.4. Kết luận chung

Đề tài đặt ra một bài toán có ràng buộc rõ: nhận dạng biển số xe Việt Nam, hỗ trợ **cả biển một dòng và biển hai dòng**, suy luận hoàn toàn trên CPU. Hệ thống bàn giao đáp ứng ràng buộc vận hành và đạt toàn bộ chỉ tiêu ở tầng phát hiện với biên rộng, nhưng **chưa đạt** chỉ tiêu độ chính xác ở tầng nhận dạng ký tự — và phần thiếu hụt nằm gần như trọn ở biển hai dòng.

Giá trị của đề án vì vậy không nằm ở một con số cao nhất. Nó nằm ở ba chỗ: **một hệ thống hoàn chỉnh chạy được trong đúng ràng buộc phần cứng đã tuyên bố; bốn đại lượng đo được mà tài liệu trong nước chưa công bố tách bạch**, trong đó có benchmark ba engine OCR trên chính ảnh biển số Việt Nam; và **một cách báo cáo trong đó phần chưa đạt được trình bày với cùng mức chi tiết như phần đạt** — kể cả khi điều đó có nghĩa là công bố rằng một đóng góp kỹ thuật lõi không độc lập với engine như đã kỳ vọng.

Với một hệ thống mà nút thắt đã được định vị bằng số liệu, bước tiếp theo là rõ ràng: thay module nhận dạng ký tự bằng mô hình huấn luyện riêng cho biển số Việt Nam. Ràng buộc kiến trúc NFR-M5 — nay đã được chứng minh bằng chính phép đo ba engine — bảo đảm việc thay thế đó không chạm tới phần còn lại của hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Báo Dân trí, "Việt Nam có 77 triệu xe máy, cứ 1.000 dân có 770 người sở hữu xe máy," Báo Dân trí, 2024. [Trực tuyến]. Địa chỉ: <https://dantri.com.vn/thoi-su/viet-nam-co-77-trieu-xe-may-cu-1000-dan-co-770-nguoi-so-huu-xe-may-20241104141910472.htm> (truy cập ngày 2026-07-19).
- [2] C. E. Anagnostopoulos, I. E. Anagnostopoulos, I. D. Psoroulas, V. Loumos, E. Kayafas, "License Plate Recognition From Still Images and Video Sequences: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, q. 9, s. 3, tr. 377–391, 2008. doi: 10.1109/TITS.2008.922938.
- [3] S. Du, M. Ibrahim, M. Shehata, W. Badawy, "Automatic License Plate Recognition (ALPR): A State-of-the-Art Review," *IEEE Transactions on Circuits and Systems for Video Technology*, q. 23, s. 2, tr. 311–325, 2013. doi: 10.1109/TCSVT.2012.2203741.
- [4] eParking, "Nhận dạng biển số xe tự động trong bãi giữ xe thông minh," eParking, không rõ năm. [Trực tuyến]. Địa chỉ: <https://eparking.vn/nhan-dang-bien-so-xe/> (truy cập ngày 2026-07-19).
- [5] VETC, "Ô tô đi qua trạm thu phí không dừng sẽ quét biển hay quét mã thẻ," VETC, không rõ năm. [Trực tuyến]. Địa chỉ: <https://vetc.com.vn/o-to-di-qua-tram-thu-phi-khong-dung-se-quet-bien-hay-quet-ma-the-n114.html> (truy cập ngày 2026-07-19).
- [6] VisCom Solution, "VietANPR — phần mềm nhận diện biển số xe máy & xe hơi," VisCom Solution, không rõ năm. [Trực tuyến]. Địa chỉ: <https://viscomsolution.com/viet-anpr-phan-mem-nhan-dien-bien-so-xe-may-xe-hoi/> (truy cập ngày 2026-07-19).
- [7] R. Laroca, E. V. Cardoso, D. R. Lucio, V. Estevam, D. Menotti, "On the Cross-Dataset Generalization in License Plate Recognition," trong *International Conference on Computer Vision Theory and Applications (VISAPP)*, 2022. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2201.00267> (truy cập ngày 2026-07-19).
- [8] Bộ Công an, "Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," 2024. [Trực tuyến]. Địa chỉ: <https://chinhphu.vn/?pageid=27160&docid=211945&classid=1&orggroupid=4> (truy cập ngày 2026-07-19).
- [9] Bộ Công an, "Thông tư số 13/2025/TT-BCA sửa đổi, bổ sung một số điều của Thông tư số 79/2024/TT-BCA," 2025.
- [10] Bộ Công an, "Thông tư số 51/2025/TT-BCA sửa đổi, bổ sung một số điều của Thông tư số 79/2024/TT-BCA đã được sửa đổi tại Thông tư số 13/2025/TT-BCA," 2025. [Trực tuyến]. Địa chỉ: <https://congbao.chinhphu.vn/van-ban/thong-tu-so-51-2025-tt-bca-45356.htm> (truy cập ngày 2026-07-19).

[11] Bộ Công an, "Quy chuẩn kỹ thuật quốc gia về biển số xe QCVN 08:2024/BCA," 2024. [Trực tuyến]. Địa chỉ: <https://mps.gov.vn/chinh-sach-phap-luat/bai-viet/quy-chuan-ky-thuat-quoc-gia-ve-bien-so-xe-d1-t1592> (truy cập ngày 2026-07-19).

[12] Bộ Công an, "Thông tư số 24/2023/TT-BCA quy định về cấp, thu hồi đăng ký, biển số xe cơ giới," 2023. [Trực tuyến]. Địa chỉ: <https://xaydungchinhhsach.chinhphu.vn/toan-van-thong-tu-24-2023-tt-bca-quy-dinh-ve-cap-thu-hoi-dang-ky-bien-so-xe-co-gioi-119230712221629971.htm> (truy cập ngày 2026-07-19).

[13] Bộ Công an, "Nhận diện màu sắc, seri, ký hiệu biển số xe của cơ quan, tổ chức, cá nhân từ 01/01/2025," Cổng Thông tin điện tử Bộ Công an, 2024. [Trực tuyến]. Địa chỉ: <https://bocongan.gov.vn/chinh-sach-phap-luat/bai-viet/nhan-dien-mau-sac-seri-ky-hieu-bien-so-xe-cua-co-quan-to-chuc-ca-nhan-tu-01012025-d1-t1617> (truy cập ngày 2026-07-19).

[14] Thư viện Nhà đất, "Chính thức ký hiệu biển số xe 34 tỉnh thành sau sáp nhập theo Thông tư 51/2025/TT-BCA," Thư viện Nhà đất, 2025. [Trực tuyến]. Địa chỉ: <https://thuviennhadat.vn/phap-luat/chinh-thuc-ky-hieu-bien-so-xe-34-tinh-thanh-sau-sap-nhap-theo-thong-tu-51-2025-tt-bca-690227.html> (truy cập ngày 2026-07-19).

[15] Bộ Quốc phòng, "Thông tư 169/2021/TT-BQP quy định về đăng ký, quản lý, sử dụng xe cơ giới, xe máy chuyên dùng trong Bộ Quốc phòng," 2021. [Trực tuyến]. Địa chỉ: <https://luatvietnam.vn/giao-thong/thong-tu-169-2021-tt-bqp-bo-quoc-phong-216143-d1.html> (truy cập ngày 2026-07-19).

[16] G. Jocher, J. Qiu, "Ultralytics YOLO11," Ultralytics, 2024. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolo11/> (truy cập ngày 2026-07-19).

[17] C. Cui, "PP-OCrV5: A Specialized 5M-Parameter Model Rivaling Billion-Parameter Vision-Language Models on OCR Tasks," *CVPR 2026 / arXiv:2603.24373*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2603.24373v1> (truy cập ngày 2026-07-19).

[18] Ultralytics, "Intel OpenVINO Export — Ultralytics Docs (ma nguồn markdown, day du bang benchmark CPU/GPU/NPU)," GitHub / Ultralytics Docs, 2026. [Trực tuyến]. Địa chỉ: https://raw.githubusercontent.com/ultralytics/ultralytics/main/docs/en/integrations/open_vino.md (truy cập ngày 2026-07-19).

[19] F. Meyer, L. Guichard, D. Coquenot, G. Gravier, Y. Soullard, B. Couasnon, "Relaxed syntax modeling in Transformers for future-proof license plate recognition," *arXiv preprint arXiv:2506.17051*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2506.17051> (truy cập ngày 2026-07-19).

[20] Z. Li, M. A. Ghaffar, "A detailed review on license plate detection and recognition methods," *Journal of Traffic and Transportation Engineering (English Edition)*, q. 13, s. 3, tr. 974–1005, 2026. doi: 10.1016/j.jtte.2024.10.007.

[21] D. H. Le, D. Mazumder, L. D. Quach, S. Banerjee, V. D. Nguyen, "Robust Vietnam's Motorcycle License Plate Detection and Recognition Using Deep Learning Model," trong

Future Data and Security Engineering (FDSE), Springer, 2023. doi: 10.1007/978-981-99-8296-7_5.

[22] S. M. Silva, C. R. Jung, "License Plate Detection and Recognition in Unconstrained Scenarios," trong *European Conference on Computer Vision (ECCV)*, 2018. doi: 10.1007/978-3-030-01258-8_36.

[23] R. Laroca, L. A. Zanolensi, G. R. Gonçalves, E. Todt, W. R. Schwartz, D. Menotti, "An efficient and layout-independent automatic license plate recognition system based on the YOLO detector," *IET Intelligent Transport Systems*, q. 15, s. 4, tr. 483–503, 2021. doi: 10.1049/itr2.12030.

[24] G. Hsu, J. Chen, Y. Chung, "Application-Oriented License Plate Recognition," *IEEE Transactions on Vehicular Technology*, q. 62, s. 2, tr. 552–561, 2013. [Trực tuyến]. Địa chỉ: https://www.researchgate.net/publication/260498098_Application-Oriented_License_Plate_Recognition (truy cập ngày 2026-07-19).

[25] R. Laroca, "UFPR-ALPR dataset — kho mã nguồn chính thức," GitHub / VRI Lab, Federal University of Parana, 2018. [Trực tuyến]. Địa chỉ: <https://github.com/raysonlaroca/ufpr-alpr-dataset> (truy cập ngày 2026-07-19).

[26] Cổng Thông tin điện tử Chính phủ, "Quy định ký hiệu biển số xe ô tô, xe máy tại các địa phương (theo Thông tư 51/2025/TT-BCA)," Xây dựng chính sách, pháp luật — Chinhphu.vn, 2025. [Trực tuyến]. Địa chỉ: <https://xaydungchinh sach.chinhphu.vn/quy-dinh-ky-hieu-bien-so-xe-o-to-xe-may-tai-cac-dia-phuong-119250704073354464.htm> (truy cập ngày 2026-07-19).

[27] Cổng Thông tin điện tử Chính phủ, "Từ 15/8, sêri biển số xe máy cấp cho xe cá nhân có 2 chữ cái," Xây dựng chính sách, pháp luật — Chinhphu.vn, 2023. [Trực tuyến]. Địa chỉ: <https://xaydungchinh sach.chinhphu.vn/tu-15-8-seri-bien-so-xe-may-cap-cho-xe-ca-nhan-co-2-chu-cai-11923082122483385.htm> (truy cập ngày 2026-07-19).

[28] Oto.com.vn, "Bỏ quy định phân biệt seri đăng ký với một số dòng xe," Oto.com.vn, 2025. [Trực tuyến]. Địa chỉ: <https://oto.com.vn/thi-truong-o-to/bo-quy-dinh-phan-biet-seri-dang-ky-voi-mot-so-dong-xe-articleid-6ehu4o0> (truy cập ngày 2026-07-19).

[29] Kho Biển Số Đẹp, "Những quy định bạn cần biết về biển số xe kể từ năm 2025," Kho Biển Số Đẹp, 2025. [Trực tuyến]. Địa chỉ: <https://khobiensodep.vn/blogs/news/nhung-quy-dinh-ban-can-biet-ve-bien-so-xe-ke-tu-nam-2025> (truy cập ngày 2026-07-19).

[30] VietNamNet, "Cách đọc ký hiệu biển số xe ngoại giao, nước ngoài ở Việt Nam," VietNamNet, 2023. [Trực tuyến]. Địa chỉ: <https://vietnamnet.vn/cach-doc-ky-hieu-bien-so-xe-ngoai-giao-nuoc-ngoai-o-viet-nam-333426.html> (truy cập ngày 2026-07-19).

[31] Công an tỉnh Lạng Sơn, "Một số quy định mới của Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," Cổng thông tin Công an tỉnh Lạng Sơn, 2024. [Trực tuyến]. Địa chỉ: <https://congan.langson.gov.vn/9688/pho-bien-giao-duc-phap-luat/68/mot-so-quy-dinh->

[moi-cua-thong-tu-so-79-2024-tt-bca-quy-dinh-ve-cap-thu-hoi-chung-nhan-dang-ky-xe-bien-so-xe-co-gioi-xe-may-chuyen-dung/9688.aspx](#) (truy cập ngày 2026-07-19).

[32] Thư viện Pháp luật, "Quy định về màu sắc, seri biển số xe của cơ quan, tổ chức, cá nhân trong nước từ năm 2025," Thư viện Pháp luật, 2025. [Trực tuyến]. Địa chỉ: <https://thuvienphapluat.vn/banan/tin-tuc/quy-dinh-ve-mau-sac-seri-bien-so-xe-cua-co-quan-to-chuc-ca-nhan-trong-nuoc-tu-nam-2025-12612.html> (truy cập ngày 2026-07-19).

[33] "License Plate Localization Based on Edge Detection and Morphology," trong *Lecture Notes in Electrical Engineering*, Springer, 2012. doi: 10.1007/978-3-642-25899-2_92.

[34] "License plate localization based on edge-geometrical features using morphological approach," trong *IEEE International Conference*, 2013. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/6738937/> (truy cập ngày 2026-07-19).

[35] "Research on Characters Segmentation in One-Row and Two-Row of Vietnam License Plates," *Advanced Materials Research*, q. 479-481, 2012. [Trực tuyến]. Địa chỉ: <https://www.scientific.net/AMR.479-481.2293> (truy cập ngày 2026-07-19).

[36] mrzaizai2k, "mrzaizai2k/VIETNAMESE_LICENSE_PLATE," GitHub, 2025. [Trực tuyến]. Địa chỉ: https://github.com/mrzaizai2k/VIETNAMESE_LICENSE_PLATE (truy cập ngày 2026-07-19).

[37] R. Laroca và cộng sự, "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector," trong *International Joint Conference on Neural Networks (IJCNN)*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/1802.09567> (truy cập ngày 2026-07-19).

[38] Z. Xu và cộng sự, "Towards End-to-End License Plate Detection and Recognition: A Large Dataset and Baseline," trong *European Conference on Computer Vision (ECCV)*, 2018. [Trực tuyến]. Địa chỉ: https://openaccess.thecvf.com/content_ECCV_2018/papers/Zhenbo_Xu_Towards_End-to-End_License_ECCV_2018_paper.pdf (truy cập ngày 2026-07-19).

[39] H. Li, P. Wang, C. Shen, "Toward End-to-End Car License Plate Detection and Recognition With Deep Neural Networks," *IEEE Transactions on Intelligent Transportation Systems*, q. 20, s. 3, tr. 1126–1136, 2019. doi: 10.1109/TITS.2018.2847291.

[40] S. Zherzdev, A. Gruzdev, "LPRNet: License Plate Recognition via Deep Neural Networks," *arXiv preprint arXiv:1806.10447*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/1806.10447> (truy cập ngày 2026-07-19).

[41] L. Zhang, P. Wang, H. Li, Z. Li, C. Shen, Y. Zhang, "A Robust Attentional Framework for License Plate Recognition in the Wild," *IEEE Transactions on Intelligent Transportation Systems*, q. 22, s. 11, tr. 6967–6976, 2020. doi: 10.1109/TITS.2020.3000072.

[42] E. Shabaninia, F. Asadi-zeydabadi, H. Nezamabadi-pour, "Layout-Independent License Plate Recognition via Integrated Vision and Language Models," *arXiv preprint*

arXiv:2510.10533, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2510.10533> (truy cập ngày 2026-07-19).

[43] N. AlDahoul và cộng sự, "Advancing Vehicle Plate Recognition: Multitasking Visual Language Models with VehiclePaliGemma," *arXiv preprint arXiv:2412.14197*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2412.14197> (truy cập ngày 2026-07-19).

[44] H. Gong, H. Liu, "LP-LLM: End-to-End Real-World Degraded License Plate Text Recognition via Large Multimodal Models," *arXiv preprint arXiv:2601.09116*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2601.09116> (truy cập ngày 2026-07-19).

[45] Y. Wang, Z. Bian, Y. Zhou, L. Chau, "Rethinking and Designing a High-performing Automatic License Plate Recognition Approach," *IEEE Transactions on Intelligent Transportation Systems*, 2021. doi: 10.1109/TITS.2021.3087158.

[46] A. Wang và cộng sự, "YOLOv10: Real-Time End-to-End Object Detection," *arXiv preprint arXiv:2405.14458*, 2024. doi: 10.48550/arXiv.2405.14458.

[47] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, M. E. Kalfaoglu, "Ultralytics YOLO26," Ultralytics, 2025. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolo26/> (truy cập ngày 2026-07-19).

[48] "Advanced deep learning techniques for automated license plate recognition," *Scientific Reports*, 2025. [Trực tuyến]. Địa chỉ: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12639091/> (truy cập ngày 2026-07-19).

[49] G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolov8/> (truy cập ngày 2026-07-19).

[50] R. Khanam, M. Hussain, "YOLOv11: An Overview of the Key Architectural Enhancements," *arXiv preprint arXiv:2410.17725*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2410.17725> (truy cập ngày 2026-07-19).

[51] Ultralytics, "ultralytics/nn/modules/block.py — định nghĩa C2f, C3k, C3k2, C2PSA, PSABlock, Attention," GitHub, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ultralytics/ultralytics/blob/main/ultralytics/nn/modules/block.py> (truy cập ngày 2026-07-19).

[52] Ultralytics, "YOLO11 vs YOLOv8 — so sánh chính thức," Ultralytics, 2025. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/compare/yolo11-vs-yolov8/> (truy cập ngày 2026-07-19).

[53] C. Wang, I. Yeh, H. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," *ECCV 2024 / Ultralytics Docs*, 2024. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolov9/> (truy cập ngày 2026-07-19).

- [54] Y. Tian, Q. Ye, D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," *arXiv preprint arXiv:2502.12524*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2502.12524> (truy cập ngày 2026-07-19).
- [55] M. Lei và cộng sự, "YOLOv13: Real-Time Object Detection with Hypergraph-Enhanced Adaptive Visual Perception," *arXiv preprint arXiv:2506.17733*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2506.17733> (truy cập ngày 2026-07-19).
- [56] P. Batra và cộng sự, "A Novel Memory and Time-Efficient ALPR System Based on YOLOv5," *Sensors*, q. 22, s. 14, tr. 5283, 2022. doi: 10.3390/s22145283.
- [57] "Automatic License Plate Detection System with YOLOv11 Algorithm," *Journal of Applied Informatics and Computing (JAIC)*, 2025. [Trực tuyến]. Địa chỉ: <https://jurnal.polibatam.ac.id/index.php/JAIC/article/view/11484> (truy cập ngày 2026-07-19).
- [58] "Vehicle License Plate Number Detection with YOLO11," *Journal of Computer Science and Informatics Engineering (J-Cosine)*, 2025. [Trực tuyến]. Địa chỉ: <https://jcosine.if.unram.ac.id/index.php/jcosine/article/view/656> (truy cập ngày 2026-07-19).
- [59] Le Quy Don Technical University, "An efficient method to improve the accuracy of Vietnamese vehicle license plate recognition in unconstrained environment," trong *International Conference on Information and Computer Science (NICS)*, IEEE, 2021. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/9585279/> (truy cập ngày 2026-07-19).
- [60] Jaided AI, "JaidedAI/EasyOCR — DeepWiki (kien truc CRAFT + CRNN, kích thuoc model)," DeepWiki, 2025. [Trực tuyến]. Địa chỉ: <https://deepwiki.com/JaidedAI/EasyOCR> (truy cập ngày 2026-07-19).
- [61] "A Feasible Framework for Arbitrary-Shaped Scene Text Recognition," *arXiv preprint arXiv:1912.04561*, 2019. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/1912.04561> (truy cập ngày 2026-07-19).
- [62] PaddleOCR community, "PaddleOCR Issue #14109 — rec_image_shape mac dinh '3, 48, 320' tu PP-OCRv3," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: <https://github.com/PaddlePaddle/PaddleOCR/issues/14109> (truy cập ngày 2026-07-19).
- [63] "PatrolVision: Automated License Plate Recognition in the wild," *arXiv preprint arXiv:2504.10810*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2504.10810v1> (truy cập ngày 2026-07-19).
- [64] we0091234, "double_plate_split_merge.py — mã tách và ghép biển hai tầng (5/12 và 1/3 + hstack)," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: https://raw.githubusercontent.com/we0091234/Chinese_license_plate_detection_recognition/main/plate_recognition/double_plate_split_merge.py (truy cập ngày 2026-07-19).

- [65] PaddlePaddle, "PaddleOCR 3.x — OCR Pipeline Usage Tutorial," PaddleOCR, không rõ năm. [Trực tuyến]. Địa chỉ: https://www.paddleocr.ai/latest/en/version3.x/pipeline_usage/OCR.html (truy cập ngày 2026-07-19).
- [66] trungdinh22, "function/helper.py — logic phân biệt biển một dòng / hai dòng bằng kiểm tra thẳng hàng (abs_tol=3) và ghép theo y_mean," GitHub, không rõ năm. [Trực tuyến]. Địa chỉ: <https://raw.githubusercontent.com/trungdinh22/License-Plate-Recognition/main/function/helper.py> (truy cập ngày 2026-07-19).
- [67] PaddlePaddle, "PaddleOCR — ứng dụng nhận dạng biển số nhẹ (CCPD, PP-OCRv3, số liệu tinh chỉnh)," PaddleOCR v2.9, không rõ năm. [Trực tuyến]. Địa chỉ: <http://www.paddleocr.ai/v2.9/applications/%E8%BD%BB%E9%87%8F%E7%BA%A7%E8%BD%A6%E7%89%8C%E8%AF%86%E5%88%AB.html> (truy cập ngày 2026-07-19).
- [68] M. Li, "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," *arXiv preprint arXiv:2109.10282*, 2021. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2109.10282> (truy cập ngày 2026-07-19).
- [69] Roboflow, "TrOCR — Roboflow Inference Models," Roboflow, 2025. [Trực tuyến]. Địa chỉ: <https://inference-models.roboflow.com/models/trocr/> (truy cập ngày 2026-07-19).
- [70] L. Dang, V. Duong Ngoc, L. T. V. Pham Cung, "Vietnam Vehicle Number Recognition Based on an Improved CRNN with Attention Mechanism," *International Journal of Intelligent Transportation Systems Research*, 2024. doi: 10.1007/s13177-024-00402-7.
- [71] R. Laroca và cộng sự, "ICPR 2026 Competition on Low-Resolution License Plate Recognition," trong *International Conference on Pattern Recognition (ICPR)*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2604.22506> (truy cập ngày 2026-07-19).
- [72] M. Del Castillo Velarde, G. Velarde, "Benchmarking Algorithms for Automatic License Plate Recognition," *arXiv preprint arXiv:2203.14298*, 2022. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2203.14298> (truy cập ngày 2026-07-19).
- [73] L. Tao, S. Hong, Y. Lin, Y. Chen, P. He, Z. Tie, "A Real-Time License Plate Detection and Recognition Model in Unconstrained Scenarios," *Sensors*, q. 24, s. 9, tr. 2791, 2024. doi: 10.3390/s24092791.
- [74] M. Shpir, N. Shvai, A. Nakib, "License Plate Images Generation with Diffusion Models," *arXiv preprint arXiv:2501.03374*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2501.03374> (truy cập ngày 2026-07-19).
- [75] G. Xu, P. Zuo, Z. Ke, B. Lei, "LPTR-AFLNet: Lightweight Integrated Chinese License Plate Rectification and Recognition Network," *arXiv preprint arXiv:2507.16362*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2507.16362> (truy cập ngày 2026-07-19).
- [76] L. Wojcik, G. E. Lima, V. Nascimento, E. Nascimento Jr., R. Laroca, D. Menotti, "LPLC: A Dataset for License Plate Legibility Classification," trong *Conference on Graphics*,

Patterns and Images (SIBGRAPI), 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2508.18425> (truy cập ngày 2026-07-19).

[77] Z. Ebrahimi Vargoorani, A. M. Ghoreyshi, C. Y. Suen, "Efficient License Plate Recognition via Pseudo-Labeled Supervision with Grounding DINO and YOLOv8," *arXiv preprint arXiv:2510.25032*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2510.25032> (truy cập ngày 2026-07-19).

[78] V. Nascimento, R. Laroca, R. O. Ribeiro, W. R. Schwartz, D. Menotti, "Enhancing License Plate Super-Resolution: A Layout-Aware and Character-Driven Approach," trong *Conference on Graphics, Patterns and Images (SIBGRAPI)*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2408.15103> (truy cập ngày 2026-07-19).

[79] D. Tran-Anh, K. L. Tran, H. Vu, "License Plate Recognition Based on Multi-Angle View Model," *arXiv preprint arXiv:2309.12972*, 2023. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2309.12972> (truy cập ngày 2026-07-19).

[80] Tran, Bui, "Implementation of a License Plate Recognition System in Vietnam Using Embedding Devices," trong *Multi-disciplinary Trends in Artificial Intelligence (MIWAI)*, Springer, 2024. doi: 10.1007/978-981-96-0695-5_19.

[81] H. Tran, G. Ma, T. Nguyen, T. Cao, "Building Vietnam's License Plate Recognition System Based on OpenALPR," *International Journal of Multidisciplinary Research and Publications (IJMRAP)*, q. 5, s. 11, tr. 133–137, 2023. [Trực tuyến]. Địa chỉ: <http://ijmrmap.com/wp-content/uploads/2023/05/IJMRAP-V5N11P102Y23.pdf> (truy cập ngày 2026-07-19).

[82] "Nghiên cứu các phiên bản YOLOv8 và YOLO-NAS trong phát hiện biển số xe," *Tạp chí Khoa học Đại học Đà Lạt*, 2024. [Trực tuyến]. Địa chỉ: <https://scholar.dlu.edu.vn/thuvienso/bitstream/DLU123456789/290338/1/100567-1297-209737-1-10-20240806.pdf> (truy cập ngày 2026-07-19).

[83] "Building a license plate recognition system for Vietnam tollbooth," trong *Proceedings of the 3rd Symposium on Information and Communication Technology (SoICT)*, ACM, 2012. doi: 10.1145/2350716.2350734.

[84] Vietnamese Association for Pattern Recognition (VAPR), "Vietnamese Bike License Plate Recognition Challenge (MAPR 2018)," 1st International Conference on Multimedia Analysis and Pattern Recognition, UIT — DHQG TP.HCM, 2018. [Trực tuyến]. Địa chỉ: <https://mapr.uit.edu.vn/2018/vietnamese-bike-license-plate-recognition> (truy cập ngày 2026-07-19).

[85] T. L. Nguyễn, X. P. Đào, T. T. U. Nguyễn, H. P. Nguyễn, "Đề xuất mô hình YOLO V5 ứng dụng trong nhận diện biển số xe," *Tạp chí Khoa học Trường Đại học Mở Hà Nội*, 2023. [Trực tuyến]. Địa chỉ: <https://vjol.info.vn/index.php/jshou/article/view/86449> (truy cập ngày 2026-07-19).

- [86] Z. Xu, "CCPD: a diverse and well-annotated dataset for license plate detection and recognition," GitHub (detectRecog / USTC), 2018. [Trực tuyến]. Địa chỉ: <https://github.com/detectRecog/CCPD> (truy cập ngày 2026-07-19).
- [87] HyperAI, "AOLP Application-Oriented License Plate Dataset," HyperAI — hyper.ai, không rõ năm. [Trực tuyến]. Địa chỉ: <https://hyper.ai/en/datasets/19309> (truy cập ngày 2026-07-19).
- [88] R. Laroca, E. V. Cardoso, D. R. Lucio, V. Estevam, D. Menotti, "RodoSol-ALPR dataset — kho mã nguồn chính thức," GitHub, 2022. [Trực tuyến]. Địa chỉ: <https://github.com/raysonlaroca/rodosol-alpr-dataset> (truy cập ngày 2026-07-19).
- [89] OpenALPR, "OpenALPR benchmarks — kho mã nguồn chính thức," GitHub, 2016. [Trực tuyến]. Địa chỉ: <https://github.com/openalpr/benchmarks> (truy cập ngày 2026-07-19).
- [90] S. Agrawal, "Global License Plate Dataset," *arXiv preprint arXiv:2405.10949*, 2024. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2405.10949v1> (truy cập ngày 2026-07-19).
- [91] fict-labs, "VNLP — Vietnamese license plate dataset," GitHub, 2025. [Trực tuyến]. Địa chỉ: <https://github.com/fict-labs/VNLP> (truy cập ngày 2026-07-19).
- [92] "How many labeled license plates are needed?," *arXiv preprint arXiv:1808.08410*, 2018. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/1808.08410> (truy cập ngày 2026-07-19).
- [93] NNDam, "Vietnamese License Plate Generator," GitHub, 2024. [Trực tuyến]. Địa chỉ: <https://github.com/NNDam/Vietnamese-License-Plate-Generator> (truy cập ngày 2026-07-19).
- [94] Ultralytics, "Object Detection task docs — chú thích phần cứng dùng để benchmark," GitHub / Ultralytics Docs, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ultralytics/ultralytics/blob/main/docs/en/tasks/detect.md> (truy cập ngày 2026-07-19).
- [95] Sutikno, A. Sugiharto, R. Kusumaningrum, "Enhanced Automatic License Plate Detection and Recognition using CLAHE and YOLOv11 for Seat Belt Compliance Detection," *Engineering, Technology & Applied Science Research*, q. 15, s. 1, tr. 20271–20278, 2025. doi: 10.48084/etasr.9629.
- [96] Ultralytics, "Model Export with Ultralytics YOLO — danh sách hơn 20 định dạng xuất và tham số," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/modes/export/> (truy cập ngày 2026-07-19).
- [97] Ultralytics, "Model Benchmarking with Ultralytics YOLO — che do benchmark tu dong tren CPU," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/modes/benchmark/> (truy cập ngày 2026-07-19).

- [98] Ultralytics, "Ultralytics Licensing (AGPL-3.0 va Enterprise)," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://www.ultralytics.com/license> (truy cập ngày 2026-07-19).
- [99] "Comparative study of YOLO models for Oman car plate detection," *ScienceDirect*, 2026. [Trực tuyến]. Địa chỉ: <https://www.sciencedirect.com/science/article/pii/S277318632600068X> (truy cập ngày 2026-07-19).
- [100] "Optimized YOLOv8 for Automatic License Plate Recognition on Resource Constrained Devices," *Engineering, Technology & Applied Science Research*, q. 15, s. 2, 2025. doi: 10.48084/etasr.9983.
- [101] Tesseract OCR project, "Tesseract Release Notes," Tesseract OCR, 2026. [Trực tuyến]. Địa chỉ: <https://tesseract-ocr.github.io/tessdoc/ReleaseNotes.html> (truy cập ngày 2026-07-19).
- [102] PaddlePaddle, "Text Detection Module — PaddleX Documentation," PaddleX, 2026. [Trực tuyến]. Địa chỉ: https://paddlepaddle.github.io/PaddleX/3.4/en/module_usage/tutorials/ocr_modules/text_detection.html (truy cập ngày 2026-07-19).
- [103] PaddlePaddle, "Text Recognition Module — PaddleOCR/PaddleX Documentation," PaddleOCR / PaddleX, 2026. [Trực tuyến]. Địa chỉ: http://www.paddleocr.ai/main/en/version3.x/module_usage/text_recognition.html (truy cập ngày 2026-07-19).
- [104] A. Rosebrock, "Tesseract Page Segmentation Modes (PSMs) Explained: How to Improve Your OCR Accuracy," PyImageSearch, 2021. [Trực tuyến]. Địa chỉ: <https://pyimagesearch.com/2021/11/15/tesseract-page-segmentation-modes-psms-explained-how-to-improve-your-ocr-accuracy/> (truy cập ngày 2026-07-19).
- [105] PaddleOCR community, "Is there any option to whitelist or blacklist character in PaddleOCR — Discussion 7515," GitHub, 2022. [Trực tuyến]. Địa chỉ: <https://github.com/PaddlePaddle/PaddleOCR/discussions/7515> (truy cập ngày 2026-07-19).
- [106] Jaided AI, "EasyOCR API Documentation," Jaided AI, 2025. [Trực tuyến]. Địa chỉ: <https://www.jaided.ai/easyocr/documentation/> (truy cập ngày 2026-07-19).
- [107] A. Rosebrock, "Whitelisting and Blacklisting Characters with Tesseract and Python," PyImageSearch, 2021. [Trực tuyến]. Địa chỉ: <https://pyimagesearch.com/2021/09/06/whitelisting-and-blacklisting-characters-with-tesseract-and-python/> (truy cập ngày 2026-07-19).
- [108] OpenMMLab, "open-mmlab/mmodocr — GitHub," GitHub, 2023. [Trực tuyến]. Địa chỉ: <https://github.com/open-mmlab/mmodocr> (truy cập ngày 2026-07-19).

- [109] A. Kandratavicius, "ankandrew/fast-plate-ocr — GitHub," GitHub, 2026. [Trực tuyến]. Địa chỉ: <https://github.com/ankandrew/fast-plate-ocr> (truy cập ngày 2026-07-19).
- [110] Mindee, "Choosing the right model — docTR documentation," Mindee, 2026. [Trực tuyến]. Địa chỉ: https://mindee.github.io/doctr/latest/using_doctr/using_models.html (truy cập ngày 2026-07-19).
- [111] "Enhancing automated vehicle identification by integrating YOLO v8 and OCR techniques for high-precision license plate detection and recognition," *Scientific Reports*, 2024. doi: 10.1038/s41598-024-65272-1.
- [112] PaddlePaddle Team, "PP-OCRv6: From 1.5M to 34.5M Parameters, Surpassing Billion-Scale VLMs on OCR Tasks," *arXiv preprint arXiv:2606.13108*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2606.13108v1> (truy cập ngày 2026-07-19).
- [113] Y. Du, "PP-OCR: A Practical Ultra Lightweight OCR System," *arXiv preprint arXiv:2009.09941*, 2020. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2009.09941> (truy cập ngày 2026-07-19).
- [114] Y. Du, "PP-OCRv2: Bag of Tricks for Ultra Lightweight OCR System," *arXiv preprint arXiv:2109.03144*, 2021. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2109.03144> (truy cập ngày 2026-07-19).
- [115] Reddy, Shruthi, "License Plate Detection using YOLO v8 and Performance Evaluation of EasyOCR, PaddleOCR and Tesseract," trong *IEEE International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 2024. [Trực tuyến]. Địa chỉ: <https://ieeexplore.ieee.org/document/10725878/> (truy cập ngày 2026-07-19).
- [116] Intel OpenVINO, "Precision Control — OpenVINO documentation (FP16 chuyển về FP32 trên CPU, bf16/AMX)," Intel, 2025. [Trực tuyến]. Địa chỉ: <https://docs.openvino.ai/2025/openvino-workflow/running-inference/optimize-inference/precision-control.html> (truy cập ngày 2026-07-19).
- [117] Microsoft ONNX Runtime, "Thread management — ONNX Runtime Performance Tuning (intra/inter op threads, spinning, NUMA)," Microsoft, 2025. [Trực tuyến]. Địa chỉ: <https://onnxruntime.ai/docs/performance/tune-performance/threading.html> (truy cập ngày 2026-07-19).
- [118] PaddlePaddle, "Introduction to PP-OCRv5 — PaddleOCR Documentation," PaddleOCR, 2026. [Trực tuyến]. Địa chỉ: <http://www.paddleocr.ai/main/en/version3.x/algorithm/PP-OCRv5/PP-OCRv5.html> (truy cập ngày 2026-07-19).
- [119] Ultralytics, "Model Evaluation Insights — hướng dẫn vật thể nhỏ, imgsz, rect, SAH tiling," Ultralytics, 2026. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/guides/model-evaluation-insights/> (truy cập ngày 2026-07-19).

[120] Intel OpenVINO, "Performance Hints and Thread Scheduling — OpenVINO CPU Device (LATENCY hint, hyper-threading, inference_num_threads)," Intel, 2024. [Trực tuyến]. Địa chỉ: <https://docs.openvino.ai/2024/openvino-workflow/running-inference/inference-devices-and-modes/cpu-device/performance-hint-and-thread-scheduling.html> (truy cập ngày 2026-07-19).

[121] "TransLPRNet: Lite Vision-Language Network for Single/Dual-line Chinese License Plate Recognition," *arXiv preprint arXiv:2507.17335*, 2025. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2507.17335> (truy cập ngày 2026-07-19).

[122] "Advancing Multinational License Plate Recognition Through Synthetic and Real Data Fusion: A Comprehensive Evaluation," *arXiv preprint arXiv:2601.07671*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/pdf/2601.07671> (truy cập ngày 2026-07-19).

[123] Microsoft ONNX Runtime, "Quantize ONNX models — ONNX Runtime (dynamic vs static, VNNI/AVX512, cảnh báo phần cứng cũ)," Microsoft, 2025. [Trực tuyến]. Địa chỉ: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html> (truy cập ngày 2026-07-19).

PHỤ LỤC

Phụ lục cung cấp các thông tin chi tiết được nhắc tới trong thân đồ án nhưng không đưa vào thân bài để giữ mạch đọc: quy mô và tổ chức mã nguồn, cấu hình huấn luyện đầy đủ, xuất xứ và giấy phép của từng bộ dữ liệu, hướng dẫn cài đặt, kết quả kiểm thử và đặc tả giao diện lập trình.

Mọi số liệu trong phụ lục lấy trực tiếp từ kho mã nguồn và các tệp kết quả đã được lưu, không có số nào nhập tay.

Phụ lục A. Mã nguồn

A.1. Quy mô

Số liệu đếm trên kho mã tại thời điểm nộp, không tính thư mục phụ thuộc (node_modules, môi trường ảo), dữ liệu ảnh, trọng số mô hình và các tệp sinh tự động.

Bảng A.1. Quy mô mã nguồn theo thành phần

Thành phần	Ngôn ngữ	Số tệp	Số dòng
ai/ — tầng trí tuệ nhân tạo	Python	31	16.949
scripts/ — công cụ dựng dữ liệu, đo đạc, xuất tài liệu	Python	32	17.395
tests/ — kiểm thử tự động	Python	23	9.246
backend/ — dịch vụ web và truy cập dữ liệu	Python	29	9.992
frontend/ — giao diện người dùng	TypeScript / TSX / CSS	52	10.471
ai/ — cấu hình huấn luyện và bộ dữ liệu	YAML	3	328
deployment/, docker-compose.yml	Dockerfile / YAML	5	780
Tổng		178	65.805

Tỷ trọng đáng chú ý: phần **kiểm thử và công cụ đo đạc** (tests/ + scripts/) chiếm **26.641 dòng, tức 40,5%** toàn bộ mã nguồn — nhiều hơn cả tầng AI. Đây là hệ quả trực tiếp của nguyên tắc trình bày đã nêu ở mục 5.1.2: mỗi con số công bố trong Chương 5 phải sinh ra được bằng một lệnh chạy lại được.

A.2. Tổ chức thư mục

ai/	
inference/	pipeline suy luận – detector, recognizer, hậu xử lý, hai dò
ng	
evaluation/	đo độ chính xác, hiệu năng, phân tích lỗi, kiểm rò rỉ

training/	cấu hình và notebook huấn luyện
backend/	
api/	các route FastAPI
services/	tầng nghiệp vụ – điều phối, không chứa logic AI
repositories/	truy cập cơ sở dữ liệu
models/	mô hình dữ liệu SQLAlchemy
migrations/	Alembic
frontend/	
src/pages/	ba trang: nhận dạng ảnh, nhận dạng video, lịch sử
src/components/	thành phần dùng chung
src/api/	tầng gọi API và ánh xạ kiểu dữ liệu
tests/	kiểm thử đơn vị, tích hợp, kiến trúc
scripts/	dựng bộ dữ liệu, đo đạc, dựng quyền và slide
deployment/	Dockerfile, nginx, entrypoint
models/	trọng số đã huấn luyện
docs/	tài liệu, báo cáo, quyền đồ án

Ranh giới quan trọng nhất trong cây thư mục: **ai/ không được import bất cứ thứ gì từ backend/**. Ràng buộc này là NFR-M1 và được canh giữ tự động bởi `tests/test_architecture.py` — không phải bằng quy ước mà bằng một test sẽ fail nếu ai đó vi phạm. Lý do và hệ quả trình bày ở mục 4.2.1 và 5.5.1.

Phụ lục B. Siêu tham số huấn luyện

Cấu hình đầy đủ của lượt huấn luyện sinh ra `models/best.pt` — mô hình được dùng cho mọi số liệu công bố trong Chương 5. Nguồn: `runs/final-640-v3/args.yaml`.

Bảng B.1. Siêu tham số huấn luyện YOLO11n

Nhóm	Tham số	Giá trị	Ghi chú
Mô hình	model	yolo11n.pt	Khởi tạo từ trọng số tiền huấn luyện COCO
	Số tham số	2.590.035	Biến thể nano — do ràng buộc CPU
Dữ liệu	data	yolo_v3/data.yaml	Split v3
	imgsz	640	Đúng độ phân giải mà NFR-A1/A2 đặt chỉ tiêu
	fraction	1,0	Dùng toàn bộ dữ liệu
Lịch huấn luyện	epochs	20	
	patience	20	Dừng sớm không kích hoạt
	batch	8	Giới hạn bởi RAM và tốc độ CPU

Nhóm	Tham số	Giá trị	Ghi chú
	close_mosaic	10	Tắt mosaic trong 10 epoch cuối
Tối ưu hoá	optimizer	AdamW	
	lr0 / lr1	0,001 / 0,01	Tốc độ học đầu và hệ số cuối
	cos_lr	true	Lịch cosine
	momentum	0,937	
	weight_decay	0,0005	
	warmup_epochs	3,0	
Trọng số mất mát	box / cls / dfl	8,0 / 0,5 / 1,5	
Tăng cường dữ liệu	hsv_h / hsv_s / hsv_v	0,015 / 0,7 / 0,4	
Thiết bị	device	cpu	Không có GPU CUDA (ràng buộc CON-02)

Hai giá trị đáng giải thích thêm. batch = 8 không phải lựa chọn tối ưu mà là giới hạn phần cứng; epochs = 20 là con số bị ngân sách thời gian CPU quyết định chứ không phải điểm hội tụ — chi phí và hệ quả của cả hai trình bày ở mục 4.5.1 và 3.6.

Cấu hình tinh chỉnh bộ nhận dạng ký tự (30 epoch, 6.672 mẫu, bộ ký tự 36) trình bày tại mục 4.5.3 cùng kết quả đo bốn cấu hình. **Bản giao hàng không dùng mô hình tinh chỉnh** — lý do ở cùng mục.

Phụ lục C. Bộ dữ liệu

C.1. Nguồn và giấy phép — nhánh phát hiện biển số

Bảng C.1. Bảy bộ dữ liệu đã hợp nhất, kèm giấy phép và số ảnh còn lại

#	Bộ (slug)	Nguồn	Giấy phép	Vào gộp	Còn lại
1	roboflow_school_fuh ih	Roboflow school-fuhih/vietnamese- license-plate-tptd0v1	CC BY 4.0	8.35 7	6.86 8
2	hf_vn_plates_segmen t	HuggingFace hoanglvuit/Vietnam_License_Plate_Se gment_Datasets	⚠ chưa xác nhận	4.57 8	4.37 5
3	roboflow_traffic_ca	Roboflow traffic-camera/vietnam-	CC	3.84	3.16

#	Bộ (slug)	Nguồn	Giấy phép	Vào gộp	Còn lại
	mera	license-plate-hayn8 v4	BY 4.0	3	2
4	roboflow_eric_nguyen	Roboflow eric-nguyen-knfxn/vietnam-license-plate-curhr v1	CC BY 4.0	840	353
5	roboflow_demo_tracking	Roboflow demo-tracking/license-plate-vietnam-car v2	CC BY 4.0	236	235
6	roboflow_cuong_ta	Roboflow cuong-ta-ulxex/vietnamese-car-license-plate v1	Public Domain (người đăng tự khai)	8.25 4	140
7	roboflow_tran_ngoc_xuan_tin	Roboflow tran-ngoc-xuan-tin-k15-hcm-dpuid/vietnam-license-plate-h8t3n v1	CC BY 4.0	1.00 5	0
Tổng				27.1 13	15.1 33

Ghi công theo giấy phép. Năm bộ ở trên phát hành theo **CC BY 4.0**, bắt buộc ghi công tác giả — bảng này chính là phần ghi công đó. Một bộ được người đăng tự khai **Public Domain**, nhưng đồ án **không khẳng định** đó là Public Domain thật vì ảnh nguồn có dấu hiệu là ảnh báo chí. Một bộ trên HuggingFace **chưa xác nhận được giấy phép**; nó đóng góp 28,91% corpus nên đây là rủi ro pháp lý phải nêu chứ không phải chi tiết bỏ qua được.

Bộ thứ bảy còn lại 0 ảnh sau khử trùng lặp — toàn bộ 1.005 ảnh của nó trùng với ảnh đã có ở các bộ khác. Con số "hợp nhất từ 7 bộ" vì vậy phải đọc là **6 nguồn nguyên tố**, và điều này được nêu nhất quán ở mục 4.4.2 và 6.3.1.

Phụ lục D. Hướng dẫn cài đặt và chạy

D.1. Yêu cầu

Hạng mục	Yêu cầu
Hệ điều hành	Windows 10/11, macOS hoặc Linux
Docker	Docker Engine 24+ và Docker Compose v2
Bộ nhớ	Tối thiểu 4 GB RAM trống

Hạng mục	Yêu cầu
Đĩa	Khoảng 6 GB cho image và dữ liệu
GPU	Không cần — toàn hệ thống chạy trên CPU

D.2. Chạy bằng Docker Compose (khuyến nghị)

```
git clone <địa chỉ kho mã>
cd vn-license-plate-recognition
docker compose up -d --build
```

Sau khi các container khởi động, mở trình duyệt tại:

Địa chỉ	Nội dung
http://localhost:5173	Giao diện người dùng
http://localhost:8000/docs	Tài liệu API (Swagger UI, tự sinh)
http://localhost:8000/health	Trạng thái hệ thống

Kiểm tra hệ thống đã nạp được mô hình:

```
curl http://localhost:8000/health
```

Trường `model_loaded` phải trả về `true`. Nếu trả về `false`, hệ thống vẫn chạy nhưng mọi yêu cầu nhận dạng sẽ trả lỗi thay vì trả kết quả bìa — cơ chế `UnavailablePipeline`, trình bày ở mục 4.7.4.

D.3. Chạy trực tiếp không dùng Docker

```
# Tầng AI và backend
python -m venv backend/.venv
backend/.venv/Scripts/pip install -r backend/requirements.txt
backend/.venv/Scripts/alembic upgrade head
backend/.venv/Scripts/uvicorn backend.main:app --port 8000
```

```
# Giao diện, ở một cửa sổ lệnh khác
cd frontend && npm install && npm run dev
```

Lưu ý về môi trường ảo. Đồ án dùng **ba môi trường ảo Python tách biệt**, không phải một. Lý do bắt buộc phải tách — xung đột phiên bản giữa hai framework học sâu — trình bày ở mục 4.3.2. Gộp chúng lại sẽ hỏng.

D.4. Biến môi trường đáng chú ý

Biến	Mặc định	Tác dụng
ALPR_MODEL_PATH	models/best.pt	Đường dẫn trọng số bộ phát hiện
ALPR_RECTIFY_ENABLED	true	Bật bước nắn hình biến nghiêng
ALPR_SR_RETRY_ENABLED	false	Bậc siêu phân giải — tắt mặc định , xem mục 5.5.7

Biến	Mặc định	Tác dụng
ALPR_OCR_SKIP_DETECTION	false	Bỏ bước phát hiện chữ — tắt mặc định , xem mục 5.6.6
ALPR_OCR_REC_MODEL_DIR	(rỗng)	Thư mục mô hình nhận dạng tinh chỉnh; để rỗng là dùng mô hình gốc

Chi tiết đầy đủ về triển khai — kiến trúc mạng Docker, các volume, cách xử lý sự cố thường gặp và lưu ý dung lượng image — ở deployment/README.md.

Phụ lục E. Kết quả kiểm thử

E.1. Tổng hợp

Bảng E.1. Kết quả chạy bộ kiểm thử tự động

Hạng mục	Kết quả
Số test thu thập	1.001
Đạt	1.000
xfail (dự kiến hỏng, có ghi lý do)	1
Fail	0
Skip	0
Ngày chạy	02/08/2026

E.2. Phân nhóm

Nhóm	Kiểm chứng điều gì
Kiểm thử đơn vị	Bộ luật hậu xử lý theo vị trí, phân loại layout, chuẩn hoá chuỗi, quy tắc hiển thị
Kiểm thử tích hợp	Toàn bộ 10 endpoint qua HTTP thật, kèm cơ sở dữ liệu thật và migration
Kiểm thử kiến trúc	Ranh giới ai/ không import backend/ (NFR-M1) — fail nếu ai đó vi phạm
Kiểm thử hồi quy	Các ca lỗi đã từng xảy ra, mỗi ca một test để không tái diễn

Ý nghĩa của con số 0 fail cần được đọc đúng. Nó nói rằng hệ thống làm đúng những gì bộ kiểm thử kiểm; nó **không** nói rằng hệ thống đạt mọi chỉ tiêu. Ba chỉ tiêu phi chức năng hiện không đạt (NFR-A5, A6, A7) và một chỉ tiêu trượt sà (NFR-P2) — bảng đối chiếu đầy đủ ở mục 5.7 và phân tích ở mục 5.9.2.

Báo cáo kiểm thử chi tiết theo từng nhóm: docs/reports/07-testing-report.md.

Phụ lục F. Giao diện lập trình và cấu hình triển khai

F.1. Danh sách endpoint

Bảng F.1. Mười endpoint của hệ thống

#	Phương thức	Đường dẫn	Chức năng
1	POST	/api/detect/image	Nhận dạng biến số từ một ảnh tĩnh
2	POST	/api/detect/video	Tạo tác vụ nhận dạng trên video, xử lý nền
3	POST	/api/detect/frame	Nhận dạng một khung hình — dùng cho chế độ thời gian thực
4	GET	/api/jobs/{job_id}	Trạng thái và tiến độ của một tác vụ video
5	GET	/api/history	Danh sách lịch sử, có tìm kiếm, lọc, phân trang
6	GET	/api/history/{detection_id}	Chi tiết một lần nhận dạng
7	GET	/api/history/export	Xuất lịch sử theo bộ lọc hiện hành
8	DELETE	/api/history/{detection_id}	Xoá một bản ghi
9	GET	/api/statistics	Số liệu thống kê tổng hợp theo cửa sổ thời gian
10	GET	/health	Trạng thái hệ thống và tình trạng nạp mô hình

Đặc tả đầy đủ — kiểu dữ liệu đầu vào, cấu trúc đầu ra, mã trạng thái và các quyết định thiết kế API — ở mục 4.7.3. Tài liệu OpenAPI do FastAPI **tự sinh** tại /docs và /openapi.json, nên nó không bao giờ lệch với mã nguồn.

F.2. Cấu hình Docker Compose

Bảng F.2. Thành phần trong docker-compose.yml

Thành phần	Loại	Vai trò
backend	dịch vụ	FastAPI + uvicorn, chạy pipeline AI trên CPU
frontend	dịch vụ	nginx:alpine — phục vụ tệp tĩnh và reverse proxy sang backend
alpr-net	mạng	Mạng nội bộ giữa hai dịch vụ
alpr-data	volume	Cơ sở dữ liệu SQLite — dữ liệu sống qua lần khởi động lại
alpr-model-cache	volume	Bộ nhớ đệm trọng số PaddleOCR — tránh tải lại mỗi lần dựng

Ngoài hai volume có tên ở trên, thư mục `./storage` (ảnh và video đã tải lên) và `./models` (trọng số bộ phát hiện, gắn **chỉ đọc**) được gắn trực tiếp từ máy chủ.

Bộ ba tệp triển khai: `deployment/docker/Dockerfile.backend` (build hai giai đoạn), `Dockerfile.frontend` (build rồi phục vụ tĩnh) và `nginx.conf`. Phân tích từng tệp ở mục 4.9.

Phụ lục H. Đặc tả yêu cầu và thiết kế dữ liệu

Bốn mục dưới đây là **tài liệu tra cứu**, không phải mạch lập luận: đặc tả từng use case, bảng 34 yêu cầu chức năng, bảng chỉ tiêu phi chức năng, và đặc tả từng trường của cơ sở dữ liệu. Chương 4 nêu quyết định thiết kế và lý do; phần liệt kê đầy đủ để ở đây.

H.2. Bảng 34 yêu cầu chức năng

Đồ án đặc tả **34 yêu cầu chức năng** trong **6 nhóm**, mỗi yêu cầu có mã, mức MoSCoW và một tiêu chí chấp nhận kiểm chứng được. Phân bố: FR-1 (ảnh tĩnh) **7 Must**; FR-2 (video) **5 Must + 1 Should**; FR-3 (thời gian thực, tầng API) **3 Must + 2 Won't**; FR-4 (thống kê – lịch sử – tra cứu) **4 Must + 1 Should + 1 Could + 2 Won't**; FR-5 (quản lý dữ liệu) **2 Should + 2 Could**; FR-6 (hệ thống, vận hành) **2 Must + 2 Should**. Tổng **21 Must, 6 Should, 3 Could, 4 Won't = 34**.

FR-1: tiếp nhận, kiểm tra hợp lệ, phát hiện *tất cả* vùng biển, cắt và nhận dạng, hậu xử lý, lưu kết quả, hiển thị có bounding box. FR-1.5 quy định lưu **cả chuỗi OCR thô lẫn chuỗi đã sửa** — điều kiện cần để đo đóng góp hậu xử lý ở Chương 5 (4.7.2b). **FR-2:** thêm trích khung theo bước nhảy, **gộp trùng** (FR-2.4 — thiếu nó một video 30 giây sinh hàng nghìn bản ghi về cùng vài chiếc xe, phá hỏng thống kê FR-4), kết xuất video gắn nhãn; Should duy nhất là tiến độ phần trăm và huỷ tác vụ. **FR-3:** theo quyết định 2026-07-20, hai yêu cầu thuần giao diện FR-3.1, FR-3.4 chuyển **M → W**; FR-3.2/3.3/3.5 vẫn Must, kiểm chứng ở tầng API. **FR-4:** chỉ số tổng hợp (FR-4.1), biểu đồ theo thời gian (FR-4.2), danh sách phân trang, tìm kiếm khớp một phần, lọc, chi tiết, tải ảnh, sắp xếp; **FR-4.3 → 4.8 không đổi**. **FR-5:** xóa bản ghi kèm tệp, xuất CSV/JSON (CSV phải UTF-8 **có BOM** kéo Excel hiển thị sai tiếng Việt), dọn tệp mờ côi, xóa hàng loạt. **FR-6:** health check báo trạng thái mô hình và CSDL; log có cấu trúc; thông báo lỗi thân thiện không lộ stack trace; cấu hình qua biến môi trường.

Bốn yêu cầu mức Won't và hai đợt thu gọn phạm vi ngày 2026-07-20

Cả bốn yêu cầu Won't đều **thuần giao diện**, chuyển mức trong cùng ngày qua hai đợt: đợt 1 gỡ trang Webcam (FR-3.1, FR-3.4 **M → W**; năng lực còn ở POST `/api/detect/frame`); đợt 2 gỡ trang Tổng quan (**FR-4.1 M → W**, FR-4.2 **S → W**; năng lực còn ở GET `/api/statistics` và GET `/health`).

Phải nói thẳng: FR-4.1 là yêu cầu mức **Must** đầu tiên — và duy nhất — bị đưa ra khỏi phạm vi trong toàn bộ đồ án. Con số đếm vì vậy giảm từ 22/7/3/2 xuống 21/6/3/4, và mục 6.3 ghi nhận đây là **hạn chế thật**. Điều **không** thay đổi: cả hai

đợt chỉ gỡ **màn hình hiển thị**, không gỡ **năng lực hệ thống** — các endpoint vẫn phục vụ, vẫn trong tài liệu OpenAPI, vẫn có kiểm thử tích hợp (tests/integration/test_api_statistics.py, test_api_health.py), thiết kế API ở 4.7.3 giữ nguyên không sửa một dòng — bằng chứng thực tế cho nguyên tắc tách tầng ở 4.2. Đánh đổi đo được của đợt 2: gỡ recharts làm gói tải về giảm từ ~730 KB xuống **328,8 KB** (–55%).

Ma trận truy vết: mỗi nhóm truy vết tới giai đoạn cài đặt và hình thức kiểm chứng (FR-1: unit + integration; FR-2: integration + performance; FR-3: performance ở tầng API; FR-4: integration + UI test cho FR-4.3→4.8; FR-5: unit; FR-6: smoke + stress). Kết quả ở Chương 5.

H.3. Bảng chỉ tiêu phi chức năng

Bảy nhóm: hiệu năng (NFR-P), độ chính xác (NFR-A), tin cậy (NFR-R), khả dụng (NFR-U), bảo trì (NFR-M), bảo mật (NFR-S), tương thích – triển khai (NFR-C), mở rộng (NFR-SC).

a) Nguyên tắc nền tảng: mọi chỉ tiêu hiệu năng đều là chỉ tiêu CPU

Toàn bộ chỉ tiêu hiệu năng của đồ án là chỉ tiêu đo trên CPU.

Máy thực hiện chạy Windows 11, Python 3.13, **không có GPU CUDA** (Intel UHD 770 tích hợp, PyTorch không dùng được để tăng tốc). Huấn luyện trên GPU miễn phí Colab/Kaggle, nhưng **suy luận và buổi bảo vệ chạy trên CPU máy cá nhân**. Đây là **ràng buộc thiết kế**, không phải hạn chế tạm thời, vì bốn lẽ: nó cố định trong toàn bộ vòng đời và tại chính buổi bảo vệ; nó đổi *bậc độ lớn* của độ trễ (ở 20 ms/khung, video đồng bộ và webcam xử lý mọi khung là hợp lý — ở mốc thực tế 400 ms cả hai bất khả thi, trực tiếp sinh ra hai quyết định kiến trúc: video bất đồng bộ AD-02 và webcam bỏ khung hàng đợi một khe); nó chỉ phối chọn biến thể mô hình (n/s/m), biến thể OCR (mobile/server), kích thước ảnh và **backend suy luận** — benchmark chính thức trên CPU i7-13700H cho thấy YOLOv8n qua ONNX Runtime nhanh hơn PyTorch khoảng **3,73 lần** (104,61 → 28,02 ms) [18], lợi ích lớn nhất đúng ở phân khúc mô hình nhỏ [117]; và nó buộc phương pháp công bố chặt hơn — quy tắc CON-06: **mọi số liệu hiệu năng phải kèm model CPU, số luồng, kích thước ảnh, backend suy luận và cỡ mẫu đo**. Các chỉ tiêu độ trễ vì vậy "rộng rãi" hơn văn liệu quốc tế đo trên GPU — đó là trung thực về điều kiện đo, không phải dễ dãi.

Cảnh báo trích dẫn. Bảng benchmark nguồn có cột mAP nhưng đo trên tập coco8 chỉ **8 ảnh**, không có ý nghĩa thống kê; đồ án chỉ dùng cột thời gian và cố ý lược bỏ cột độ chính xác.

b) Chỉ tiêu định lượng nhóm hiệu năng và nhóm độ chính xác

Bảng 4.1. Chỉ tiêu phi chức năng định lượng: hiệu năng (NFR-P) và độ chính xác (NFR-A)

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
NFR-	Độ trễ toàn trình một ảnh (p95)	≤ 800 ms	≤ 1500 ms

Mã	Chỉ tiêu	Mục tiêu	Ngưỡng tối thiểu
P1			
NFR-P2	Tốc độ khung hình thời gian thực (webcam — đo ở tầng API)	≥ 5 FPS hiệu dụng	≥ 3 FPS
NFR-P3	Tốc độ xử lý video	≥ 0,3× thời gian thực	≥ 0,15×
NFR-P4	Thời gian nạp mô hình khi khởi động	≤ 15 giây	≤ 30 giây
NFR-P5	Overhead của tầng API (không tính suy luận)	≤ 50 ms	≤ 100 ms
NFR-P6	Thời gian truy vấn lịch sử (10.000 bản ghi)	≤ 500 ms	≤ 1000 ms
NFR-P7	Bộ nhớ thường trú của backend	≤ 2 GB	≤ 4 GB
NFR-A1	mAP@0.5 của bộ phát hiện	≥ 0,90	≥ 0,85
NFR-A2	mAP@0.5:0.95 của bộ phát hiện	≥ 0,65	≥ 0,55
NFR-A3	Precision / Recall phát hiện	≥ 0,92 / ≥ 0,90	≥ 0,88 / ≥ 0,85
NFR-A4	Độ chính xác OCR mức ký tự (1 – CER)	≥ 0,95	≥ 0,92
NFR-A5	Độ chính xác biến đầy đủ trước hậu xử lý	≥ 0,85	≥ 0,80
NFR-A6	Độ chính xác biến đầy đủ sau hậu xử lý	≥ 0,90	≥ 0,85
NFR-A7	Độ chính xác toàn trình (ảnh vào → biến đúng)	≥ 0,88	≥ 0,82

Phương pháp đo NFR-P: P1 trên 100 ảnh test, báo p50/p95/p99; P2 đo liên tục 60 giây; P3 bằng video 60 giây phải xong trong ≤ 200 giây; P4 từ khởi động đến khi /health sẵn sàng; P5 là hiệu tổng thời gian request trừ thời gian pipeline; P6 có phân trang và bộ lọc trên 10.000 bản ghi; P7 theo dõi RSS khi chạy tải liên tục.

NFR-P1 xuất phát từ **phân rã ngân sách độ trễ**: giải mã ~50 ms; phát hiện @640 px ~150 ms; cắt ~30 ms; OCR mỗi biến ~120 ms; hậu xử lý < 5 ms; ghi CSDL ~50 ms — **tổng ~405 ms cho ảnh một biến**; ngân sách 800 ms để dự phòng ảnh nhiều biến và biến động tải. Đây là **ước lượng thiết kế, không phải kết quả đo** (số đo ở Chương 5). Ngân sách lập cho runtime mặc định đã chốt ở mục 3.4 là **ONNX Runtime** — điểm đã đổi so với AD-05 sơ bộ.

Nếu vượt ngưỡng, thứ tự giảm tải định trước: (1) INT8 OpenVINO; (2) giảm ảnh xuống 480 px; (3) biến thể OCR nhẹ hơn — chỉ hạ chỉ tiêu **sau khi** thử hết ba phương án.

Cặp NFR-A5/A6 đặt **tách bạch** có chủ đích: hiệu số giữa chúng là đóng góp định lượng của khối hậu xử lý — đo được nhờ quyết định lưu cả chuỗi thô lẫn chuỗi sửa ở tầng dữ liệu (4.7.2b). Bổ sung: **NFR-A8** — báo cáo độ chính xác **tách riêng biến một dòng và hai dòng**, căn cứ số liệu 94,3% / 45,7% **do trên bộ RodoSol-ALPR (Brazil)** đã dẫn ở 4.1.1, vì một con số tổng thể sẽ che giấu đúng điểm gây cần phân tích; **NFR-A9** — báo cáo theo điều kiện ảnh nếu bộ dữ liệu có nhãn phù hợp.

c) Các nhóm yêu cầu phi chức năng còn lại

NFR-R: không sập với đầu vào hỏng/độc hại (100% lỗi bị bắt); ảnh không biến trả rỗng hợp lệ HTTP 200; video thất bại không để lại rác; tỉ lệ thành công chạy liên tục một giờ $\geq 99\%$; CSDL sống sót khởi động lại. **NFR-U:** lượt nhận dạng đầu tiên ≤ 3 nhấp chuột, không cần tài liệu; thao tác > 500 ms có phản hồi trực quan; thông báo lỗi tiếng Việt nêu nguyên nhân và cách khắc phục; dùng được từ 1366×768; tương phản WCAG AA $\geq 4,5:1$. **NFR-M:** mã AI tách hoàn toàn khỏi mã API (M1); bao phủ test tầng nghiệp vụ $\geq 70\%$ (M2); type hint + docstring (M3); không hard-code đường dẫn (M4); thay bộ OCR không sửa tầng API (M5); lint tự động (M6) — M1 và M5 là **yêu cầu kiến trúc**, lý do tồn tại của tầng AI độc lập (4.2). **NFR-S:** kiểm tra magic bytes; chống path traversal bằng tên tệp UUID; giới hạn kích thước phía máy chủ; CORS không ký tự đại diện; không log dữ liệu nhạy cảm; truy vấn tham số hoá qua ORM. **NFR-C:** chạy Windows/Linux/macOS qua Docker một lệnh; **không cần GPU là chế độ mặc định**; Chrome/Edge/Firefox; cài từ máy sạch ≤ 15 phút. **NFR-SC:** ổn định ≥ 5 yêu cầu đồng thời; không suy giảm ở 100.000 bản ghi; video nền không chặn yêu cầu khác.

Giới hạn đã biết cần công bố. SQLite chỉ cho **một tiến trình ghi tại một thời điểm** — chấp nhận được ở quy mô đồ án, nhưng phải nêu trong phần Hạn chế kèm hướng khắc phục (PostgreSQL) nếu triển khai thực tế.

Phụ lục O. Tệp cấu hình gốc, báo cáo đo và mã nguồn

Phụ lục này giữ **hiện vật thô** — thứ cần để tái lập chứ không cần để đọc hiểu. Phụ lục B trình bày siêu tham số dưới dạng bảng đã biên tập; mục O.1 dưới đây là **nguyên văn tệp máy sinh**, vì một bảng biên tập lại không thay được tệp gốc khi có người muốn chạy lại đúng lượt huấn luyện ấy.

O.2. Danh mục báo cáo đo dạng JSON

Toàn bộ số liệu công bố trong quyền sinh ra từ các tệp dưới đây, nằm ở docs/reports/.
Danh mục **không chép nội dung tệp**: gộp lại chúng dài hàng chục nghìn dòng, chép vào thì

quyển phình mà vẫn không ai đọc. Thay vào đó mỗi dòng ghi **câu hỏi tệp đó trả lời** và **mục nào trong quyển dùng nó**, để một con số bất kỳ đều truy ngược được về tệp sinh ra nó.

Bảng O.1. Báo cáo đo dạng JSON và mục sử dụng

Tệp trong docs/reports/	Trả lời câu hỏi gì	Dùng ở mục
03-evaluation-ch5-best-test.json	mAP, Precision, Recall của best.pt trên tập test v3	5.4.1
07-leak-check-t10.json	Số cặp ảnh gần trùng train↔test theo từng ngưỡng Hamming	5.3.2
17-plate-type-audit.json	Phân bố màu nền của 2.801 mẫu có nhãn chuỗi — căn cứ cảnh báo 97,68% biển trắng	5.3, 6.2
04-ocr-accuracy.json	A4–A7 lượt đo cơ sở	5.5
16-ocr-accuracy-rescued.json	A4–A7 sau khi thêm bước cứu dòng trên	5.5.6
28-ocr-accuracy-finetuned.json	A4–A7 của bộ nhận dạng đã tinh chỉnh	4.5.3
29-reconly-ablation.json	Bốn cấu hình det+rec ↔ chỉ-rec, hai model	4.5.3, 5.6.6
15-two-line-ab.json	A/B ghép-rời-đọc ↔ đọc-từng-nửa, 200 biển hai dòng	5.5.6
15-two-line-fallback-700.json	A/B bước cứu dòng trên, mẫu 700 biển	5.5.6
15-two-line-fallback.json	A/B bước cứu dòng trên, mẫu 200 biển	5.5.6
15-two-line-rescue-ladder.json	Chi phí và lợi ích từng bậc của bậc thang thử-lại	5.5.7
15-fragment-height-ab.json	Ngưỡng lọc mảnh văn bản theo hình học	4.6.3
19-color-accuracy.json	Độ chính xác bộ nhận màu nền trên 1.565 ảnh ngoài hiệu chỉnh	4.6.7
07-benchmark-p1-resolved.json	Độ trễ đầu-cuối p50/p95 và phân rã theo bước	5.6.1, 5.6.2
07-api-overhead.json	Overhead của tầng API so với gọi pipeline trực tiếp	5.6.5
07-stress-load.json	Chịu tải đồng thời và tỉ lệ thành công khi chạy liên tục	5.6.5
07-stress-db.json	Thời gian truy vấn lịch sử trên 10.000 bản ghi	5.6.5
07-benchmark-optimized.json	(chưa chạy) So sánh PyTorch ↔ ONNX Runtime ↔ OpenVINO	5.6.3

Thư mục còn **23 tệp JSON khác** thuộc các lượt đo trung gian đã bị lượt sau thay thế; chúng được giữ lại trong kho để đối chiếu lịch sử chứ không được trích dẫn trong quyền. Nguyên tắc áp dụng xuyên suốt: **một số liệu chỉ được đưa vào quyền khi tệp sinh ra nó còn trong kho và chạy lại được.**
